END-TO-END PRODUCTION AI SYSTEM REFERENCE

Crop Disease & Pest Classifier

Complete Theory, Definitions, Training & Testing Data Design, Mathematical Formulas, Deep Learning Model Architectures, Frontend & Backend Parameters, and Full Line-by-Line Code Walkthrough

1. Deep Learning & Computer Vision Stack

Python 3.11 • PyTorch 2.x • timm (EfficientNetV2-S, ConvNeXt-Tiny, Swin-Tiny, MobileNetV3) • Albumentations • Optuna

2. Trustworthy ML & Edge Export

Leak-free pHash Splits • Temperature Scaling (ECE) • Energy OOD + PlantGate • Dual-Output ONNX Grad-CAM (<300 ms CPU)

3. Production Backend API

FastAPI • ONNX Runtime (no PyTorch in prod) • Pydantic v2 • SQLAlchemy 2.0 (SQLite / PostgreSQL) • SlowAPI • Pytest

4. Mobile-First Frontend & DevOps

React 18 + TypeScript + Vite • Tailwind CSS • i18next (en/hi/es) • Dark Mode • Docker Compose • GitHub Actions CI

Table of Contents

Chapter 1 — Core Theory, Foundational Definitions & System Architecture — Theory & Stack

Chapter 2 — Training, Validation, Testing & Real-Field Datasets (Leak-Free Design) — Data Engineering

Chapter 3 — Every Mathematical Formula Used in the Project — Mathematics

Chapter 4 — Deep Learning Models Used, Comparison & Need for Each Model — Architectures

Chapter 5 — Every Parameter: Training Configs, Optuna, Calibration & OOD — ML Parameters

Chapter 6 — Backend & Frontend Architecture, Libraries & Every Parameter — Full-Stack

Chapter 7 — Complete Source Code & Line-by-Line Explanations (Data & Models) — Code Part A

Chapter 8 — Complete Source Code & Line-by-Line Explanations (Training, Eval & ONNX) — Code Part B

Chapter 9 — Complete Source Code & Line-by-Line Explanations (Backend & Frontend) — Code Part C

Chapter 1: Core Theory, Definitions & System Architecture

1.1 Project Problem Statement & Goal

Smallholder farmers and agronomists need rapid, reliable, field-ready diagnosis of plant diseases and agricultural pests from a single smartphone photograph. Traditional visual inspection requires domain expertise, while naive deep learning classifiers fail in production for three classic reasons: (1) **data leakage** from near-duplicate images across splits, (2) **overconfident wrong predictions** on blurry or non-plant images, and (3) the **lab-to-field domain shift** when models trained on clean laboratory photos are deployed on messy outdoor farm photos.

This system solves all three problems end-to-end: given an uploaded photo of a leaf, fruit, or pest, it returns the canonical diagnosis (`Crop___Disease`), Top-3 **calibrated** probabilities, a **Grad-CAM visual attention heatmap** showing which pixels drove the decision, a structured **agronomic advice card**, and an explicit **"Unsure / Not a Plant"** rejection whenever confidence is low or the input is Out-of-Distribution (OOD).

1.2 Foundational Definitions

Supervised Deep Learning (Multi-Class Image Classification): Learning a parametric mapping $f_\theta: \mathbb{R}^{3\times H\times W} \to \mathbb{R}^K$ from labeled RGB images (x_i, y_i) where $y_i \in \{0, \ldots, K-1\}$ represents a standardized crop disease, healthy state, or pest class, by minimizing a differentiable loss function using gradient descent.

Transfer Learning & Two-Stage Fine-Tuning: Reusing convolutional/transformer feature extractors pretrained on ImageNet-1k/21k (which already encode edges, textures, veins, and shapes) and adapting them to agricultural pathology in two stages: **Stage 1** freezes the backbone and warms up the random linear head so large random head gradients do not corrupt pretrained filters; **Stage 2** unfreezes the full network with *discriminative learning rates* (smaller LR on backbone, larger LR on head).

Data Leakage & Perceptual Hashing (pHash): Data leakage occurs when visually identical or augmented versions of the same leaf appear in both training and test sets, artificially inflating accuracy. Unlike cryptographic SHA-256 (which changes if a single pixel or JPEG compression changes), **pHash** transforms the image via Discrete Cosine Transform (DCT) into a 64-bit structural fingerprint. Near-duplicates have small *Hamming distance* and are clustered into one atomic group before splitting.

Lab-to-Field Generalization Gap: The empirical drop in accuracy and macro-F1 when a model trained on controlled studio images (e.g., PlantVillage uniform backgrounds) is evaluated on uncontrolled outdoor smartphone photos (PlantDoc with shadows, soil, overlapping leaves, and glare).

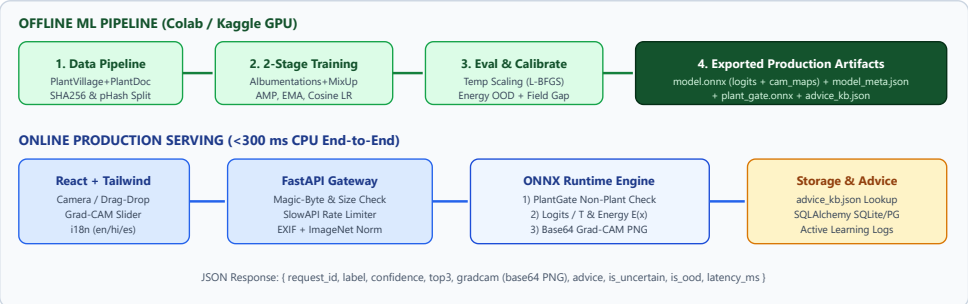
Probability Calibration & Expected Calibration Error (ECE): Modern neural networks are often overconfident (predicting 99% confidence even when wrong 15% of the time). A model is *calibrated* when a predicted probability of p truly corresponds to p empirical accuracy. **Temperature Scaling** rescales logits by scalar T post-hoc without changing top-1 accuracy, drastically lowering ECE.

Out-of-Distribution (OOD) Detection: Identifying inputs that do not belong to any training class (e.g., a selfie, tractor, wall, or unknown plant). We combine three orthogonal guards: (a) a binary **PlantGate** CNN, (b) the **Helmholtz Energy Score** over logits, and (c) calibrated max-softmax probability thresholding.

Grad-CAM (Gradient-weighted Class Activation Mapping): An Explainable AI (XAI) technique that highlights spatial regions in the input image contributing most to the predicted class score. In our production ONNX graph, we prove algebraic equivalence for GAP+Linear heads and compute Grad-CAM directly via a **1×1 Convolution** in <0.1 ms on CPU without shipping PyTorch.

ONNX Runtime (Open Neural Network Exchange): A cross-platform, hardware-accelerated C++ inference engine that executes the exported static computational graph (`model.onnx`) on CPU in ~35–90 ms per image without requiring the heavy PyTorch training library in production containers.

1.3 End-to-End System Architecture



Chapter 2: Training, Validation, Testing & Field Data Design

2.1 Public Datasets Used & Domain Characteristics

Dataset	Role in Project	Domain	Classes & Characteristics	Canonical Naming Rule
PlantVillage `spMohanty/PlantVillage-Dataset`	Primary supervised training, validation & in-domain test corpus	`lab`	38 classes across 14 crops (Apple, Tomato, Potato, Corn, Grape, Pepper, etc.) on uniform backgrounds.	`Crop___Disease` or `Crop___healthy` (e.g., `Tomato___Early_blight`)
PlantDoc `pratikkayal/PlantDoc-Dataset`	Real-field benchmark (`test_field.parquet`) measuring Lab-to-Field gap	`field`	27 classes of real outdoor farm photographs with clutter, variable sunlight, shadows, and multiple leaves.	Mapped via `canonical_class_map` in `datasets.yaml` to match PlantVillage labels
IP102 `rtlmdd/ip102` (Kaggle)	Agricultural insect & pest classification	`field`	102 pest species (configurable `top_k_classes: 15` to avoid severe tail collapse).	`Pest___<Normalized_Name>` (e.g., `Pest___Aphids`)
Rice & Cassava (Optional Kaggle)	Additional tropical staple crop coverage	`field`	Rice (Bacterial leaf blight, Brown spot, Leaf smut, Blast) & Cassava (CBB, CBSD, CGM, CMD, Healthy).	`Rice___<Disease>`, `Cassava___<Disease>`
OOD Non-Plant `torchvision.datasets.CIFAR10`	OOD threshold calibration & `PlantGateNet` negative training	`non-plant`	1,500 natural images from 10 non-plant classes (airplane, car, bird, cat, deer, dog, frog, horse, ship, truck), resized cleanly to 96×96.	Indexed separately in `ood_negatives.parquet`

2.2 Data Cleaning & Quality Filtering Pipeline (`ml/data/clean_and_merge.py`)

1. **Integrity Verification:** Every candidate file is opened with `PIL.Image.verify()` and re-decoded via `ImageOps.exif_transpose(img).convert("RGB")` so rotated smartphone photos are upright before any tensor conversion.
2. **Resolution & Aspect Guard:** Images smaller than `min_resolution = 64 px` or with extreme aspect ratios (`max(w/h, h/w) > 6.0`) are rejected.
3. **Exact Byte Deduplication:** A `SHA-256` hash of raw bytes removes identical files. When identical hashes exist across domains, real-field records are kept preferentially (`domain_priority = {"field": 0, "lab": 1}`).
4. **Optional Materialization (`--materialize`):** Writes clean, EXIF-corrected RGB JPEGs capped at max dimension `512 px` to `data/processed/images/`, dramatically speeding up Colab/Kaggle epoch I/O.

2.3 Leak-Free Stratified Splitting Strategy (70% Train / 15% Val / 15% Test)

PlantVillage contains augmented/rotated derivatives and burst shots of the exact same leaf. A standard random `train_test_split` leaks near-identical leaves into validation and test sets. Our pipeline prevents this rigorously:

1. **64-bit Perceptual Hashing:** Each clean image receives a 64-bit `imagehash.phash` stored as a 16-character hex string (preventing signed int64 Parquet overflow).
2. **Vectorized Hamming Clustering:** Using a precomputed 256-entry popcount lookup table `_POPCOUNT_LUT` over XORed `uint64` blocks, all pairs with Hamming distance `≤ 6` are merged via a path-compressed **Union-Find** structure into atomic `dup_group_id` clusters.
3. **Stratified Group Assignment:** Each `dup_group_id` is assigned atomically to **one** split (Train 70%, Val 15%, Test 15%), greedily minimizing per-class deficit relative to target ratios.
4. **PlantDoc Holdout Isolation:** When `plantdoc_eval_only: true`, PlantDoc is held out into `test_field.parquet` and any PlantDoc image clustering with training pHashes is removed, guaranteeing a pure zero-leakage lab-to-field benchmark.
5. **Automated Leakage Audit:** Saved in `split_audit.json` confirming `sha256_overlaps == 0`, `dup_group_overlaps == 0`, and `cross_split_phash_hits == 0`.

2.4 Handling Class Imbalance

Crop datasets are heavily skewed (e.g., Soybean healthy >5,000 images vs. minor diseases ~150 images). We support two complementary mechanisms controlled by `configs/base.yaml`:

- **Class-Weighted Loss (Default):** Computes per-class weights w_c via smoothed inverse square-root frequency (or effective number of samples) clipped to `[0.25, 5.0]`, applied inside both standard Cross-Entropy and our custom `WeightedSoftTargetCrossEntropy` during MixUp/CutMix.
- **WeightedRandomSampler (Optional):** Controlled by `use_weighted_sampler: true` to oversample minority classes at batch construction time.

Chapter 3: Every Mathematical Formula Used in the Project

1. ImageNet Channel Normalization (Preprocessing & Inference)

After resizing to `round(224 × 1.14) = 256` and center-cropping to `224 × 224`, pixel values $x \in [0, 255]$ are standardized per RGB channel:

$$x_{\text{norm},c} = \frac{(x_c / 255.0) - \mu_c}{\sigma_c}, \quad \mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

2. 64-Bit Perceptual Hash Hamming Distance (Near-Duplicate Detection)

Given two 64-bit unsigned integer pHashes h_a, h_b, bitwise XOR marks differing frequency bits and byte-wise popcount sums them:

$$d_H(h_a, h_b) = \sum_{b=0..7} \text{LUT}[\ ((h_a \oplus h_b) >> 8b)\ \&\ 0xFF\ ] \le \tau_{\text{pHash}} = 6$$

3. Class Imbalance Weighting (Inverse Square-Root & Effective Number)

For total training count N and class c count N_c, raw weights are normalized by their mean $\bar{w}$ and clipped to `[0.25, 5.0]`:

$$w_c^{\text{raw}} = \sqrt{(N / \max(N_c, 1))} \quad \text{or} \quad \frac{1-\beta}{1-\beta^{N_c}} \ (\beta{=}0.999), \quad w_c = \text{clip}\left(\frac{w_c^{\text{raw}}}{\bar{w}}, 0.25, 5.0\right)$$

4. Label Smoothing, MixUp & CutMix Soft Targets

With label smoothing $\varepsilon = 0.1$ over K classes, one-hot labels become smoothed vectors y_{smooth}. With probability $p = 0.5$, two samples (i, j) are blended with $\lambda \sim Beta(\alpha, \alpha)$:

$$y_{\text{smooth},k} = (1-\varepsilon) \cdot \mathbb{I}[k = y] + \frac{\varepsilon}{K} \quad | \quad \text{MixUp: } \tilde{x} = \lambda x_i + (1{-}\lambda) x_j, \ \tilde{y} = \lambda y_i + (1{-}\lambda) y_j$$

$$\text{CutMix: } \lambda_{\text{adj}} = 1 - \frac{(x_2 - x_1)(y_2 - y_1)}{W \cdot H}, \ \tilde{y} = \lambda_{\text{adj}}\, y_i + (1 - \lambda_{\text{adj}})\, y_j$$

5. Weighted Soft-Target Cross-Entropy Loss

Used during training with MixUp/CutMix so class weights remain active even on blended targets:

$$w_{\text{sample},b} = \sum_{k=1..K} \tilde{y}_{b,k} \cdot w_k, \quad L_{\text{WCE}} = -\frac{\sum_{b=1..B} w_{\text{sample},b} \sum_{k=1..K} \tilde{y}_{b,k} \log(\text{softmax}(z_b)_k)}{\max(\sum_{b=1..B} w_{\text{sample},b},\ 10^{-8})}$$

6. Knowledge Distillation Loss (Teacher → Lightweight Student)

Combines hard ground-truth Cross-Entropy with temperature-scaled Kullback-Leibler divergence ($T = 4.0$, $\alpha_{KD} = 0.5$):

$$L_{\text{KD}} = (1 - \alpha_{\text{KD}})\, L_{\text{CE}}(z_s, y) + \alpha_{\text{KD}} \cdot T^2 \cdot \text{KL}(\ \text{softmax}(z_t / T) \parallel \text{softmax}(z_s / T)\ )$$

7. Step-Level Cosine Learning Rate with Linear Warmup & EMA

For step t, warmup steps T_w, total steps T_{tot}, and minimum ratio $r_{min} = \eta_{min} / \eta_{max}$:

$$\lambda(t) = \begin{cases} \frac{t+1}{T_w} & \text{if } t < T_w; \\ r_{\min} + \frac{1 - r_{\min}}{2}\left(1 + \cos\left(\pi\ \frac{t - T_w}{T_{\text{tot}} - T_w}\right)\right) & \text{otherwise} \end{cases} \quad | \quad \text{EMA: } \theta_{\text{EMA}}^{(t)} = 0.999\, \theta_{\text{EMA}}^{(t-1)} + 0.001\, \theta^{(t)}$$

8. Evaluation Metrics & Lab-to-Field Generalization Gap

$$\text{Macro-F1} = \frac{1}{K} \sum_{c=1..K} \frac{2\, P_c\, R_c}{P_c + R_c}, \quad \text{Gap}_{\text{Acc}} = \text{Acc}_{\text{test_in_domain}} - \text{Acc}_{\text{PlantDoc_field}}, \quad \text{Gap}_{\text{F1}} = \text{F1}_{\text{test_in_domain}} - \text{F1}_{\text{PlantDoc_field}}$$

9. Post-Hoc Temperature Scaling & Expected Calibration Error (ECE)

A single positive parameter $T = clip(exp(log\ T), 0.1, 10.0)$ is fitted on validation logits via L-BFGS minimizing NLL. Predictions are binned into $M = 15$ equal-width bins:

$$\hat{p}_c(x) = \frac{\exp(z_c / T)}{\sum_{j=1..K} \exp(z_j / T)}, \quad \text{ECE} = \sum_{b=1..M} \frac{|B_b|}{N} \left| \text{acc}(B_b) - \text{conf}(B_b) \right|$$

10. Helmholtz Free-Energy OOD Score & PlantGate Probability

Log-sum-exp is shifted by z_{max} for numerical stability. More negative energy = confident in-distribution; energy above threshold triggers OOD rejection:

$$E(x) = -(z_{max} + \log \sum_{c=1..K} \exp(z_c - z_{max})) > \tau_{energy} \Rightarrow \text{OOD}, \quad P(\text{plant}) = \frac{1}{1 + \exp(-z_{gate})} < 0.45 \Rightarrow \text{Non-Plant}$$

11. Grad-CAM & Algebraic Equivalence to ONNX 1×1 Convolution

In standard Grad-CAM on spatial map $A \in \mathbb{R}^{C_f \times H_f \times W_f}$, gradient weights are averaged spatially. For Global Average Pooling + Linear head ($y_c = b_c + \sum_k W_{c,k} (1/H_f W_f) \sum_{i,j} A_{k,i,j}$), the partial derivative is constant across (i, j):

$$\alpha_k^c = \frac{1}{H_f W_f} \sum_{i,j} \frac{\partial y_c}{\partial A_{k,i,j}} = \frac{1}{H_f W_f} W_{c,k} \Rightarrow L_{\text{Grad-CAM}}^c = \text{ReLU}(\sum_k W_{c,k} A_k) = \text{ReLU}(\text{Conv}_{1\times1}(A; W))$$

This exact algebraic equivalence allows the exported ONNX model to emit both `logits` and `cam_maps` in a single CPU forward pass in **<0.1 ms** without backpropagation or PyTorch in production.

Chapter 4: Models Used & Why Each Model Is Needed

Model / timm ID	Architecture Family	Why It Is Needed in This Project	Strengths & Trade-offs
EfficientNetV2-S `tf_efficientnetv2_s.in21k_ft_in1k`	Modern CNN (Fused-MBConv + MBConv + SE attention)	**Primary Production Model:** Best accuracy-to-CPU-latency ratio. Pretrained on ImageNet-21k and fine-tuned on 1k. Pure GAP+Linear head makes 1×1 ONNX CAM 100% exact to PyTorch Grad-CAM.	Fast convergence, ~21M params, <90 ms CPU inference, high resilience to leaf texture noise.
ConvNeXt-Tiny `convnext_tiny.fb_in22k_ft_in1k`	Modernized Pure ConvNet (7×7 depthwise conv, LayerNorm, GELU)	**CNN Benchmark Challenger:** Uses large 7×7 kernels that capture wider spatial context across larger necrotic leaf blotches and stem lesions without attention overhead.	~28M params; strong accuracy on field images (PlantDoc); slightly higher CPU MACs than EfficientNetV2-S.
Swin-Tiny `swin_tiny_patch4_window7_224.ms_in22k_ft_in1k`	Hierarchical Vision Transformer (Shifted Window Self-Attention)	**Transformer Benchmark Challenger:** Tests whether global/regional self-attention outperforms local convolutions on scattered pest damage or multi-spot viral mosaics.	~28M params; outputs NHWC features (automatically permuted to NCHW in `CropClassifier`); slower on CPU.
ViT-Small (Supported) `vit_small_patch16_224.augreg_in21k_ft_in1k`	Isotropic Vision Transformer (16×16 patches + CLS token)	**Alternative Pure Transformer:** Supported seamlessly via `_to_nchw_spatial_map` which strips prefix tokens and reshapes 196 patch tokens into a 14×14 spatial map.	Strong global modeling; weaker inductive bias on small datasets without heavy MixUp/CutMix.
MobileNetV3-Large `mobilenetv3_large_100.ra_in1k`	Lightweight Edge CNN (Hard-Swish + Depthwise Separable Conv)	**Distilled Student Model:** Optional target for `distill.py` when deploying onto low-power Raspberry Pi, mobile TFLite, or 1-vCPU budget servers while retaining teacher accuracy.	~5.4M params; ultra-low CPU latency (~15–25 ms).
PlantGateNet `mobilenetv3_small_100.lamb_in1k`	Ultra-Compact Binary CNN (1 output logit)	**OOD Non-Plant Guardrail:** Softmax classifiers always sum to 100% even on a car or face. PlantGate runs in ~4 ms ahead of or alongside the main model to reject non-plant uploads before classification.	~1.5M params (~6 MB ONNX); high ROC-AUC separating crops/pests from CIFAR-10 objects/animals.

Starter vs. Trained Artifacts: Before running GPU training, `scripts/init_starter_artifacts.py` creates a structurally valid placeholder ONNX model with random weights (yielding ~10–15% confidence across 38 classes, which safely triggers the Low Confidence warning). Replacing `artifacts/model.onnx` and `artifacts/model_meta.json` with trained Kaggle/Colab GPU outputs elevates accuracy to ~97–99% in-domain and confidence to 85–99%.

Chapter 5: Every Parameter Explained (Data, Training, Optuna & Export)

5.1 Base & Data Parameters (`configs/base.yaml` & `configs/datasets.yaml`)

Parameter	Default Value	Exact Purpose & Effect
`seed`	`42`	Master RNG seed passed to Python, NumPy, PyTorch CPU/CUDA, DataLoader workers, Optuna TPE, and split shuffling.
`image.size`	`224`	Spatial resolution `224×224` fed into backbones (matches Swin-224 and fast CPU <300 ms budget).
`image.mean` / `std`	`[0.485, 0.456, 0.406]` / `[0.229, 0.224, 0.225]`	ImageNet channel statistics required by pretrained `timm` backbones and replicated identically in FastAPI & Gradio.
`image.min_resolution`	`64`	Minimum shorter side in pixels; filters tiny/corrupt thumbnails during cleaning.
`image.max_aspect_ratio`	`6.0`	Drops malformed banner/strip images where `max(W/H, H/W) > 6.0` .
`split.train_ratio / val_ratio / test_ratio`	`0.70 / 0.15 / 0.15`	Target per-class split proportions assigned atomically over near-duplicate visual clusters.
`split.phash_hamming_threshold`	`6`	Maximum 64-bit Hamming bit flips to consider two images near-duplicates (0 = exact perceptual match, 6 catches crops/flips/recompressions).
`split.min_samples_per_class`	`15`	Drops classes with fewer than 15 images so every kept class can populate train/val/test cleanly.
`split.plantdoc_eval_only`	`true`	Holds PlantDoc out exclusively as `test_field.parquet` to measure pure zero-shot Lab-to-Field gap.
`imbalance.weight_strategy`	`inverse_sqrt`	Class weight formula (`inverse_sqrt` , `effective_num` , or `none`).
`imbalance.min_weight / max_weight`	`0.25 / 5.0`	Clamps class weights to prevent gradient spikes on rare classes.
`imbalance.use_weighted_sampler`	`false`	If `true` , uses `WeightedRandomSampler` instead of loss weighting.
`ip102.top_k_classes`	`15`	Keeps the top 15 most frequent IP102 pest species when IP102 is enabled (set `null` for all 102).
`ood_non_plant.max_samples`	`1500`	Number of CIFAR-10 non-plant negatives exported for OOD calibration & PlantGate training.

5.2 Model Training Parameters (`train_*.yaml` & `distill_mobilenetv3.yaml`)

Parameter	EfficientNetV2-S	ConvNeXt-Tiny	Swin-Tiny	Purpose
`model.backbone`	`tf_efficientnetv2_s.in21k_ft_in1k`	`convnext_tiny.fb_in22k_ft_in1k`	`swin_tiny_patch4_window7_224.ms_in22k_ft_in1k`	Pretrained `timm` identifier.
`model.drop_rate`	`0.3`	`0.3`	`0.2`	Dropout probability before linear classifier head.
`model.drop_path_rate`	`0.2`	`0.2`	`0.2`	Stochastic depth drop rate across residual/attention blocks.
`training.batch_size`	`32`	`32`	`32`	Images per step (48 for MobileNetV3 distillation).
`training.amp`	`true`	`true`	`true`	FP16 Automatic Mixed Precision on CUDA (halves VRAM, 2× speed).
`training.grad_clip_norm`	`1.0`	`1.0`	`1.0`	Max L2 gradient norm preventing exploding updates on Stage 2 unfreeze.
`training.label_smoothing`	`0.1`	`0.1`	`0.1`	Prevents overconfident logit saturation and improves generalization.
`training.ema.decay`	`0.999`	`0.999`	`0.999`	Exponential Moving Average decay rate tracking Stage 2 weights.
`training.mixup`	`α=0.2, cutmix=1.0, p=0.5`	same	same	50% batch MixUp/CutMix probability (50/50 switch

Parameter	EfficientNetV2-S	ConvNeXt-Tiny	Swin-Tiny	Purpose
				between MixUp & CutMix).
`stage1.epochs / lr`	`4 / 1e-3`	`4 / 1e-3`	`4 / 8e-4`	Frozen-backbone classifier head warmup epochs & AdamW LR.
`stage2.epochs / warmup`	`16 / 2`	`16 / 2`	`16 / 2`	End-to-end fine-tuning epochs & linear warmup length.
`stage2.backbone_lr`	`1.0e-4`	`8.0e-5`	`5.0e-5`	Conservative LR for pretrained feature layers.
`stage2.head_lr`	`5.0e-4`	`4.0e-4`	`3.0e-4`	Higher LR (5–6×) for classifier head.
`stage2.weight_decay`	`1e-2`	`2e-2`	`5e-2`	Decoupled L2 penalty (0.0 on biases and normalization weights).
`early_stopping_patience`	`5`	`5`	`5`	Stops Stage 2 if `val_macro_f1` fails to improve for 5 epochs.

5.3 Optuna Search Space & Distillation Parameters

- **Optuna HPO (`tune_optuna.py`):** Tunes `backbone_lr ∈ [3e-5, 3e-4]` (log), `head_lr_mult ∈ [3.0, 10.0]`, `weight_decay ∈ [1e-3, 5e-2]` (log), `drop_rate ∈ [0.1, 0.45]`, `label_smoothing ∈ [0.0, 0.15]`, and `mixup_prob ∈ [0.2, 0.7]` using `TPESampler(seed=42)` and `MedianPruner(n_startup_trials=2, n_warmup_steps=3)`.
- **Distillation (`distill_mobilenetv3.yaml`):** `temperature = 4.0` (softens teacher logits to expose inter-class similarity), `alpha_kd = 0.5` (equal weight to KL distillation and hard labels), `epochs = 20`, `lr = 3e-4`.

Chapter 6: Backend & Frontend Architecture and Every Parameter

6.1 Backend Stack & Why Each Library Is Used

- **FastAPI + Uvicorn:** Asynchronous Python web framework providing automatic OpenAPI/Swagger docs, dependency injection, and fast multipart file handling.
- **ONNX Runtime (`onnxruntime`):** Executes `model.onnx` and `plant_gate.onnx` on CPU with `ORT_ENABLE_ALL` graph fusion and `intra_op_num_threads ≤ 4`, keeping the Docker image slim without PyTorch.
- **Pydantic v2 & `pydantic-settings` :** Enforces strict schema validation on requests/responses and loads configuration cleanly from `.env`.
- **SQLAlchemy 2.0:** ORM database layer supporting SQLite (with WAL mode + foreign keys) locally and instant PostgreSQL migration via `DATABASE_URL`.
- **SlowAPI:** Token-bucket IP rate limiter preventing API abuse (default `60/minute`).
- **Pillow + OpenCV-Headless:** Magic-byte checking, EXIF orientation correction, bilinear resizing, JET colormap blending for Grad-CAM PNGs.
- **`python-json-logger` :** Emits machine-parseable JSON logs stamped with per-request `X-Request-ID`.

6.2 Complete Backend Configuration Parameters (`backend/app/core/__init__.py` & `.env`)

Env Var / Setting	Default	Description
`PROJECT_NAME`	`Crop Disease & Pest Classifier`	OpenAPI documentation title.
`APP_ENV`	`development`	Controls SQLAlchemy query echo and Uvicorn hot-reload.
`LOG_LEVEL`	`INFO`	JSON logger verbosity (`DEBUG` , `INFO` , `WARNING` , `ERROR`).
`API_HOST / API_PORT`	`0.0.0.0 / 8000`	Bind interface and TCP port for Uvicorn.
`CORS_ORIGINS`	`http://localhost:5173,...`	Comma-separated allowed frontend origins for browser CORS preflight.
`RATE_LIMIT`	`60/minute`	SlowAPI global per-IP rate limit string.
`MAX_UPLOAD_MB`	`10`	Maximum upload size in MB (enforced in bytes via `max_upload_bytes`).
`MODEL_ONNX_PATH`	`artifacts/model.onnx`	Filesystem path to exported dual-output ONNX model.
`MODEL_META_PATH`	`artifacts/model_meta.json`	JSON containing classes, mean/std, calibrated `temperature` , and OOD thresholds.
`PLANT_GATE_ONNX_PATH`	`artifacts/plant_gate.onnx`	Optional binary non-plant gate ONNX path.
`ADVICE_KB_PATH`	`configs/advice_kb.json`	Path to agronomic treatment advice knowledge base.
`HF_MODEL_REPO / HF_TOKEN`	`""`	If set and local `.onnx` is missing, downloads artifacts automatically from Hugging Face Hub on startup.
`UNCERTAINTY_CONF_THRESHOLD`	`0.55`	Fallback calibrated confidence threshold below which `is_uncertain=true` .
`OOD_ENERGY_THRESHOLD`	`-3.5`	Fallback Helmholtz energy threshold above which `is_ood=true` and `label="Unknown"` .
`PLANT_GATE_THRESHOLD`	`0.45`	Minimum sigmoid probability from PlantGate to accept an image as plant/pest.
`DATABASE_URL`	`sqlite:///./backend_data.sqlite3`	SQLAlchemy DSN (swap to `postgresql+psycopg://...` for cloud DB).

6.3 Backend REST Endpoints & Database Schema

Endpoint / Table	Method / Columns	Behavior & Purpose
`/health`	`GET`	Returns `{status, model_loaded, num_classes, backend_version}` for container healthchecks.
`/classes`	`GET`	Returns `{num_classes, classes}` used to populate the frontend feedback dropdown.
`/advice/{class_id}`	`GET`	Returns `AdviceEntry` (`severity, symptoms, organic_treatment, chemical_treatment, prevention`) with healthy/pest/disease fallbacks.
`/predict`	`POST` (multipart `file`)	Validates magic bytes (JPEG/PNG/WebP/BMP) & size ≤10 MB, runs PlantGate + ONNX, calibrates logits, checks OOD, renders Grad-CAM PNG base64, logs to DB, returns `PredictionResponse` .
`/feedback`	`POST` (JSON)	Records `{request_id, is_correct, corrected_class, comment}` ; requires `corrected_class` if `is_correct=false` .
`prediction_logs`	`id, request_id, predicted_class, confidence, top3 (JSON), is_uncertain, is_ood, filename, file_size_bytes, latency_ms, created_at`	Audit trail & latency monitoring table.
`user_feedback`	`id, request_id, predicted_class, is_correct, corrected_class, comment, created_at`	Active-learning queue queried by `scripts/export_feedback_dataset.py` .

6.4 Frontend Stack & Component Parameters (React 18 + Vite + Tailwind + i18next)

- **Build & Proxy (`vite.config.ts`):** Runs on port `5173` ; proxies `/api/*` to `VITE_API_BASE_URL` (default `http://localhost:8000`). In production Docker, Nginx serves static assets on port `80` and proxies `/api/` to `http://backend:8000/` .
- **Styling (`tailwind.config.js`):** Uses `darkMode: "class"` toggled on `<html>` , custom `brand` agricultural green palette (`#f0fdf4` to `#14532d`), and `:focus-visible` accessibility rings.
- **Internationalization (`src/i18n/index.ts`):** Configured with `LanguageDetector` (order: `localStorage` → `navigator`) and `fallbackLng: "en"` , shipping full translations for English (`en`), Hindi (`hi`), and Spanish (`es`).

Component / Hook	Key Props & State Parameters	Responsibility
`App.tsx`	`isLoading`, `errorMessage`, `currentImageSrc`, `currentPrediction`, `availableClasses`, `isHistoryOpen`	Root state orchestrator; fetches `/classes` on mount, invokes `api.predict`, saves thumbnails to history.
`Header.tsx`	`isDark`, `onToggleDark`, `onOpenHistory`, `historyCount`	Sticky header with history badge counter, language `<select>`, and theme toggle.
`UploadZone.tsx`	`onImageSelected(file)`, `isLoading`, `isDragOver`	Drag-and-drop, file picker (`image/jpeg,png,webp,bmp`), and mobile rear camera capture (`capture="environment"`).
`GradCamViewer.tsx`	`originalSrc`, `gradcamBase64`, `showHeatmap (true)`, `opacity (0.5)`	Stacks base64 Grad-CAM PNG over original image with `mix-blend-screen`, toggle checkbox, and `0.1–1.0` slider.
`ResultCard.tsx`	`prediction: PredictionResponse`	Displays OOD (rose) or uncertain (amber) banner, formatted label, color-coded confidence bar, latency ms, and Top-3 list.
`AdviceCard.tsx`	`advice: AdviceEntry`, `isHealthy`	Color-coded severity badge (`healthy/low/moderate/high/critical`), symptoms, organic, chemical (hidden if healthy), prevention, disclaimer.
`FeedbackWidget.tsx`	`requestId`, `predictedLabel`, `availableClasses`, `submitted`, `showCorrectionForm`	Yes/No confirmation buttons + searchable correction dropdown & optional 300-char note calling `POST /feedback`.
`HistoryDrawer.tsx`	`isOpen`, `onClose`, `history`, `onSelect`, `onClear`	Slide-over drawer backed by `usePredictionHistory` (`localStorage` key `crop_pred_history_v1`, max 20 scans).

Chapter 7: Data Pipeline, Reproducibility & Model Source Code (Line-by-Line)

This chapter reproduces the exact source code and a line-by-line engineering explanation for reproducibility utilities, dataset cleaning/fingerprinting, leak-free pHash splitting, PyTorch dataset loading, the unified `timm` backbone factory, custom loss functions, and the `PlantGateNet` OOD detector.

7.1 Deterministic Seeding & Device Selection — ml/utils/seed.py

File Summary: Locks Python, NumPy, and PyTorch CPU/CUDA random number generators and guards against unsupported GPU SM architectures (e.g., Blackwell sm_120 on older CUDA 11.8 builds).

ml/utils/seed.py (Lines 1–65)

Line	Source Code	Line-by-Line Explanation
1	`"""Deterministic random seed utilities for reproducible PyTorch training."""`	Module docstring stating the purpose: deterministic random seed helpers for repeatable training.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed evaluation of type annotations (all annotations become strings until inspected).
4		*Blank visual separator line.*
5	`import os`	Imports Python's standard `os` module for reading environment variables and setting `PYTHONHASHSEED`.
6	`import random`	Imports Python's built-in `random` PRNG module.
7		*Blank visual separator line.*
8	`import numpy as np`	Imports NumPy for seeding NumPy's global random generator used in splits and Albumentations.
9	`import torch`	Imports PyTorch for CPU/CUDA tensor seeding and backend cuDNN flags.
10		*Blank visual separator line.*
11		*Blank visual separator line.*
12	`def seed_everything(seed: int = 42, deterministic: bool = False) -> None:`	Defines `seed_everything` taking an integer `seed` (default `42`) and `deterministic` flag (default `False`).
13	`    """Seed Python, NumPy, and PyTorch RNGs for reproducible runs.`	Function docstring opening.
14		*Blank visual separator line.*
15	`    Args:`	Docstring Args header.
16	`        seed: Integer random seed.`	Documents `seed` parameter.
17	`        deterministic: If True, forces deterministic cuDNN kernels (slightly slower).`	Documents `deterministic` trade-off (reproducibility vs. kernel speed).
18	`    """`	Closes docstring.
19	`    os.environ["PYTHONHASHSEED"] = str(seed)`	Sets `PYTHONHASHSEED` so dictionary/set iteration hashes stay identical across runs.
20	`    random.seed(seed)`	Seeds Python's built-in Mersenne Twister PRNG.
21	`    np.random.seed(seed)`	Seeds NumPy's legacy global RNG (`np.random`).
22	`    torch.manual_seed(seed)`	Seeds PyTorch's CPU RNG.
23	`    torch.cuda.manual_seed(seed)`	Seeds the currently active CUDA GPU RNG.
24	`    torch.cuda.manual_seed_all(seed)`	Seeds all available CUDA GPUs for multi-GPU setups.
25		*Blank visual separator line.*
26	`    if deterministic:`	Branches on strict cuDNN determinism mode.
27	`        torch.backends.cudnn.deterministic = True`	Forces cuDNN to pick only deterministic convolution algorithms (zero non-deterministic atomics).
28	`        torch.backends.cudnn.benchmark = False`	Disables cuDNN auto-tuner benchmarking because algorithm selection could vary across runs.
29	`    else:`	Fallback fast branch when `deterministic=False`.
30	`        torch.backends.cudnn.deterministic = False`	Allows faster non-deterministic cuDNN kernels when slight floating-point variance is acceptable.
31	`        torch.backends.cudnn.benchmark = True`	Enables cuDNN auto-tuner to cache the fastest kernel for fixed `(224x224)` input shapes.
32		*Blank visual separator line.*
33		*Blank visual separator line.*
34	`def seed_worker(worker_id: int) -> None:`	Defines `seed_worker(worker_id)` required by PyTorch `DataLoader(worker_init_fn=...)`.
35	`    """DataLoader worker initialization function to ensure per-worker determinism."""`	Explains that each spawned DataLoader subprocess needs an independent deterministic seed.
36	`    worker_seed = torch.initial_seed() % (2**32)`	Extracts the worker-unique 32-bit seed derived by PyTorch from the DataLoader generator.
37	`    np.random.seed(worker_seed)`	Seeds NumPy inside this worker process so Albumentations random transforms are reproducible.
38	`    random.seed(worker_seed)`	Seeds Python's `random` module inside this worker process.
39		*Blank visual separator line.*
40		*Blank visual separator line.*
41	`def make_generator(seed: int = 42) -> torch.Generator:`	Defines factory helper `make_generator(seed)` returning a `torch.Generator`.
42	`    """Create a seeded PyTorch CPU RNG generator for DataLoaders and samplers."""`	Explains its role for `DataLoader(..., generator=...)` and `WeightedRandomSampler`.
43	`    generator = torch.Generator()`	Instantiates a new PyTorch CPU random generator object.
44	`    generator.manual_seed(seed)`	Applies the integer `seed` to the generator.
45	`    return generator`	Returns the seeded generator instance.
46		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
47		*Blank visual separator line.*
48	`def get_default_device() -> torch.device:`	Defines `get_default_device()` to safely pick `'cuda'` or `'cpu'`.
49	`    """Select the best available device, safely handling uncompiled CUDA architectures."""`	Explains GPU capability safeguard.
50	`    forced = os.getenv("DEVICE")`	Reads optional `DEVICE` environment override (e.g., `DEVICE=cpu`).
51	`    if forced:`	Checks if user explicitly forced a device string.
52	`        return torch.device(forced)`	Immediately returns the forced `torch.device`.
53		*Blank visual separator line.*
54	`    if torch.cuda.is_available():`	Checks if PyTorch detects an active CUDA runtime and driver.
55	`        try:`	Starts try-block to safely query GPU compute capability without crashing.
56	`            major, minor = torch.cuda.get_device_capability()`	Queries `(major, minor)` compute capability of GPU 0 (e.g., `(7, 5)` for T4, `(12, 0)` for RTX 5050).
57	`            # sm_90 or lower is supported by cu118; newer sm_120 (Blackwell) requires cu128+`	Comment explaining why SM major > 9 (Blackwell `sm_120`) fails on CUDA 11.8 wheels.
58	`            if major <= 9:`	Accepts GPU only if compute capability major version is `<= 9` (Volta/Turing/Ampere/Ada/Hopper).
59	`                return torch.device("cuda")`	Returns CUDA device when architecture is supported by installed PyTorch.
60	`        except Exception:`	Catches any driver query error gracefully.
61	`            pass`	Ignores the query failure and falls through to CPU fallback.
62		*Blank visual separator line.*
63	`    return torch.device("cpu")`	Returns `torch.device('cpu')` as safe universal fallback.
64		*Blank visual separator line.*
65		*Blank visual separator line.*

7.2 Image Validation, Hashing & Deduplication — ml/data/clean_and_merge.py (Lines 235–424)

File Summary: Core validation, cryptographic SHA-256 + perceptual pHash fingerprinting, exact deduplication prioritizing real-field metadata, Parquet cataloging, and OOD negative indexing.

ml/data/clean_and_merge.py (Lines 235–424)

Line	Source Code	Line-by-Line Explanation
235	`def _sha256_file(path: Path) -> str:`	Defines `_sha256_file(path)` to compute a collision-free SHA-256 hex digest of raw image bytes.
236	`    h = hashlib.sha256()`	Creates a new `hashlib.sha256` streaming hasher.
237	`    with path.open("rb") as f:`	Opens the image file in binary read mode (`'rb'`).
238	`        for chunk in iter(lambda: f.read(1024 * 64), b""):`	Iterates in 64 KB chunks (`1024 * 64` bytes) until EOF (`b''`) so large files don't spike RAM.
239	`            h.update(chunk)`	Feeds each 64 KB chunk into the SHA-256 state machine.
240	`    return h.hexdigest()`	Returns the 64-character lowercase hexadecimal digest string.
241		*Blank visual separator line.*
242		*Blank visual separator line.*
243	`def _validate_and_fingerprint(`	Defines `_validate_and_fingerprint` to check file integrity, geometry, and compute fingerprints.
244	`    path: Path,`	Parameter `path`: path to candidate raw image.
245	`    min_resolution: int,`	Parameter `min_resolution`: minimum allowed width or height (default `64` px).
246	`    max_aspect_ratio: float,`	Parameter `max_aspect_ratio`: maximum allowed aspect distortion (default `6.0`).
247	`    materialize_path: Path | None = None,`	Optional `materialize_path`: if provided, saves standardized `<=512px` RGB JPEG copy.
248	`) -> dict[str, Any] | None:`	Return annotation: metadata dictionary or `None`.
249	`    """Verify readability, EXIF-transpose to RGB, enforce size checks, and compute pHash."""`	Docstring summarizing verification, EXIF rotation, size filtering, and hashing.
250	`    try:`	Starts `try` block to catch unreadable/truncated/corrupt image files.
251	`        with Image.open(path) as img_verify:`	Opens image with PIL solely for structural header verification.
252	`            img_verify.verify()`	`img_verify.verify()` parses file headers/CRC to reject corrupt images before pixel decoding.
253	`        with Image.open(path) as img_raw:`	Re-opens the verified image fresh (`verify()` leaves PIL file handle un-reusable).
254	`            img = ImageOps.exif_transpose(img_raw).convert("RGB")`	Applies EXIF Orientation tag (`exif_transpose`) so smartphone portrait/landscape rotation is fixed, then converts RGBA/CMYK/Grayscale to 3-channel `RGB`.
255	`            width, height = img.size`	Extracts `(width, height)` pixel dimensions.
256	`            if min(width, height) < min_resolution:`	Checks whether the shorter edge is smaller than `min_resolution` (`64px`).
257	`                return {"status": "too_small"}`	Flags tiny thumbnails with status `'too_small'` for exclusion.
258	`            ratio = max(width / height, height / width)`	Computes symmetric aspect ratio `max(w/h, h/w) >= 1.0`.
259	`            if ratio > max_aspect_ratio:`	Checks if image is an extreme banner/strip exceeding `max_aspect_ratio` (`6.0`).
260	`                return {"status": "bad_aspect"}`	Flags extreme aspect-ratio files with `'bad_aspect'`.
261	`            phash_hex = str(imagehash.phash(img))`	Computes 64-bit DCT perceptual hash (`imagehash.phash`) as a 16-char hex string.
262	`            sha256 = _sha256_file(path)`	Computes exact cryptographic `sha256` digest of the source file.
263		*Blank visual separator line.*
264	`            final_path = path`	Defaults output catalog `final_path` to the original source path.
265	`            if materialize_path is not None:`	Checks whether standardized JPEG materialization was requested (`--materialize`).

Line	Source Code	Line-by-Line Explanation
266	`                ensure_dir(materialize_path.parent)`	Creates parent class folder under `data/processed/images/` if absent.
267	`                img_out = img.copy()`	Copies decoded RGB PIL image before resizing.
268	`                img_out.thumbnail((512, 512), Image.Resampling.LANCZOS)`	Downsamples proportionally so longest edge `<= 512` px using high-quality LANCZOS filter.
269	`                img_out.save(materialize_path, format="JPEG", quality=92)`	Writes clean RGB JPEG at quality `92` without EXIF bloat.
270	`                final_path = materialize_path`	Updates `final_path` to point to the materialized JPEG.
271		*Blank visual separator line.*
272	`            return {`	Returns structured dictionary of validated metadata and hashes.
273	`                "status": "ok",`	Marks validation result `'ok'`.
274	`                "final_path": str(final_path.as_posix()),`	Stores POSIX-normalized path to clean/original image.
275	`                "width": width,`	Stores original width in pixels.
276	`                "height": height,`	Stores original height in pixels.
277	`                "sha256": sha256,`	Stores SHA-256 fingerprint.
278	`                "phash_hex": phash_hex,`	Stores 16-hex-char 64-bit perceptual hash.
279	`            }`	Closes returned dict.
280	`    except Exception:`	Catches any PIL/IO/truncated-stream exception.
281	`        return {"status": "corrupt"}`	Returns `{'status': 'corrupt'}` so caller counts and skips bad files.
282		*Blank visual separator line.*
283		*Blank visual separator line.*
284	`def clean_and_merge(`	Defines main orchestrator `clean_and_merge(cfg, materialize, max_per_class)`.
285	`    cfg: dict[str, Any],`	Parameter `cfg`: merged datasets + base YAML dictionary.
286	`    materialize: bool = False,`	Parameter `materialize`: whether to save standardized JPEGs.
287	`    max_per_class: int | None = None,`	Parameter `max_per_class`: optional debug/smoke cap per label.
288	`) -> pd.DataFrame:`	Return type: `pd.DataFrame` catalog.
289	`    """Collect, validate, deduplicate, and catalog all enabled datasets."""`	Docstring describing end-to-end collection, validation, deduplication, and cataloging.
290	`    raw_dir = Path(cfg["paths"]["raw_dir"])`	Resolves root raw downloads directory (`data/raw`).
291	`    processed_dir = ensure_dir(cfg["paths"]["processed_dir"])`	Ensures `data/processed` exists.
292	`    manifests_dir = ensure_dir(cfg["paths"]["manifests_dir"])`	Ensures `data/manifests` exists.
293	`    canonical_map: dict[str, str] = cfg.get("canonical_class_map", {})`	Loads raw-label to canonical `Crop___Disease` dictionary from YAML.
294	`    sources = cfg.get("sources", {})`	Reads `sources` config block controlling which datasets are enabled.
295		*Blank visual separator line.*
296	`    min_res = int(cfg["image"].get("min_resolution", 64))`	Reads `min_resolution` threshold (default `64`).
297	`    max_ratio = float(cfg["image"].get("max_aspect_ratio", 6.0))`	Reads `max_aspect_ratio` threshold (default `6.0`).
298		*Blank visual separator line.*
299	`    raw_records: list[dict[str, Any]] = []`	Initializes accumulator list for raw `(path, label, source, domain)` records.
300	`    if sources.get("plantvillage", {}).get("enabled", False):`	Checks if PlantVillage dataset source is enabled.
301	`        raw_records.extend(_collect_plantvillage(raw_dir, sources["plantvillage"], canonical_map))`	Collects PlantVillage lab images (`domain='lab'`).
302	`    if sources.get("plantdoc", {}).get("enabled", False):`	Checks if PlantDoc real-field dataset is enabled.
303	`        raw_records.extend(_collect_plantdoc(raw_dir, sources["plantdoc"], canonical_map))`	Collects PlantDoc field images (`domain='field'`).
304	`    if sources.get("ip102", {}).get("enabled", False):`	Checks if IP102 insect pest dataset is enabled.
305	`        raw_records.extend(_collect_ip102(raw_dir, sources["ip102"], canonical_map))`	Collects top-K IP102 pest images (`domain='field'`).
306	`    if sources.get("rice", {}).get("enabled", False):`	Checks if Rice Leaf Diseases dataset is enabled.
307	`        raw_records.extend(_collect_rice(raw_dir, canonical_map))`	Collects Rice field images (`domain='field'`).
308	`    if sources.get("cassava", {}).get("enabled", False):`	Checks if Cassava Leaf Disease dataset is enabled.
309	`        raw_records.extend(_collect_cassava(raw_dir, canonical_map))`	Collects Cassava field images (`domain='field'`).
310		*Blank visual separator line.*
311	`    if not raw_records:`	Guards against empty raw directory before scanning.
312	`        raise RuntimeError(`	Raises descriptive error pointing user to `ml.data.download`.
313	`            f"No dataset images found under {raw_dir}. Run `python -m ml.data.download` first."`	Error message string.
314	`        )`	Closes `RuntimeError`.
315		*Blank visual separator line.*
316	`    if max_per_class is not None and max_per_class > 0:`	Checks whether optional per-class debug cap `max_per_class` is active.
317	`        per_class_counter: Counter[str] = Counter()`	Creates per-label counter for capping.
318	`        capped = []`	Stores capped subset.
319	`        for rec in raw_records:`	Iterates through scanned records.
320	`            if per_class_counter[rec["label"]] < max_per_class:`	Keeps record only if its class count is still below cap.
321	`                capped.append(rec)`	Appends kept record.
322	`                per_class_counter[rec["label"]] += 1`	Increments class count.
323	`        raw_records = capped`	Replaces `raw_records` with capped subset.
324		*Blank visual separator line.*
325	`    stats: Counter[str] = Counter()`	Initializes `Counter` for cleaning telemetry (`dropped_corrupt`, `dropped_too_small`, etc.).
326	`    stats["total_scanned"] = len(raw_records)`	Records total candidate files before filtering.

Line	Source Code	Line-by-Line Explanation
327	`    clean_rows: list[dict[str, Any]] = []`	Initializes list of clean row dicts for Parquet export.
328		*Blank visual separator line.*
329	`    mat_root = processed_dir / "images" if materialize else None`	Sets root folder for materialized JPEGs if enabled.
330	`    for idx, rec in enumerate(tqdm(raw_records, desc="Validating & fingerprinting images")):`	Loops over every candidate record with `tqdm` progress bar.
331	`        src_path = Path(rec["path"])`	Wraps string path into `pathlib.Path`.
332	`        mat_path = (`	Builds deterministic target JPEG filename `<source>_<idx:07d>.jpg` inside class subfolder.
333	`            mat_root / rec["label"] / f"{rec['source']}_{idx:07d}.jpg"`	Format string for deterministic JPEG naming.
334	`            if mat_root is not None`	Uses materialized path only when `mat_root` is active.
335	`            else None`	Otherwise passes `None`.
336	`        )`	Closes path conditional.
337	`        res = _validate_and_fingerprint(`	Runs full validation, EXIF-fix, size check, SHA-256, and pHash computation.
338	`            src_path,`	Passes source image path.
339	`            min_resolution=min_res,`	Passes `min_resolution`.
340	`            max_aspect_ratio=max_ratio,`	Passes `max_aspect_ratio`.
341	`            materialize_path=mat_path,`	Passes optional materialization target path.
342	`        )`	Closes call.
343	`        status = res["status"] if res else "corrupt"`	Extracts `'ok'`, `'too_small'`, `'bad_aspect'`, or `'corrupt'`.
344	`        if status != "ok":`	Skips any non-ok image.
345	`            stats[f"dropped_{status}"] += 1`	Increments appropriate rejection counter (`dropped_*`).
346	`            continue`	Continues to next candidate.
347		*Blank visual separator line.*
348	`        clean_rows.append(`	Appends normalized catalog dictionary for valid image.
349	`            {`	Opens dict literal.
350	`                "image_path": res["final_path"],`	Catalog path used later by PyTorch Dataset.
351	`                "raw_path": str(src_path.as_posix()),`	Preserves original raw download path for provenance.
352	`                "label": rec["label"],`	Canonical `Crop___Disease` label.
353	`                "crop": rec["label"].split("___", 1)[0],`	Extracts crop name prefix before `___`.
354	`                "disease": rec["label"].split("___", 1)[1] if "___" in rec["label"] else "unknown",`	Extracts disease/pest suffix after `___`.
355	`                "source": rec["source"],`	Dataset origin (`plantvillage`, `plantdoc`, `ip102`, `rice`, `cassava`).
356	`                "domain": rec["domain"],`	Domain tag (`lab` vs. `field`).
357	`                "width": res["width"],`	Validated image width.
358	`                "height": res["height"],`	Validated image height.
359	`                "sha256": res["sha256"],`	Exact byte SHA-256 hash.
360	`                "phash_hex": res["phash_hex"],`	64-bit perceptual pHash hex.
361	`            }`	Closes row dict.
362	`        )`	Closes `append`.
363		*Blank visual separator line.*
364	`    df = pd.DataFrame(clean_rows)`	Converts valid rows into a `pandas.DataFrame`.
365	`    if df.empty:`	Guards against 0 valid images remaining.
366	`        raise RuntimeError("All scanned images failed validation.")`	Raises fatal error if nothing survived validation.
367		*Blank visual separator line.*
368	`    # Prefer keeping real-field metadata when deduplicating exact SHA-256 duplicates`	Comment explaining domain priority during exact deduplication.
369	`    domain_priority = {"field": 0, "lab": 1}`	Assigns priority `0` to `field` so real-field metadata wins over `lab` if exact bytes collide.
370	`    df["_prio"] = df["domain"].map(lambda d: domain_priority.get(d, 2))`	Maps temporary `_prio` column for sorting.
371	`    df = df.sort_values(["_prio", "source", "image_path"]).drop(columns=["_prio"])`	Sorts by `_prio`, `source`, and `image_path`, then drops `_prio`.
372	`    before_dedup = len(df)`	Records row count prior to exact deduplication.
373	`    df = df.drop_duplicates(subset=["sha256"], keep="first").reset_index(drop=True)`	Removes exact byte-for-byte duplicate files using `sha256`, keeping highest-priority first row.
374	`    stats["dropped_exact_duplicate"] = before_dedup - len(df)`	Logs exact duplicates removed.
375	`    stats["final_clean_images"] = len(df)`	Logs final clean unique image count.
376	`    stats["num_classes"] = int(df["label"].nunique())`	Logs number of distinct canonical classes.
377		*Blank visual separator line.*
378	`    catalog_path = processed_dir / "catalog.parquet"`	Target Parquet catalog path (`data/processed/catalog.parquet`).
379	`    df.to_parquet(catalog_path, index=False)`	Writes catalog without row index for fast columnar loading.
380	`    save_json(dict(stats), processed_dir / "cleaning_summary.json")`	Saves cleaning telemetry JSON to `data/processed/cleaning_summary.json`.
381		*Blank visual separator line.*
382	`    # Index OOD non-plant images if downloaded`	Comment: check for CIFAR-10 / non-plant OOD folder.
383	`    ood_dir = raw_dir / "ood_non_plant"`	Path to optional non-plant negative images (`data/raw/ood_non_plant`).
384	`    if ood_dir.exists():`	Checks if OOD folder was downloaded.
385	`        ood_rows = []`	Initializes list for OOD image rows.
386	`        for p in sorted(ood_dir.rglob("*")):`	Recursively walks sorted paths under `ood_non_plant`.
387	`            if p.is_file() and p.suffix.lower() in IMAGE_EXTS:`	Keeps only regular files with valid image extensions.

Line	Source Code	Line-by-Line Explanation
388	`                ood_rows.append({"image_path": str(p.as_posix()), "category": p.parent.name})`	Stores POSIX image path and parent CIFAR-10 category name.
389	`        if ood_rows:`	Writes OOD manifest if non-empty.
390	`            pd.DataFrame(ood_rows).to_parquet(manifests_dir / "ood_negatives.parquet", index=False)`	Saves `data/manifests/ood_negatives.parquet` for PlantGate & OOD calibration.
391	`            print(f"[clean] Indexed {len(ood_rows)} OOD non-plant negatives.")`	Logs count of indexed OOD negative samples.
392		*Blank visual separator line.*
393	`    print(f"[clean] Saved catalog ({len(df)} clean images, {df['label'].nunique()} classes) -> {catalog_path}")`	Logs final saved catalog count and path.
394	`    print(f"[clean] Summary: {dict(stats)}")`	Prints full cleaning summary dict.
395	`    return df`	Returns catalog DataFrame.
396		*Blank visual separator line.*
397		*Blank visual separator line.*
398	`def parse_args() -> argparse.Namespace:`	Defines CLI argument parser `parse_args()`.
399	`    parser = argparse.ArgumentParser(description="Clean, deduplicate, and merge datasets.")`	Creates `ArgumentParser` with description.
400	`    parser.add_argument("--config", default="configs/datasets.yaml", help="Datasets YAML path.")`	`--config` pointing to `configs/datasets.yaml`.
401	`    parser.add_argument("--base", default="configs/base.yaml", help="Base YAML path.")`	`--base` pointing to `configs/base.yaml`.
402	`    parser.add_argument(`	Adds `--materialize` boolean flag.
403	`        "--materialize",`	Flag name.
404	`        action="store_true",`	Boolean flag store_true.
405	`        help="Write standardized RGB JPEGs (<=512px) into data/processed/images.",`	Help string.
406	`    )`	Closes `--materialize`.
407	`    parser.add_argument(`	Adds `--max-per-class` optional integer cap.
408	`        "--max-per-class",`	Flag name.
409	`        type=int,`	Integer type.
410	`        default=None,`	Defaults to `None` (no cap).
411	`        help="Optional cap per class for fast smoke testing.",`	Help string.
412	`    )`	Closes `--max-per-class`.
413	`    return parser.parse_args()`	Parses and returns CLI arguments.
414		*Blank visual separator line.*
415		*Blank visual separator line.*
416	`def main() -> None:`	CLI entrypoint `main()`.
417	`    args = parse_args()`	Parses arguments.
418	`    cfg = load_config(args.config, base_path=args.base)`	Merges base + dataset YAML configs.
419	`    clean_and_merge(cfg, materialize=args.materialize, max_per_class=args.max_per_class)`	Invokes `clean_and_merge`.
420		*Blank visual separator line.*
421		*Blank visual separator line.*
422	`if __name__ == "__main__":`	Standard Python module execution guard.
423	`    main()`	Runs `main()` when invoked via `python -m ml.data.clean_and_merge`.
424		*Blank visual separator line.*

7.3 Leak-Free Perceptual-Hash Splitting & Class Weights — ml/data/split.py

File Summary: Groups visually similar images via 64-bit pHash Hamming distance + Union-Find, greedily allocates atomic clusters 70/15/15 per class, isolates PlantDoc to test_field, computes inverse-sqrt weights, and runs a zero-leakage audit.

ml/data/split.py (Lines 1–335)

Line	Source Code	Line-by-Line Explanation
1	`"""Leak-free stratified group splitting (70/15/15) using perceptual-hash clustering.`	Module docstring describing leak-free pHash clustering and 70/15/15 stratified splitting.
2		Blank separator inside module docstring.
3	`Near-duplicate images (pHash Hamming distance <= threshold) are grouped into atomic clusters`	Explains atomic cluster invariant.
4	`so augmented variants or burst photos never leak across train/val/test or lab/field splits.`	Explains why augmented or burst photos never straddle splits.
5		Blank separator.
6	`Usage:`	Usage header.
7	`    python -m ml.data.split --base configs/base.yaml`	Example CLI command.
8	`"""`	Closes module docstring.
9		*Blank visual separator line.*
10	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
11		*Blank visual separator line.*
12	`import argparse`	Imports `argparse` for CLI flags.
13	`from collections import defaultdict`	Imports `defaultdict` for grouping clusters per class and tracking split counts.
14	`from pathlib import Path`	Imports `Path` for filesystem paths.

Line	Source Code	Line-by-Line Explanation
15	`from typing import Any`	Imports `Any` type hint.
16		*Blank visual separator line.*
17	`import numpy as np`	Imports NumPy for vectorized bitwise XOR, argmax, and array ops.
18	`import pandas as pd`	Imports Pandas for Parquet reading/writing and tabular operations.
19		*Blank visual separator line.*
20	`from ml.utils.io import ensure_dir, load_yaml, save_json, save_label_map`	Imports local I/O helpers (`ensure_dir` , `load_yaml` , `save_json` , `save_label_map`).
21	`from ml.utils.seed import seed_everything`	Imports `seed_everything` .
22		*Blank visual separator line.*
23	`# Precomputed 8-bit popcount lookup table for ultra-fast vectorized Hamming distances`	Comment explaining precomputed 256-entry popcount lookup table.
24	`_POPCOUNT_LUT = np.array([bin(i).count("1") for i in range(256)], dtype=np.uint8)`	Builds `_POPCOUNT_LUT[b]` = number of 1-bits in byte `b` (`0..255`) for `O(1)` byte Hamming distance.
25		*Blank visual separator line.*
26		*Blank visual separator line.*
27	`class _UnionFind:`	Defines `_UnionFind` Disjoint Set Union (DSU) data structure.
28	`    def __init__(self, n: int) -> None:`	Initializes `n` singleton sets `0..n-1` .
29	`        self.parent = list(range(n))`	`parent[i] = i` initially.
30	`        self.rank = [0] * n`	`rank[i] = 0` for union-by-rank tree balancing.
31		*Blank visual separator line.*
32	`    def find(self, x: int) -> int:`	Defines `find(x)` returning connected-component root.
33	`        while self.parent[x] != x:`	Iterates upward until reaching self-parent root.
34	`            self.parent[x] = self.parent[self.parent[x]]`	Applies path halving (`parent[x] = parent[parent[x]]`) for near-O(1) amortized lookup.
35	`            x = self.parent[x]`	Moves pointer up to compressed parent.
36	`        return x`	Returns component root.
37		*Blank visual separator line.*
38	`    def union(self, a: int, b: int) -> None:`	Defines `union(a, b)` merging components containing `a` and `b` .
39	`        ra, rb = self.find(a), self.find(b)`	Finds roots of `a` and `b` .
40	`        if ra == rb:`	Early exit if already in same visual cluster.
41	`            return`	Returns None.
42	`        if self.rank[ra] < self.rank[rb]:`	Ensures `ra` is the taller/higher-rank root.
43	`            ra, rb = rb, ra`	Swaps `ra, rb` if needed.
44	`        self.parent[rb] = ra`	Attaches shorter tree `rb` under `ra` .
45	`        if self.rank[ra] == self.rank[rb]:`	Checks tie in tree rank.
46	`            self.rank[ra] += 1`	Increments rank of merged root `ra` .
47		*Blank visual separator line.*
48		*Blank visual separator line.*
49	`def _phash_hex_to_uint64(hex_series: pd.Series) -> np.ndarray:`	Defines `_phash_hex_to_uint64` converting 16-char hex pHash strings to `np.uint64` .
50	`    return np.array([int(str(h), 16) for h in hex_series], dtype=np.uint64)`	Parses base-16 strings into contiguous 64-bit unsigned NumPy array.
51		*Blank visual separator line.*
52		*Blank visual separator line.*
53	`def _hamming_matrix_block(a: np.ndarray, b: np.ndarray) -> np.ndarray:`	Defines vectorized blockwise Hamming matrix `_hamming_matrix_block(a, b)` .
54	`    """Compute pairwise 64-bit Hamming distances between uint64 vectors `a` (M,) and `b` (N,)."""`	Docstring noting shapes `(M,)` and `(N,)` -> `(M, N)` distances.
55	`    xor = np.bitwise_xor(a[:, None], b[None, :])`	Computes broadcasted 64-bit bitwise XOR of shapes `(M, 1) ^ (1, N) -> (M, N)` .
56	`    xor_bytes = xor.view(np.uint8).reshape(a.shape[0], b.shape[0], 8)`	Reinterprets each `uint64` as 8 raw `uint8` bytes -> shape `(M, N, 8)` with zero copy.
57	`    return _POPCOUNT_LUT[xor_bytes].sum(axis=-1)`	Looks up set-bit count per byte via `_POPCOUNT_LUT` and sums across all 8 bytes -> exact Hamming distance.
58		*Blank visual separator line.*
59		*Blank visual separator line.*
60	`def cluster_near_duplicates(`	Defines `cluster_near_duplicates(phash_uint64, threshold=6, chunk_size=1024)` .
61	`    phash_uint64: np.ndarray,`	Parameter: array of 64-bit pHashes.
62	`    threshold: int = 6,`	Parameter: max bit flips (`<= 6` out of 64 bits) treated as near-duplicates.
63	`    chunk_size: int = 1024,`	Parameter: tile size `1024` bounding RAM usage to ~8 MB per chunk.
64	`) -> np.ndarray:`	Returns compact integer cluster IDs `0..G-1` .
65	`    """Assign a connected-component group ID to near-duplicate images via pHash Hamming distance."""`	Docstring summarizing connected-component clustering.
66	`    n = len(phash_uint64)`	Stores dataset size `n` .
67	`    uf = _UnionFind(n)`	Instantiates `_UnionFind(n)` .
68	`    if n <= 1 or threshold < 0:`	Fast-path for empty/single-image datasets or disabled threshold.
69	`        return np.arange(n, dtype=np.int64)`	Returns identity IDs `[0, 1, ..., n-1]` .
70		*Blank visual separator line.*
71	`    for start in range(0, n, chunk_size):`	Loops over upper-triangular row chunks `[start : start+1024]` .
72	`        end = min(start + chunk_size, n)`	Computes exclusive upper chunk boundary.
73	`        block_a = phash_uint64[start:end]`	Slices query block `block_a` .

Line	Source Code	Line-by-Line Explanation
74	`        # Only compare with indices >= start to avoid redundant work`	Comment explaining upper-triangle symmetry optimization.
75	`        block_b = phash_uint64[start:]`	Slices target block `block_b` from `start` to `n` (skips already-checked earlier rows).
76	`        dists = _hamming_matrix_block(block_a, block_b)`	Computes exact pairwise Hamming distance matrix `(M, N-start)`.
77	`        rows, cols = np.where(dists <= threshold)`	Finds all `(row, col)` pairs with Hamming distance `<= threshold`.
78	`        for r, c in zip(rows.tolist(), cols.tolist(), strict=True):`	Iterates over matching near-duplicate pairs.
79	`            i = start + r`	Maps local row offset to global image index `i`.
80	`            j = start + c`	Maps local column offset to global image index `j`.
81	`            if i < j:`	Unions strictly upper-triangular pairs `i < j`.
82	`                uf.union(i, j)`	Merges near-duplicate images `i` and `j` into one cluster.
83		*Blank visual separator line.*
84	`    roots = np.array([uf.find(i) for i in range(n)], dtype=np.int64)`	Resolves final root cluster ID for every image `0..n-1`.
85	`    _, compact_ids = np.unique(roots, return_inverse=True)`	Compresses sparse root IDs into dense sequential cluster IDs `0..num_groups-1`.
86	`    return compact_ids`	Returns compact cluster ID array.
87		*Blank visual separator line.*
88		*Blank visual separator line.*
89	`def _stratified_group_assign(`	Defines `_stratified_group_assign(df, ratios=(0.70, 0.15, 0.15), seed=42)`.
90	`    df: pd.DataFrame,`	Parameter `df`: DataFrame containing `dup_group_id` and `label`.
91	`    ratios: tuple[float, float, float] = (0.70, 0.15, 0.15),`	Parameter `ratios`: target `(train, val, test)` fractions.
92	`    seed: int = 42,`	Parameter `seed`: random tie-break seed.
93	`) -> pd.Series:`	Returns Pandas Series mapping every row to `'train'`, `'val'`, or `'test'`.
94	`    """Assign each `dup_group_id` atomically to train/val/test while preserving class ratios."""`	Docstring explaining atomic group assignment.
95	`    rng = np.random.default_rng(seed)`	Initializes modern NumPy `default_rng(seed)` for deterministic shuffling.
96	`    split_names = ("train", "val", "test")`	Fixed tuple of target split names matching `ratios` order.
97		*Blank visual separator line.*
98	`    # Build atomic groups; majority label defines stratification bucket if group spans classes`	Comment explaining majority-label rule if a rare cluster mixes two visually similar classes.
99	`    group_info: list[tuple[int, str, int]] = []`	List holding `(dup_group_id, majority_label, cluster_size)`.
100	`    for gid, grp in df.groupby("dup_group_id", sort=False):`	Groups rows by atomic `dup_group_id` without re-sorting.
101	`        maj_label = str(grp["label"].mode().iloc[0])`	Finds the most frequent label in that visual cluster.
102	`        group_info.append((int(gid), maj_label, len(grp)))`	Appends cluster tuple.
103		*Blank visual separator line.*
104	`    # Shuffle within equal sizes for randomness, then sort largest groups first per label`	Comment explaining random tie-break followed by greedy descending-size bin-packing.
105	`    rng.shuffle(group_info)`	Shuffles `group_info` randomly so equal-size clusters allocate without ordering bias.
106	`    label_totals = df["label"].value_counts().to_dict()`	Computes total samples per class across `main_df`.
107	`    groups_by_label: dict[str, list[tuple[int, int]]] = defaultdict(list)`	Groups `(gid, size)` clusters by their majority label.
108	`    for gid, lbl, size in group_info:`	Iterates shuffled clusters.
109	`        groups_by_label[lbl].append((gid, size))`	Appends cluster under its label bucket.
110		*Blank visual separator line.*
111	`    group_to_split: dict[int, str] = {}`	Output dictionary mapping `dup_group_id -> 'train'/'val'/'test'`.
112	`    split_class_counts: dict[str, dict[str, int]] = {`	Nested counter tracking current allocated image counts `split_class_counts[split][label]`.
113	`        s: defaultdict(int) for s in split_names`	Initializes zeroed `defaultdict(int)` per split.
114	`    }`	Closes dict comprehension.
115		*Blank visual separator line.*
116	`    for lbl, grp_list in sorted(groups_by_label.items()):`	Processes labels in sorted deterministic order.
117	`        grp_list.sort(key=lambda x: x[1], reverse=True)`	Sorts clusters largest-first so big duplicate bursts place before tiny singletons.
118	`        total_lbl = label_totals[lbl]`	Looks up total image count for this class.
119	`        targets = [r * total_lbl for r in ratios]`	Computes floating-point target image counts `[0.70*N, 0.15*N, 0.15*N]`.
120		*Blank visual separator line.*
121	`        for gid, size in grp_list:`	Assigns each cluster greedily to whichever split currently needs images most.
122	`            # Choose split with largest positive deficit relative to target count`	Comment explaining deficit metric.
123	`            deficits = [`	Computes `target - current` count for train, val, and test.
124	`                targets[idx] - split_class_counts[s][lbl]`	Formula for deficit per split index.
125	`                for idx, s in enumerate(split_names)`	Loops over `(0, 'train'), (1, 'val'), (2, 'test')`.
126	`            ]`	Closes list comprehension.
127	`            best_idx = int(np.argmax(deficits))`	Picks split index maximizing positive deficit.
128	`            chosen = split_names[best_idx]`	Resolves split string name.
129	`            group_to_split[gid] = chosen`	Records assignment for this entire `dup_group_id`.
130	`            split_class_counts[chosen][lbl] += size`	Adds whole cluster's `size` to chosen split's class count.
131		*Blank visual separator line.*
132	`    return df["dup_group_id"].map(group_to_split)`	Broadcasts group-level split back to every image row.
133		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
134		*Blank visual separator line.*
135	`def _compute_class_weights(`	Defines `_compute_class_weights` balancing rare vs. frequent classes.
136	`    counts: dict[str, int],`	Parameter: class sample counts dictionary.
137	`    strategy: str = "inverse_sqrt",`	Parameter: `'inverse_sqrt'`, `'effective_num'`, or `'none'`.
138	`    min_w: float = 0.25,`	Minimum clipped weight (`0.25`).
139	`    max_w: float = 5.0,`	Maximum clipped weight (`5.0`).
140	`) -> dict[str, float]:`	Returns dict `{class_name: float_weight}`.
141	`    if strategy == "none":`	Handles unweighted baseline `'none'`.
142	`        return {k: 1.0 for k in counts}`	Assigns uniform weight `1.0` to every class.
143		*Blank visual separator line.*
144	`    total = sum(counts.values())`	Sums total training samples across classes.
145	`    raw: dict[str, float] = {}`	Holds unnormalized raw per-class weights.
146	`    if strategy == "effective_num":`	Supports Cui et al. (2019) Effective Number of Samples weighting.
147	`        beta = 0.999`	Smoothing hyperparameter `beta = 0.999`.
148	`        for cls_name, cnt in counts.items():`	Loops over classes and sample counts.
149	`            eff = 1.0 - (beta**cnt)`	Computes effective volume `1 - beta^n`.
150	`            raw[cls_name] = (1.0 - beta) / max(eff, 1e-8)`	Computes `(1 - beta) / eff` safely guarded against zero.
151	`    else:  # inverse_sqrt (smooth balance without exploding on tail classes)`	Default `inverse_sqrt` branch (`sqrt(N / n_c)`).
152	`        for cls_name, cnt in counts.items():`	Loops over classes.
153	`            raw[cls_name] = np.sqrt(total / max(cnt, 1))`	Computes smooth `sqrt(total / count)` damping dominance without gradient explosions.
154		*Blank visual separator line.*
155	`    mean_w = float(np.mean(list(raw.values())))`	Computes mean weight across all classes for unit-mean normalization.
156	`    normed = {k: float(np.clip(v / max(mean_w, 1e-8), min_w, max_w)) for k, v in raw.items()}`	Divides by `mean_w` so average loss scale stays `~1.0`, then clips cleanly to `[0.25, 5.0]`.
157	`    return normed`	Returns final weight lookup dict.
158		*Blank visual separator line.*
159		*Blank visual separator line.*
160	`def _audit_leakage(`	Defines `_audit_leakage` proving 0 cross-split overlap post-split.
161	`    train_df: pd.DataFrame,`	Train split DataFrame.
162	`    val_df: pd.DataFrame,`	Val split DataFrame.
163	`    test_df: pd.DataFrame,`	Test split DataFrame.
164	`    field_df: pd.DataFrame,`	External field test DataFrame.
165	`    threshold: int,`	pHash Hamming threshold used during audit.
166	`) -> dict[str, Any]:`	Returns JSON-serializable audit dictionary.
167	`    """Verify zero exact or near-duplicate leakage across splits."""`	Docstring summarizing leakage verification.
168	`    splits = {"train": train_df, "val": val_df, "test": test_df}`	Packs internal splits into dict.
169	`    sha_sets = {k: set(v["sha256"]) for k, v in splits.items()}`	Collects SHA-256 sets per split.
170	`    grp_sets = {k: set(v["dup_group_id"]) for k, v in splits.items()}`	Collects `dup_group_id` sets per split.
171		*Blank visual separator line.*
172	`    sha_overlaps = {`	Checks exact SHA-256 set intersections across `(train, val, test)`.
173	`        "train_val": len(sha_sets["train"] & sha_sets["val"]),`	Exact byte overlap between train & val.
174	`        "train_test": len(sha_sets["train"] & sha_sets["test"]),`	Exact byte overlap between train & test.
175	`        "val_test": len(sha_sets["val"] & sha_sets["test"]),`	Exact byte overlap between val & test.
176	`    }`	Closes dict.
177	`    group_overlaps = {`	Checks `dup_group_id` intersections across splits.
178	`        "train_val": len(grp_sets["train"] & grp_sets["val"]),`	Cluster ID overlap train & val.
179	`        "train_test": len(grp_sets["train"] & grp_sets["test"]),`	Cluster ID overlap train & test.
180	`        "val_test": len(grp_sets["val"] & grp_sets["test"]),`	Cluster ID overlap val & test.
181	`    }`	Closes dict.
182		*Blank visual separator line.*
183	`    # Check cross-split pHash leakage against train (sampled exact check if huge, full if <=50k)`	Comment noting exhaustive cross-split Hamming verification.
184	`    def _cross_phash_hits(df_a: pd.DataFrame, df_b: pd.DataFrame) -> int:`	Defines inner helper `_cross_phash_hits(df_a, df_b)` counting cross-split near-duplicates.
185	`        if df_a.empty or df_b.empty:`	Returns `0` immediately if either split is empty.
186	`            return 0`	Early zero return.
187	`        ha = _phash_hex_to_uint64(df_a["phash_hex"])`	Converts `df_a` hex hashes to `uint64`.
188	`        hb = _phash_hex_to_uint64(df_b["phash_hex"])`	Converts `df_b` hex hashes to `uint64`.
189	`        hits = 0`	Initializes hit counter.
190	`        for s in range(0, len(ha), 1024):`	Chunks `ha` in batches of `1024` for bounded memory.
191	`            dists = _hamming_matrix_block(ha[s : s + 1024], hb)`	Computes pairwise Hamming matrix against all of `df_b`.
192	`            hits += int((dists <= threshold).sum())`	Counts any cross-split pair with distance `<= threshold`.
193	`        return hits`	Returns total cross-split hits (must be `0`).

Line	Source Code	Line-by-Line Explanation
194		*Blank visual separator line.*
195	`    phash_cross_hits = {`	Runs cross-split pHash audit across all 4 pairs including `train_field`.
196	`        "train_val": _cross_phash_hits(train_df, val_df),`	Checks `train` vs `val`.
197	`        "train_test": _cross_phash_hits(train_df, test_df),`	Checks `train` vs `test`.
198	`        "val_test": _cross_phash_hits(val_df, test_df),`	Checks `val` vs `test`.
199	`        "train_field": _cross_phash_hits(train_df, field_df) if not field_df.empty else 0,`	Checks `train` vs `test_field`.
200	`    }`	Closes dict.
201		*Blank visual separator line.*
202	`    return {`	Returns full structured audit summary.
203	`        "phash_threshold": threshold,`	Records threshold used.
204	`        "sha256_overlaps": sha_overlaps,`	Records SHA-256 intersection counts.
205	`        "dup_group_overlaps": group_overlaps,`	Records cluster overlap counts.
206	`        "cross_split_phash_hits": phash_cross_hits,`	Records perceptual cross-split hit counts.
207	`        "leakage_free": (`	Computes boolean `leakage_free` invariant.
208	`            sum(sha_overlaps.values()) == 0`	Requires 0 exact byte overlaps.
209	`            and sum(group_overlaps.values()) == 0`	Requires 0 cluster ID overlaps.
210	`            and sum(phash_cross_hits.values()) == 0`	Requires 0 pHash cross-split near-duplicate hits.
211	`        ),`	Closes boolean expression.
212	`    }`	Closes returned dict.
213		*Blank visual separator line.*
214		*Blank visual separator line.*
215	`def create_splits(cfg: dict[str, Any]) -> dict[str, Any]:`	Defines top-level pipeline `create_splits(cfg)`.
216	`    """Run near-duplicate clustering, leak-free 70/15/15 split, and class weight computation."""`	Docstring summarizing clustering, 70/15/15 split, and weight calculation.
217	`    seed = int(cfg.get("seed", 42))`	Extracts global random seed from `base.yaml`.
218	`    seed_everything(seed)`	Seeds RNGs deterministically.
219		*Blank visual separator line.*
220	`    processed_dir = Path(cfg["paths"]["processed_dir"])`	Reads `data/processed` path.
221	`    manifests_dir = ensure_dir(cfg["paths"]["manifests_dir"])`	Creates `data/manifests` output directory.
222	`    split_cfg = cfg["split"]`	Extracts `split` config section.
223	`    imb_cfg = cfg["imbalance"]`	Extracts `imbalance` config section.
224		*Blank visual separator line.*
225	`    catalog_path = processed_dir / "catalog.parquet"`	Path to cleaned catalog Parquet file.
226	`    if not catalog_path.exists():`	Verifies catalog exists before attempting split.
227	`        raise FileNotFoundError(`	Raises `FileNotFoundError` with actionable instruction.
228	`            f"Catalog not found at {catalog_path}. Run `python -m ml.data.clean_and_merge` first."`	Instructs running `clean_and_merge` first.
229	`        )`	Closes exception.
230		*Blank visual separator line.*
231	`    df = pd.read_parquet(catalog_path)`	Loads clean image catalog into memory.
232	`    threshold = int(split_cfg.get("phash_hamming_threshold", 6))`	Reads `phash_hamming_threshold` (`6`).
233	`    min_samples = int(split_cfg.get("min_samples_per_class", 15))`	Reads `min_samples_per_class` (`15`).
234	`    plantdoc_eval_only = bool(split_cfg.get("plantdoc_eval_only", True))`	Reads `plantdoc_eval_only` (`True`) holding PlantDoc out strictly for field testing.
235		*Blank visual separator line.*
236	`    print(f"[split] Clustering near-duplicates across {len(df)} images (pHash <= {threshold})...")`	Logs number of catalog images being clustered.
237	`    phashes = _phash_hex_to_uint64(df["phash_hex"])`	Converts hex pHash column to 64-bit NumPy vector.
238	`    df["dup_group_id"] = cluster_near_duplicates(phashes, threshold=threshold)`	Assigns `dup_group_id` via connected components.
239	`    num_groups = int(df["dup_group_id"].nunique())`	Counts total distinct visual clusters.
240	`    print(f"[split] Found {num_groups} unique visual clusters from {len(df)} images.")`	Prints compression from total images to unique clusters.
241		*Blank visual separator line.*
242	`    if plantdoc_eval_only and (df["source"] == "plantdoc").any():`	Checks if PlantDoc should be isolated strictly to `test_field.parquet`.
243	`        field_df = df[df["source"] == "plantdoc"].copy().reset_index(drop=True)`	Extracts all PlantDoc rows into `field_df`.
244	`        main_df = df[df["source"] != "plantdoc"].copy().reset_index(drop=True)`	Extracts all non-PlantDoc sources into `main_df` for 70/15/15 splitting.
245	`        # Drop any PlantDoc sample whose pHash cluster overlaps main_df to avoid lab-to-field leakage`	Comment explaining lab-to-field cluster leak purge.
246	`        main_groups = set(main_df["dup_group_id"])`	Builds set of all cluster IDs present in `main_df`.
247	`        before_field = len(field_df)`	Records pre-filter PlantDoc count.
248	`        field_df = field_df[~field_df["dup_group_id"].isin(main_groups)].reset_index(drop=True)`	Drops any PlantDoc image sharing a visual cluster with `main_df`.
249	`        removed_cross = before_field - len(field_df)`	Counts removed overlapping PlantDoc images.
250	`        if removed_cross > 0:`	Logs overlap count if any were dropped.
251	`            print(f"[split] Removed {removed_cross} PlantDoc images overlapping main visual clusters.")`	Prints info message.
252	`    else:`	Branch when `plantdoc_eval_only` is False.

Line	Source Code	Line-by-Line Explanation
253	`        main_df = df.copy().reset_index(drop=True)`	Uses all data in `main_df`.
254	`        field_df = df[df["domain"] == "field"].copy().reset_index(drop=True)`	Copies `domain == 'field'` rows for optional field reporting.
255		*Blank visual separator line.*
256	`    # Filter extremely rare classes that cannot form valid train/val/test splits`	Comment explaining minimum class count filter.
257	`    cls_counts = main_df["label"].value_counts()`	Counts images per canonical class in `main_df`.
258	`    dropped_classes = cls_counts[cls_counts < min_samples].index.tolist()`	Collects classes having `< 15` samples.
259	`    if dropped_classes:`	If tiny classes exist, removes them so val/test splits are never empty.
260	`        print(f"[split] Dropping {len(dropped_classes)} classes with < {min_samples} images: {dropped_classes}")`	Logs dropped tiny classes.
261	`        main_df = main_df[~main_df["label"].isin(dropped_classes)].reset_index(drop=True)`	Filters `main_df` to classes meeting minimum threshold.
262		*Blank visual separator line.*
263	`    ratios = (`	Constructs `(0.70, 0.15, 0.15)` split ratio tuple from config.
264	`        float(split_cfg.get("train_ratio", 0.70)),`	Reads `train_ratio`.
265	`        float(split_cfg.get("val_ratio", 0.15)),`	Reads `val_ratio`.
266	`        float(split_cfg.get("test_ratio", 0.15)),`	Reads `test_ratio`.
267	`    )`	Closes tuple.
268	`    main_df["split"] = _stratified_group_assign(main_df, ratios=ratios, seed=seed)`	Runs stratified cluster-level assignment producing `'split'` column.
269		*Blank visual separator line.*
270	`    train_df = main_df[main_df["split"] == "train"].reset_index(drop=True)`	Slices training DataFrame.
271	`    val_df = main_df[main_df["split"] == "val"].reset_index(drop=True)`	Slices validation DataFrame.
272	`    test_df = main_df[main_df["split"] == "test"].reset_index(drop=True)`	Slices ID test DataFrame.
273		*Blank visual separator line.*
274	`    # Build canonical label map strictly from training classes`	Comment: canonical sorted class indices originate purely from `train` labels.
275	`    classes = sorted(train_df["label"].unique().tolist())`	Sorts unique class names alphabetically (`0..C-1`).
276	`    label_payload = save_label_map(classes, manifests_dir / "label_map.json")`	Writes `data/manifests/label_map.json` and returns mapping.
277	`    class_to_idx: dict[str, int] = label_payload["class_to_idx"]`	Extracts `{label: int_index}` lookup dict.
278		*Blank visual separator line.*
279	`    for split_df in (train_df, val_df, test_df):`	Loops over train, val, and test DataFrames.
280	`        split_df["target"] = split_df["label"].map(class_to_idx)`	Maps string label to integer `target` column.
281		*Blank visual separator line.*
282	`    if not field_df.empty:`	Processes external `field_df` if present.
283	`        field_df = field_df[field_df["label"].isin(class_to_idx)].copy().reset_index(drop=True)`	Retains only PlantDoc labels present in trained model's taxonomy.
284	`        field_df["target"] = field_df["label"].map(class_to_idx)`	Maps `target` indices matching training taxonomy.
285	`        field_df["split"] = "test_field"`	Tags split name as `'test_field'`.
286		*Blank visual separator line.*
287	`    train_counts = train_df["label"].value_counts().to_dict()`	Counts per-class samples in `train_df` only (zero val/test peeking).
288	`    class_weights = _compute_class_weights(`	Computes balanced weights using configured strategy and `[0.25, 5.0]` clamp.
289	`        counts={c: int(train_counts.get(c, 1)) for c in classes},`	Ensures every class in `classes` has at least count `1`.
290	`        strategy=str(imb_cfg.get("weight_strategy", "inverse_sqrt")),`	Passes weight strategy (`inverse_sqrt`).
291	`        min_w=float(imb_cfg.get("min_weight", 0.25)),`	Passes lower weight bound (`0.25`).
292	`        max_w=float(imb_cfg.get("max_weight", 5.0)),`	Passes upper weight bound (`5.0`).
293	`    )`	Closes `_compute_class_weights` call.
294	`    save_json(class_weights, manifests_dir / "class_weights.json")`	Saves `data/manifests/class_weights.json`.
295		*Blank visual separator line.*
296	`    train_df.to_parquet(manifests_dir / "train.parquet", index=False)`	Persists `train.parquet`.
297	`    val_df.to_parquet(manifests_dir / "val.parquet", index=False)`	Persists `val.parquet`.
298	`    test_df.to_parquet(manifests_dir / "test.parquet", index=False)`	Persists `test.parquet`.
299	`    field_df.to_parquet(manifests_dir / "test_field.parquet", index=False)`	Persists `test_field.parquet` (possibly 0 rows if PlantDoc absent).
300		*Blank visual separator line.*
301	`    audit = _audit_leakage(train_df, val_df, test_df, field_df, threshold=threshold)`	Runs full cross-split zero-leakage verification audit.
302	`    audit["counts"] = {`	Attaches split sizes and metadata summary into audit payload.
303	`        "train": len(train_df),`	Records train size.
304	`        "val": len(val_df),`	Records val size.
305	`        "test": len(test_df),`	Records test size.
306	`        "test_field": len(field_df),`	Records real-field test size.
307	`        "num_classes": len(classes),`	Records total class count.
308	`        "num_visual_clusters": num_groups,`	Records unique pHash cluster count.
309	`        "dropped_tiny_classes": dropped_classes,`	Lists dropped classes.
310	`        "plantdoc_eval_only": plantdoc_eval_only,`	Records `plantdoc_eval_only` flag.
311	`    }`	Closes `counts` dict.
312	`    save_json(audit, manifests_dir / "split_audit.json")`	Writes `data/manifests/split_audit.json`.
313		*Blank visual separator line.*
314	`    print(`	Logs final split counts and `leakage_free` boolean.

Line	Source Code	Line-by-Line Explanation
315	`        f"[split] Done -> train={len(train_df)}, val={len(val_df)}, "`	First half of log format string.
316	`        f"test={len(test_df)}, test_field={len(field_df)} | leakage_free={audit['leakage_free']}"`	Second half showing `test_field` and `leakage_free` status.
317	`    )`	Closes print.
318	`    return audit`	Returns `audit` summary dictionary.
319		*Blank visual separator line.*
320		*Blank visual separator line.*
321	`def parse_args() -> argparse.Namespace:`	Defines CLI parser `parse_args()`.
322	`    parser = argparse.ArgumentParser(description="Create leak-free stratified 70/15/15 splits.")`	Initializes `ArgumentParser`.
323	`    parser.add_argument("--base", default="configs/base.yaml", help="Base config YAML path.")`	Adds `--base` flag defaulting to `configs/base.yaml`.
324	`    return parser.parse_args()`	Returns parsed args.
325		*Blank visual separator line.*
326		*Blank visual separator line.*
327	`def main() -> None:`	Defines CLI entrypoint `main()`.
328	`    args = parse_args()`	Parses CLI flags.
329	`    cfg = load_yaml(args.base)`	Loads base YAML configuration.
330	`    create_splits(cfg)`	Executes `create_splits(cfg)`.
331		*Blank visual separator line.*
332		*Blank visual separator line.*
333	`if __name__ == "__main__":`	Standard script entry guard.
334	`    main()`	Invokes `main()`.
335		*Blank visual separator line.*

7.4 PyTorch Dataset & Weighted Sampler — ml/data/dataset.py

File Summary: Reads Parquet manifests into memory, loads RGB images with EXIF orientation correction, applies Albumentations transforms, and builds WeightedRandomSampler for imbalance handling.

ml/data/dataset.py (Lines 1–91)

Line	Source Code	Line-by-Line Explanation
1	`"""PyTorch Dataset and weighted sampler utilities."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from collections.abc import Callable`	Imports `Callable` abstract type.
6	`from pathlib import Path`	Imports `Path`.
7	`from typing import Any`	Imports `Any`.
8		*Blank visual separator line.*
9	`import numpy as np`	Imports NumPy for converting decoded PIL images to HWC `uint8` arrays.
10	`import pandas as pd`	Imports Pandas for reading Parquet split manifests.
11	`import torch`	Imports PyTorch core.
12	`from PIL import Image, ImageOps`	Imports `Image` and `ImageOps` (for EXIF auto-rotation).
13	`from torch.utils.data import Dataset, WeightedRandomSampler`	Imports PyTorch `Dataset` base class and `WeightedRandomSampler`.
14		*Blank visual separator line.*
15		*Blank visual separator line.*
16	`class CropDiseaseDataset(Dataset):`	Defines `CropDiseaseDataset(Dataset)`.
17	`    """PyTorch Dataset backed by Parquet manifests with EXIF-safe RGB loading."""`	Docstring explaining EXIF-safe RGB Parquet manifest loading.
18		*Blank visual separator line.*
19	`    def __init__(`	Constructor `__init__`.
20	`        self,`	`self` reference.
21	`        manifest: str | Path | pd.DataFrame,`	`manifest`: accepts either an in-memory `DataFrame` or path to a `.parquet` file.
22	`        class_to_idx: dict[str, int] | None = None,`	`class_to_idx`: fallback label-to-index map if manifest lacks numeric `target`.
23	`        transform: Callable[..., dict[str, Any]] | None = None,`	`transform`: Albumentations pipeline taking `image=<ndarray>` and returning `{'image': Tensor}`.
24	`        return_meta: bool = False,`	`return_meta`: if `True`, returns `(tensor, target, metadata_dict)` for evaluation/Grad-CAM.
25	`    ) -> None:`	Return type `None`.
26	`        if isinstance(manifest, pd.DataFrame):`	Checks if caller passed a preloaded `DataFrame`.
27	`            self.df = manifest.reset_index(drop=True)`	Resets row index cleanly to `0..N-1`.
28	`        else:`	Otherwise treats `manifest` as a file path.
29	`            self.df = pd.read_parquet(manifest).reset_index(drop=True)`	Loads `.parquet` file via PyArrow/Pandas and resets index.
30		*Blank visual separator line.*
31	`        self.transform = transform`	Stores Albumentations transform callable.

Line	Source Code	Line-by-Line Explanation
32	`        self.return_meta = return_meta`	Stores `return_meta` flag.
33		*Blank visual separator line.*
34	`        if "target" in self.df.columns:`	Checks if precomputed integer `'target'` column exists.
35	`            self.targets = self.df["target"].astype(int).tolist()`	Extracts integer class IDs as Python `list[int]` for fast `__getitem__` indexing.
36	`        elif class_to_idx is not None:`	Otherwise maps string labels using `class_to_idx`.
37	`            self.targets = [int(class_to_idx[lbl]) for lbl in self.df["label"]]`	List comprehension resolving integer IDs.
38	`        else:`	Fallback when used without targets (e.g., OOD scoring on unlabeled negatives).
39	`            self.targets = [0] * len(self.df)`	Fills dummy `0` targets.
40		*Blank visual separator line.*
41	`        self.paths = self.df["image_path"].astype(str).tolist()`	Extracts all `image_path` values into plain Python list (avoids Pandas overhead in workers).
42	`        self.labels = (`	Stores canonical class label list if column exists.
43	`            self.df["label"].astype(str).tolist()`	Reads `'label'` column.
44	`            if "label" in self.df.columns`	Column existence check.
45	`            else ["unknown"] * len(self.df)`	Fallback `'unknown'` list.
46	`        )`	Closes conditional expression.
47	`        self.domains = (`	Stores domain tags (`'lab'` vs. `'field'`) for domain-shift evaluation.
48	`            self.df["domain"].astype(str).tolist()`	Reads `'domain'` column.
49	`            if "domain" in self.df.columns`	Column existence check.
50	`            else ["unknown"] * len(self.df)`	Fallback `'unknown'` list.
51	`        )`	Closes conditional expression.
52		*Blank visual separator line.*
53	`    def __len__(self) -> int:`	Implements `Dataset.__len__` returning dataset sample count.
54	`        return len(self.paths)`	Length of cached `self.paths`.
55		*Blank visual separator line.*
56	`    def __getitem__(self, idx: int) -> tuple[Any, ...] | tuple[torch.Tensor, int]:`	Implements `Dataset.__getitem__(idx)` called by DataLoader workers.
57	`        path = self.paths[idx]`	Looks up image path for index `idx`.
58	`        with Image.open(path) as img_raw:`	Opens image lazily from disk inside `with` context to avoid leaked file descriptors.
59	`            img = ImageOps.exif_transpose(img_raw).convert("RGB")`	Fixes EXIF camera rotation and enforces 3-channel RGB mode.
60	`            img_np = np.array(img)`	Converts PIL Image to `(H, W, 3)` NumPy `uint8` array required by Albumentations.
61		*Blank visual separator line.*
62	`        if self.transform is not None:`	Applies Albumentations transform pipeline if supplied.
63	`            tensor = self.transform(image=img_np)["image"]`	Extracts transformed `CHW float32` PyTorch tensor.
64	`        else:`	Fallback when no transform is provided.
65	`            tensor = torch.from_numpy(img_np).permute(2, 0, 1).float() / 255.0`	Transposes `HWC -> CHW` and scales `[0, 255]` into `[0.0, 1.0]` tensor.
66		*Blank visual separator line.*
67	`        target = self.targets[idx]`	Reads cached integer target ID.
68	`        if self.return_meta:`	If `return_meta=True`, returns 3-tuple including provenance metadata.
69	`            return tensor, target, {`	Starts 3-tuple return.
70	`                "path": path,`	Original image file path.
71	`                "label": self.labels[idx],`	String canonical label.
72	`                "domain": self.domains[idx],`	Source domain (`lab` / `field`).
73	`            }`	Closes metadata dict.
74	`        return tensor, target`	Standard 2-tuple `(tensor, target)` used during training/validation loops.
75		*Blank visual separator line.*
76		*Blank visual separator line.*
77	`def build_weighted_sampler(`	Defines `build_weighted_sampler(df, class_weights)` for epoch-level oversampling of rare diseases.
78	`    df: pd.DataFrame,`	`df`: training DataFrame.
79	`    class_weights: dict[str, float],`	`class_weights`: normalized per-class weights dict.
80	`) -> WeightedRandomSampler:`	Return type `WeightedRandomSampler`.
81	`    """Create a PyTorch WeightedRandomSampler aligned with per-class imbalance weights."""`	Docstring.
82	`    sample_weights = np.array(`	Maps each training row to its class's inverse-sqrt sampling weight.
83	`        [float(class_weights.get(lbl, 1.0)) for lbl in df["label"]],`	Looks up per-sample weight (defaults to `1.0`).
84	`        dtype=np.float64,`	Uses `float64` precision for multinomial sampling stability.
85	`    )`	Closes array.
86	`    return WeightedRandomSampler(`	Constructs PyTorch `WeightedRandomSampler`.
87	`        weights=torch.from_numpy(sample_weights),`	Passes sample weight tensor.
88	`        num_samples=len(sample_weights),`	Keeps epoch length equal to original dataset size `N`.
89	`        replacement=True,`	Samples with replacement so minority disease images appear multiple times per epoch.
90	`    )`	Closes sampler instantiation.
91		*Blank visual separator line.*

7.5 Unified CNN & Transformer Backbone Factory — ml/models/factory.py

File Summary: Wraps timm backbones behind a unified interface exposing dual outputs `(logits, NCHW_spatial_map)` across CNNs (ConvNeXt/EfficientNetV2/MobileNetV3), Swin NHWC maps, and ViT patch tokens, plus two-stage freezing and discriminative LR parameter groups.

ml/models/factory.py (Lines 1–179)

Line	Source Code	Line-by-Line Explanation
1	`"""Backbone factory wrapping timm models (EfficientNetV2-S, ConvNeXt-Tiny, Swin-Tiny, ViT-Small).`	Module docstring listing supported timm architectures.
2		Blank separator.
3	`Every wrapped model exposes:`	Summary header.
4	`- `forward(x) -> logits` for standard training and calibration`	Documents standard `forward(x)` signature.
5	`- `forward_with_features(x) -> (logits, spatial_feature_map_nchw)` for ONNX export & Grad-CAM`	Documents dual-output `forward_with_features(x)` for Grad-CAM & ONNX.
6	`- Two-stage freeze/unfreeze and discriminative learning-rate parameter groups`	Documents two-stage freezing & discriminative LR grouping.
7	`"""`	Closes module docstring.
8		*Blank visual separator line.*
9	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
10		*Blank visual separator line.*
11	`import math`	Imports `math` (`math.isqrt` to infer ViT token grid dimensions).
12	`from pathlib import Path`	Imports `Path` .
13	`from typing import Any`	Imports `Any` .
14		*Blank visual separator line.*
15	`import timm`	Imports `timm` (PyTorch Image Models library).
16	`import torch`	Imports `torch` .
17	`import torch.nn as nn`	Imports `torch.nn` .
18		*Blank visual separator line.*
19		*Blank visual separator line.*
20	`class CropClassifier(nn.Module):`	Defines `CropClassifier(nn.Module)` unifying CNNs and Vision Transformers.
21	`    """Unified classifier wrapper over timm CNN and Transformer backbones."""`	Class docstring.
22		*Blank visual separator line.*
23	`    def __init__(`	Constructor `__init__` .
24	`        self,`	`self` reference.
25	`        backbone_name: str,`	`backbone_name` : timm model identifier (e.g. `convnext_tiny.fb_in22k_ft_in1k`).
26	`        num_classes: int,`	`num_classes` : number of target crop/disease classes.
27	`        pretrained: bool = True,`	`pretrained` : downloads ImageNet-1k/22k pretrained weights when True.
28	`        drop_rate: float = 0.3,`	`drop_rate` : classifier dropout probability (`0.3` default).
29	`        drop_path_rate: float = 0.2,`	`drop_path_rate` : stochastic depth survival decay (`0.2` default).
30	`    ) -> None:`	Return type `None` .
31	`        super().__init__()`	Initializes `nn.Module` base.
32	`        self.backbone_name = backbone_name`	Saves `backbone_name` for checkpoint metadata.
33	`        self.num_classes = num_classes`	Saves `num_classes` .
34		*Blank visual separator line.*
35	`        self.backbone = timm.create_model(`	Instantiates timm feature backbone.
36	`            backbone_name,`	Model registry string.
37	`            pretrained=pretrained,`	Loads pretrained weights when training from scratch/fine-tuning.
38	`            num_classes=0,`	`num_classes=0` removes timm's stock linear head so `forward_head(..., pre_logits=True)` returns pooled embeddings.
39	`            drop_rate=drop_rate,`	Passes head dropout rate.
40	`            drop_path_rate=drop_path_rate,`	Passes stochastic depth drop-path rate.
41	`        )`	Closes `timm.create_model` .
42	`        self.num_features: int = int(self.backbone.num_features)`	Reads embedding channel count `C` exposed by timm.
43	`        self.dropout = nn.Dropout(p=drop_rate) if drop_rate > 0 else nn.Identity()`	Creates Dropout layer before final linear head (or `Identity` if `0`).
44	`        self.classifier = nn.Linear(self.num_features, num_classes)`	Builds linear classification layer `C -> num_classes` .
45		*Blank visual separator line.*
46	`        nn.init.trunc_normal_(self.classifier.weight, std=0.02)`	Initializes classifier weights via truncated normal (`std=0.02` , ViT/ConvNeXt best practice).
47	`        if self.classifier.bias is not None:`	Checks classifier bias presence.
48	`            nn.init.zeros_(self.classifier.bias)`	Zero-initializes classifier bias.
49		*Blank visual separator line.*
50	`    def _to_nchw_spatial_map(self, feats: torch.Tensor) -> torch.Tensor:`	Defines `_to_nchw_spatial_map(feats)` normalizing heterogeneous backbone outputs into `(B, C, H, W)` .
51	`        """Convert backbone features (NCHW, NHWC for Swin, or BLC tokens for ViT) to NCHW."""`	Docstring describing CNN vs Swin vs ViT layouts.
52	`        if feats.ndim == 4:`	Checks for 4D spatial tensor.
53	`            # Swin returns (B, H, W, C); CNNs & ConvNeXt return (B, C, H, W)`	Comment noting Swin outputs `(B, H, W, C)` (`NHWC`) while CNNs output `(B, C, H, W)` (`NCHW`).

Line	Source Code	Line-by-Line Explanation
54	`            if feats.shape[1] != self.num_features and feats.shape[-1] == self.num_features:`	Detects `NHWC` when channel dim `shape[-1]` matches `num_features` while `shape[1]` does not.
55	`                return feats.permute(0, 3, 1, 2).contiguous()`	Permutes `NHWC -> NCHW` and makes tensor memory-contiguous.
56	`            return feats`	Returns CNN/ConvNeXt `NCHW` map directly.
57		*Blank visual separator line.*
58	`        if feats.ndim == 3:`	Checks for 3D transformer sequence tensor `(B, L, C)` (e.g. `ViT-Small`).
59	`            num_prefix = int(getattr(self.backbone, "num_prefix_tokens", 1))`	Queries number of non-spatial prefix tokens (`1` for `[CLS]` token; `0` if absent).
60	`            patch_tokens = feats[:, num_prefix:, :]`	Slices off prefix token(s), keeping only spatial patch tokens.
61	`            n_tokens = patch_tokens.shape[1]`	Reads total patch count `N = H_p * W_p` (e.g., `196` for `14x14` grid).
62	`            side = int(math.isqrt(n_tokens))`	Computes integer square root `side` (`14`).
63	`            if side * side != n_tokens:`	Safety fallback if non-square aspect ratio breaks `side * side == N`.
64	`                # Fallback to 1x1 pooled spatial map if non-square tokens occur`	Comment explaining 1x1 fallback.
65	`                return patch_tokens.mean(dim=1).unsqueeze(-1).unsqueeze(-1)`	Global-average-pools tokens into `(B, C, 1, 1)`.
66	`            return (`	Reshapes `(B, side*side, C)` -> `(B, side, side, C)` and permutes to `(B, C, side, side)`.
67	`                patch_tokens.reshape(feats.shape[0], side, side, self.num_features)`	Reshape dimensions.
68	`                .permute(0, 3, 1, 2)`	Permute channels forward (`NCHW`).
69	`                .contiguous()`	Ensure contiguous memory layout for ONNX export.
70	`            )`	Closes return expression.
71		*Blank visual separator line.*
72	`        # Fallback for 2D (B, C) tensors`	Comment for rare 2D feature vector fallback.
73	`        return feats.unsqueeze(-1).unsqueeze(-1)`	Expands `(B, C)` -> `(B, C, 1, 1)`.
74		*Blank visual separator line.*
75	`    def forward_with_features(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:`	Defines `forward_with_features(x)` returning both `(logits, spatial_map_nchw)`.
76	`        """Return `(logits, spatial_feature_map_nchw)` for dual-output ONNX export and Grad-CAM."""`	Docstring explaining ONNX and Grad-CAM usage.
77	`        raw_feats = self.backbone.forward_features(x)`	Extracts unpooled backbone spatial features.
78	`        pooled = self.backbone.forward_head(raw_feats, pre_logits=True)`	Pools `raw_feats` using timm's native pooling/norm (`pre_logits=True`) into `(B, C)`.
79	`        logits = self.classifier(self.dropout(pooled))`	Applies dropout and linear classifier head producing logits `(B, K)`.
80	`        spatial_map = self._to_nchw_spatial_map(raw_feats)`	Normalizes spatial features to `(B, C, H_f, W_f)`.
81	`        return logits, spatial_map`	Returns `(logits, spatial_map)` tuple.
82		*Blank visual separator line.*
83	`    def forward(self, x: torch.Tensor) -> torch.Tensor:`	Defines standard PyTorch `forward(x)` returning logits only.
84	`        """Forward pass returning classification logits."""`	Docstring.
85	`        logits, _ = self.forward_with_features(x)`	Reuses `forward_with_features` and discards spatial map.
86	`        return logits`	Returns `(B, K)` logits tensor.
87		*Blank visual separator line.*
88	`    def freeze_backbone(self) -> None:`	Defines `freeze_backbone()` for Stage 1 head warmup.
89	`        """Freeze all backbone weights so only the classifier head trains (Stage 1)."""`	Docstring.
90	`        for param in self.backbone.parameters():`	Iterates through all backbone parameters.
91	`            param.requires_grad = False`	Locks `requires_grad = False` so backbone weights & BN running shifts don't destabilize from random head.
92	`        for param in self.classifier.parameters():`	Iterates through linear classifier head parameters.
93	`            param.requires_grad = True`	Enables gradients on classifier head.
94		*Blank visual separator line.*
95	`    def unfreeze_backbone(self) -> None:`	Defines `unfreeze_backbone()` for Stage 2 full fine-tuning.
96	`        """Unfreeze the entire model for end-to-end discriminative fine-tuning (Stage 2)."""`	Docstring.
97	`        for param in self.parameters():`	Iterates all model parameters.
98	`            param.requires_grad = True`	Unlocks `requires_grad = True` everywhere.
99		*Blank visual separator line.*
100	`    def get_param_groups(`	Defines `get_param_groups(backbone_lr, head_lr, weight_decay)`.
101	`        self,`	`self` reference.
102	`        backbone_lr: float,`	Lower LR for pretrained backbone layers (e.g., `1e-4`).
103	`        head_lr: float,`	Higher LR for task classifier head (e.g., `5e-4`).
104	`        weight_decay: float,`	AdamW L2 decoupled weight decay (`0.01`).
105	`    ) -> list[dict[str, Any]]:`	Returns list of PyTorch optimizer parameter group dicts.
106	`        """Build AdamW parameter groups with discriminative LRs and no-decay on norms/biases."""`	Docstring explaining discriminative LRs + norm/bias weight-decay exclusion.
107	`        no_decay_keywords = ("bias", "norm", "bn", "ln", "gamma", "beta")`	Tuple of parameter name substrings that must NOT receive weight decay.
108		*Blank visual separator line.*
109	`        groups: dict[tuple[str, str], list[nn.Parameter]] = {`	Initializes 4 parameter buckets: `{backbone, head} x {decay, no_decay}`.
110	`            ("backbone", "decay"): [],`	Backbone weight matrices bucket.
111	`            ("backbone", "no_decay"): [],`	Backbone 1D biases / LayerNorm / BatchNorm gamma & beta bucket.

Line	Source Code	Line-by-Line Explanation
112	`            ("head", "decay"): [],`	Classifier weight matrix bucket.
113	`            ("head", "no_decay"): [],`	Classifier bias bucket.
114	`        }`	Closes `groups` dict.
115		*Blank visual separator line.*
116	`        for name, param in self.backbone.named_parameters():`	Iterates named parameters of `self.backbone`.
117	`            if not param.requires_grad:`	Skips frozen Stage-1 parameters automatically (`not requires_grad`).
118	`                continue`	Continue if frozen.
119	`            is_no_decay = param.ndim <= 1 or any(k in name.lower() for k in no_decay_keywords)`	Marks parameter as `no_decay` if 1D/0D OR name contains `bias` / `norm` / `bn` / `ln` / `gamma` / `beta`.
120	`            bucket = "no_decay" if is_no_decay else "decay"`	Selects bucket key.
121	`            groups[("backbone", bucket)].append(param)`	Appends parameter to backbone bucket.
122		*Blank visual separator line.*
123	`        for name, param in self.classifier.named_parameters():`	Iterates named parameters of `self.classifier`.
124	`            if not param.requires_grad:`	Skips frozen parameters.
125	`                continue`	Continue.
126	`            is_no_decay = param.ndim <= 1 or name.endswith("bias")`	Marks classifier bias (`1D`) as `no_decay`.
127	`            bucket = "no_decay" if is_no_decay else "decay"`	Selects bucket key.
128	`            groups[("head", bucket)].append(param)`	Appends parameter to head bucket.
129		*Blank visual separator line.*
130	`        param_groups: list[dict[str, Any]] = []`	Output list for AdamW constructor.
131	`        lr_map = {"backbone": backbone_lr, "head": head_lr}`	Maps part name to target learning rate.
132	`        for (part, decay_kind), params in groups.items():`	Builds non-empty parameter groups with appropriate LR and `weight_decay`.
133	`            if params:`	Skips empty buckets (e.g., all backbone buckets during Stage 1).
134	`                param_groups.append(`	Appends dict matching PyTorch Optimizer API.
135	`                    {`	Opens group dict.
136	`                        "params": params,`	List of `nn.Parameter` tensors.
137	`                        "lr": lr_map[part],`	Learning rate (`backbone_lr` or `head_lr`).
138	`                        "weight_decay": 0.0 if decay_kind == "no_decay" else weight_decay,`	Sets `0.0` weight decay for `no_decay` norms/biases, `weight_decay` otherwise.
139	`                        "group_name": f"{part}_{decay_kind}",`	Human-readable tag (`backbone_decay` , etc.).
140	`                    }`	Closes dict.
141	`                )`	Closes `append`.
142	`        return param_groups`	Returns constructed optimizer groups.
143		*Blank visual separator line.*
144		*Blank visual separator line.*
145	`def create_model(`	Defines convenience factory `create_model(...)`.
146	`    backbone_name: str,`	`backbone_name` arg.
147	`    num_classes: int,`	`num_classes` arg.
148	`    pretrained: bool = True,`	`pretrained` arg.
149	`    drop_rate: float = 0.3,`	`drop_rate` arg.
150	`    drop_path_rate: float = 0.2,`	`drop_path_rate` arg.
151	`) -> CropClassifier:`	Return annotation.
152	`    """Instantiate a CropClassifier from config parameters."""`	Docstring.
153	`    return CropClassifier(`	Constructs and returns `CropClassifier` instance.
154	`        backbone_name=backbone_name,`	Forwards `backbone_name`.
155	`        num_classes=num_classes,`	Forwards `num_classes`.
156	`        pretrained=pretrained,`	Forwards `pretrained`.
157	`        drop_rate=drop_rate,`	Forwards `drop_rate`.
158	`        drop_path_rate=drop_path_rate,`	Forwards `drop_path_rate`.
159	`    )`	Closes call.
160		*Blank visual separator line.*
161		*Blank visual separator line.*
162	`def load_checkpoint_model(`	Defines `load_checkpoint_model(checkpoint_path, device='cpu')`.
163	`    checkpoint_path: str | Path,`	Path to saved `.pt` checkpoint.
164	`    device: torch.device | str = "cpu",`	Target PyTorch device.
165	`) -> tuple[CropClassifier, dict[str, Any]]:`	Returns `(eval_model, ckpt_metadata_dict)`.
166	`    """Load a saved training checkpoint (`best.pt`) and return `(model, metadata)`."""`	Docstring.
167	`    ckpt = torch.load(Path(checkpoint_path), map_location=device, weights_only=False)`	Loads checkpoint safely to target device (`weights_only=False` needed for Python metadata lists/dicts).
168	`    model = create_model(`	Reconstructs exact `CropClassifier` architecture without redownloading ImageNet weights (`pretrained=False`) or dropout randomness.
169	`        backbone_name=ckpt["backbone"],`	Reads saved timm backbone name.
170	`        num_classes=int(ckpt["num_classes"]),`	Reads saved class count.
171	`        pretrained=False,`	Skips network download since weights come from checkpoint.
172	`        drop_rate=float(ckpt.get("drop_rate", 0.0)),`	Reads saved `drop_rate`.

Line	Source Code	Line-by-Line Explanation
173	`        drop_path_rate=0.0,`	Sets `drop_path_rate=0.0` for inference.
174	`    )`	Closes `create_model`.
175	`    model.load_state_dict(ckpt["model_state_dict"], strict=True)`	Strictly loads trained weights (`strict=True` fails loudly on any layer mismatch).
176	`    model.to(device)`	Moves model parameters onto target device.
177	`    model.eval()`	Switches BatchNorm and Dropout layers to deterministic evaluation mode.
178	`    return model, ckpt`	Returns `(model, ckpt)`.
179		*Blank visual separator line.*

7.6 Class-Weighted Soft Cross-Entropy & Knowledge Distillation Loss — ml/models/losses.py

File Summary: Implements WeightedSoftTargetCrossEntropy (compatible with MixUp/CutMix soft probability vectors + per-class imbalance weights) and Hinton Knowledge Distillation KL + CE loss.

`ml/models/losses.py (Lines 1–74)`

Line	Source Code	Line-by-Line Explanation
1	`"""Loss functions supporting class imbalance weights, soft MixUp/CutMix targets, and KD."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`import torch`	Imports PyTorch.
6	`import torch.nn as nn`	Imports `nn` module base classes.
7	`import torch.nn.functional as F`	Imports `nn.functional as F` for `log_softmax`, `softmax`, and `kl_div`.
8		*Blank visual separator line.*
9		*Blank visual separator line.*
10	`class WeightedSoftTargetCrossEntropy(nn.Module):`	Defines `WeightedSoftTargetCrossEntropy(nn.Module)` for MixUp/CutMix soft labels.
11	`    """Cross-entropy loss for soft MixUp/CutMix probability targets with optional class weights."""`	Docstring.
12		*Blank visual separator line.*
13	`    def __init__(self, class_weights: torch.Tensor | None = None) -> None:`	Constructor accepting optional 1D `class_weights` tensor `(K,)`.
14	`        super().__init__()`	Initializes `nn.Module`.
15	`        if class_weights is not None:`	Checks if weights tensor was supplied.
16	`            self.register_buffer("class_weights", class_weights.float())`	Registers `class_weights` as a non-trainable buffer so `.to(device)` moves it automatically.
17	`        else:`	Else branch when unweighted.
18	`            self.class_weights = None`	Stores `None`.
19		*Blank visual separator line.*
20	`    def forward(self, logits: torch.Tensor, target_probs: torch.Tensor) -> torch.Tensor:`	Defines `forward(logits, target_probs)` where `target_probs` has shape `(B, K)`.
21	`        log_probs = F.log_softmax(logits, dim=-1)`	Computes numerically stable log-probabilities `log(softmax(z))`.
22	`        if self.class_weights is not None:`	Checks if per-class imbalance weights are active.
23	`            # Weight each sample by its soft-target mixture of class weights`	Comment explaining weighted soft target expectation.
24	`            sample_w = (target_probs * self.class_weights.unsqueeze(0)).sum(dim=-1)`	Computes per-sample effective weight `sum_c target_probs[b,c] * class_weights[c]`.
25	`            per_sample_loss = -(target_probs * log_probs).sum(dim=-1)`	Computes cross-entropy `-sum_c y_c * log(p_c)` per sample.
26	`            return (per_sample_loss * sample_w).sum() / sample_w.sum().clamp_min(1e-8)`	Returns weighted mean normalized by sum of sample weights (clamped at `1e-8` against zero division).
27	`        return -(target_probs * log_probs).sum(dim=-1).mean()`	Unweighted batch-mean soft cross-entropy.
28		*Blank visual separator line.*
29		*Blank visual separator line.*
30	`class DistillationLoss(nn.Module):`	Defines `DistillationLoss(nn.Module)` combining hard-label CE and teacher-student KL divergence.
31	`    """Knowledge distillation combining ground-truth CE and KL divergence against teacher logits."""`	Docstring.
32		*Blank visual separator line.*
33	`    def __init__(`	Constructor with `temperature=4.0`, `alpha_kd=0.5`, `class_weights`, and `label_smoothing=0.05`.
34	`        self,`	`self` reference.
35	`        temperature: float = 4.0,`	Distillation softmax temperature `tau` softening dark-knowledge logits.
36	`        alpha_kd: float = 0.5,`	Convex blend coefficient between hard CE `(1 - alpha)` and KL `(alpha)`.
37	`        class_weights: torch.Tensor | None = None,`	Optional class weights for hard ground-truth loss.
38	`        label_smoothing: float = 0.05,`	Label smoothing on hard targets.
39	`    ) -> None:`	Return `None`.
40	`        super().__init__()`	Initializes `nn.Module`.
41	`        self.temperature = temperature`	Stores temperature `tau`.
42	`        self.alpha_kd = alpha_kd`	Stores `alpha_kd`.

Line	Source Code	Line-by-Line Explanation
43	`        self.ce = nn.CrossEntropyLoss(`	Builds standard `nn.CrossEntropyLoss` for ground-truth supervision.
44	`            weight=class_weights.float() if class_weights is not None else None,`	Applies class weights if provided.
45	`            label_smoothing=label_smoothing,`	Applies label smoothing.
46	`        )`	Closes `CrossEntropyLoss`.
47		*Blank visual separator line.*
48	`    def forward(`	Defines `forward(student_logits, teacher_logits, targets)`.
49	`        self,`	`self` reference.
50	`        student_logits: torch.Tensor,`	MobileNetV3 student logits.
51	`        teacher_logits: torch.Tensor,`	Frozen ConvNeXt/EfficientNetV2 teacher logits.
52	`        targets: torch.Tensor,`	Ground-truth integer labels `(B,)`.
53	`    ) -> torch.Tensor:`	Returns scalar distillation loss.
54	`        hard_loss = self.ce(student_logits, targets)`	Computes ground-truth supervised loss `L_CE`.
55	`        t = self.temperature`	Local alias `t = self.temperature`.
56	`        student_log_probs = F.log_softmax(student_logits / t, dim=-1)`	Computes student temperature-scaled log-probabilities `log(softmax(z_S / T))`.
57	`        teacher_probs = F.softmax(teacher_logits / t, dim=-1)`	Computes teacher soft probability targets `softmax(z_T / T)`.
58	`        kd_loss = F.kl_div(student_log_probs, teacher_probs, reduction="batchmean") * (t * t)`	Computes `KL(teacher || student)` scaled by `T^2` to preserve gradient magnitude (`(1/T)^2` factor in softmax Jacobian).
59	`        return (1.0 - self.alpha_kd) * hard_loss + self.alpha_kd * kd_loss`	Returns convex combination `(1 - alpha)*L_CE + alpha*T^2*KL`.
60		*Blank visual separator line.*
61		*Blank visual separator line.*
62	`def build_loss_fn(`	Defines factory `build_loss_fn(class_weights, label_smoothing, soft_targets)`.
63	`    class_weights: torch.Tensor | None = None,`	Optional weight tensor.
64	`    label_smoothing: float = 0.1,`	Label smoothing (`0.1`).
65	`    soft_targets: bool = False,`	Switches between soft MixUp targets vs hard integer targets.
66	`) -> nn.Module:`	Returns configured `nn.Module` criterion.
67	`    """Create hard-label or soft-target cross-entropy loss with optional class weights."""`	Docstring.
68	`    if soft_targets:`	If `soft_targets=True`, returns custom `WeightedSoftTargetCrossEntropy` (supports smoothed/mixed labels).
69	`        return WeightedSoftTargetCrossEntropy(class_weights=class_weights)`	Instantiates soft-target CE.
70	`    return nn.CrossEntropyLoss(`	Otherwise returns PyTorch's native `nn.CrossEntropyLoss` for validation or unmixed training.
71	`        weight=class_weights.float() if class_weights is not None else None,`	Forwards class weights.
72	`        label_smoothing=label_smoothing,`	Forwards label smoothing.
73	`    )`	Closes `CrossEntropyLoss`.
74		*Blank visual separator line.*

7.7 Binary Plant-vs-NonPlant OOD Gate — ml/models/plant_gate.py

File Summary: Trains a compact MobileNetV3-Small binary gate distinguishing crop/pest photos from CIFAR-10 non-plant negatives, selects the best validation ROC-AUC checkpoint, and exports `artifacts/plant_gate.onnx`.

ml/models/plant_gate.py (Lines 1–186)

Line	Source Code	Line-by-Line Explanation
1	`"""Lightweight binary Plant/Pest vs. Non-Plant gate for out-of-distribution rejection.`	Module docstring.
2		Blank separator.
3	`Trains a compact MobileNetV3-Small classifier distinguishing agricultural crop/pest images`	Describes MobileNetV3-Small binary classifier role.
4	`from non-plant negatives (data/manifests/ood_negatives.parquet), and exports `plant_gate.onnx`.`	Notes negative manifest (`ood_negatives.parquet`) and ONNX export path.
5		Blank separator.
6	`Usage:`	Usage header.
7	`    python -m ml.models.plant_gate --base configs/base.yaml --epochs 4`	CLI invocation example.
8	`"""`	Closes docstring.
9		*Blank visual separator line.*
10	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
11		*Blank visual separator line.*
12	`import argparse`	Imports `argparse`.
13	`from pathlib import Path`	Imports `Path`.
14	`from typing import Any`	Imports `Any`.
15		*Blank visual separator line.*
16	`import numpy as np`	Imports NumPy.
17	`import pandas as pd`	Imports Pandas.
18	`import timm`	Imports `timm`.
19	`import torch`	Imports PyTorch.
20	`import torch.nn as nn`	Imports `nn`.

Line	Source Code	Line-by-Line Explanation
21	`from PIL import Image, ImageOps`	Imports `Image`, `ImageOps`.
22	`from sklearn.metrics import roc_auc_score`	Imports `roc_auc_score` to measure threshold-free binary discrimination quality.
23	`from torch.utils.data import DataLoader, Dataset`	Imports `DataLoader`, `Dataset`.
24	`from tqdm import tqdm`	Imports `tqdm` progress bar.
25		*Blank visual separator line.*
26	`from ml.training.augmentations import build_eval_transforms, build_train_transforms`	Imports train & eval Albumentations transforms.
27	`from ml.utils.io import ensure_dir, load_yaml, save_json`	Imports YAML/JSON helpers.
28	`from ml.utils.seed import get_default_device, seed_everything`	Imports device and seed helpers.
29		*Blank visual separator line.*
30		*Blank visual separator line.*
31	`class PlantGateNet(nn.Module):`	Defines `PlantGateNet(nn.Module)` binary classifier.
32	`    """Compact binary classifier returning scalar logit for `is_plant_or_pest`."""`	Docstring noting scalar output logit `is_plant_or_pest`.
33		*Blank visual separator line.*
34	`    def __init__(`	Constructor with default pretrained `mobilenetv3_small_100.lamb_in1k`.
35	`        self,`	`self` reference.
36	`        backbone_name: str = "mobilenetv3_small_100.lamb_in1k",`	Backbone name.
37	`        pretrained: bool = True,`	Pretrained toggle.
38	`    ) -> None:`	Returns `None`.
39	`        super().__init__()`	Calls `super().__init__()`.
40	`        self.backbone_name = backbone_name`	Stores backbone string.
41	`        self.model = timm.create_model(`	Creates compact MobileNetV3-Small with 1 output unit (`num_classes=1`) and `drop_rate=0.2`.
42	`            backbone_name,`	Backbone ID.
43	`            pretrained=pretrained,`	Pretrained weights.
44	`            num_classes=1,`	Single logit output for BCE loss.
45	`            drop_rate=0.2,`	20% classifier dropout.
46	`        )`	Closes model instantiation.
47		*Blank visual separator line.*
48	`    def forward(self, x: torch.Tensor) -> torch.Tensor:`	Defines forward pass `forward(x)`.
49	`        return self.model(x).squeeze(-1)`	Squeezes trailing dim `(B, 1) -> (B,)` matching `BCEWithLogitsLoss`.
50		*Blank visual separator line.*
51		*Blank visual separator line.*
52	`class _BinaryImageDataset(Dataset):`	Defines lightweight internal `_BinaryImageDataset(Dataset)`.
53	`    def __init__(self, paths: list[str], labels: list[float], transform: Any) -> None:`	Takes file paths, float binary labels (`1.0` =plant, `0.0` =non-plant), and Albumentations transform.
54	`        self.paths = paths`	Stores paths.
55	`        self.labels = labels`	Stores float targets.
56	`        self.transform = transform`	Stores transform pipeline.
57		*Blank visual separator line.*
58	`    def __len__(self) -> int:`	Implements `__len__`.
59	`        return len(self.paths)`	Returns path count.
60		*Blank visual separator line.*
61	`    def __getitem__(self, idx: int) -> tuple[torch.Tensor, torch.Tensor]:`	Implements `__getitem__(idx)`.
62	`        with Image.open(self.paths[idx]) as raw:`	Opens image safely.
63	`            img = ImageOps.exif_transpose(raw).convert("RGB")`	EXIF-rotates and converts to RGB.
64	`            arr = np.array(img)`	Converts to `uint8` array.
65	`        tensor = self.transform(image=arr)["image"]`	Applies transform pipeline returning `CHW` tensor.
66	`        return tensor, torch.tensor(self.labels[idx], dtype=torch.float32)`	Returns `(tensor, float32_scalar_target)`.
67		*Blank visual separator line.*
68		*Blank visual separator line.*
69	`def train_plant_gate(`	Defines `train_plant_gate(cfg, epochs=4, batch_size=64, max_pos_samples=4000)`.
70	`    cfg: dict[str, Any],`	Base config dict.
71	`    epochs: int = 4,`	Training epochs (4 is ample for binary plant vs object separation).
72	`    batch_size: int = 64,`	Mini-batch size.
73	`    max_pos_samples: int = 4000,`	Caps positive crop samples to `4000` so training stays fast and balanced with CIFAR-10 negatives.
74	`) -> Path | None:`	Returns path to exported `plant_gate.onnx` or `None`.
75	`    """Train and export a fast binary Plant-vs-NonPlant gate to `artifacts/plant_gate.onnx`."""`	Docstring.
76	`    seed = int(cfg.get("seed", 42))`	Reads random seed.
77	`    seed_everything(seed)`	Seeds RNGs.
78		*Blank visual separator line.*
79	`    manifests_dir = Path(cfg["paths"]["manifests_dir"])`	Reads `data/manifests` dir.
80	`    artifacts_dir = ensure_dir(cfg["paths"]["artifacts_dir"])`	Ensures `artifacts/` dir exists.

Line	Source Code	Line-by-Line Explanation
81	`    ood_path = manifests_dir / "ood_negatives.parquet"`	Path to OOD negative manifest.
82	`    train_path = manifests_dir / "train.parquet"`	Path to positive training manifest.
83	`    val_path = manifests_dir / "val.parquet"`	Path to positive validation manifest.
84		*Blank visual separator line.*
85	`    if not ood_path.exists() or not train_path.exists():`	Gracefully skips gate training if OOD negatives were not downloaded.
86	`        print("[plant_gate] Missing train.parquet or ood_negatives.parquet; skipping gate training.")`	Prints skip warning.
87	`        return None`	Returns `None`.
88		*Blank visual separator line.*
89	`    train_df = pd.read_parquet(train_path)`	Loads positive train manifest.
90	`    val_df = pd.read_parquet(val_path)`	Loads positive val manifest.
91	`    ood_df = pd.read_parquet(ood_path).sample(frac=1.0, random_state=seed).reset_index(drop=True)`	Loads and shuffles OOD negatives deterministically.
92		*Blank visual separator line.*
93	`    # Split OOD negatives 80/20 into gate train/val`	Comment: 80/20 split of OOD negatives into train/val.
94	`    n_ood_val = max(100, int(0.2 * len(ood_df)))`	Reserves 20% (at least `100`) of OOD images for validation.
95	`    ood_val = ood_df.iloc[:n_ood_val]`	Validation negatives slice.
96	`    ood_train = ood_df.iloc[n_ood_val:]`	Training negatives slice.
97		*Blank visual separator line.*
98	`    pos_train = train_df.sample(n=min(max_pos_samples, len(train_df)), random_state=seed)`	Subsamples up to `max_pos_samples` positive crop images from training split.
99	`    pos_val = val_df.sample(n=min(max(300, n_ood_val), len(val_df)), random_state=seed)`	Subsamples balanced positive crop validation images.
100		*Blank visual separator line.*
101	`    img_size = int(cfg["image"].get("size", 224))`	Reads target image size (`224`).
102	`    mean = list(cfg["image"]["mean"])`	Reads ImageNet channel means.
103	`    std = list(cfg["image"]["std"])`	Reads ImageNet channel stds.
104		*Blank visual separator line.*
105	`    tr_paths = pos_train["image_path"].tolist() + ood_train["image_path"].tolist()`	Concatenates positive and negative training image paths.
106	`    tr_labels = [1.0] * len(pos_train) + [0.0] * len(ood_train)`	Assigns `1.0` to crop images and `0.0` to OOD negatives.
107	`    va_paths = pos_val["image_path"].tolist() + ood_val["image_path"].tolist()`	Concatenates validation image paths.
108	`    va_labels = [1.0] * len(pos_val) + [0.0] * len(ood_val)`	Builds validation float labels.
109		*Blank visual separator line.*
110	`    tr_ds = _BinaryImageDataset(tr_paths, tr_labels, build_train_transforms(img_size, mean, std))`	Creates training `_BinaryImageDataset` with full field augmentations.
111	`    va_ds = _BinaryImageDataset(va_paths, va_labels, build_eval_transforms(img_size, mean, std))`	Creates deterministic validation `_BinaryImageDataset`.
112		*Blank visual separator line.*
113	`    tr_loader = DataLoader(tr_ds, batch_size=batch_size, shuffle=True, num_workers=2)`	Creates shuffled training `DataLoader`.
114	`    va_loader = DataLoader(va_ds, batch_size=batch_size, shuffle=False, num_workers=2)`	Creates validation `DataLoader`.
115		*Blank visual separator line.*
116	`    device = get_default_device()`	Selects CUDA GPU (if compatible) or CPU.
117	`    model = PlantGateNet(pretrained=True).to(device)`	Instantiates `PlantGateNet` and moves it to device.
118	`    pos_weight = torch.tensor([len(ood_train) / max(len(pos_train), 1)], device=device)`	Computes positive class weight `N_neg / N_pos` correcting binary class imbalance.
119	`    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)`	Creates weighted Binary Cross-Entropy with Logits loss.
120	`    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)`	Creates `AdamW` optimizer (`lr=3e-4`, `weight_decay=0.01`).
121		*Blank visual separator line.*
122	`    best_auc = 0.0`	Initializes best validation ROC-AUC tracker.
123	`    best_state: dict[str, Any] | None = None`	Initializes CPU state dict snapshot placeholder.
124		*Blank visual separator line.*
125	`    for epoch in range(1, epochs + 1):`	Runs training loop for `epochs` epochs.
126	`        model.train()`	Switches model to training mode (Dropout + BN updates).
127	`        for x, y in tqdm(tr_loader, desc=f"PlantGate Epoch {epoch}/{epochs}", leave=False):`	Iterates mini-batches with `tqdm` progress bar.
128	`            x, y = x.to(device), y.to(device)`	Moves inputs and targets onto compute device.
129	`            optimizer.zero_grad()`	Zeros gradients.
130	`            loss = criterion(model(x), y)`	Forward pass + weighted BCE loss.
131	`            loss.backward()`	Backpropagates gradients.
132	`            optimizer.step()`	Updates weights via AdamW.
133		*Blank visual separator line.*
134	`        model.eval()`	Switches model to evaluation mode.
135	`        probs, targets = [], []`	Accumulates predicted probabilities and true binary labels.
136	`        with torch.no_grad():`	Disables gradient tracking during validation.
137	`            for x, y in va_loader:`	Loops over validation batches.
138	`                p = torch.sigmoid(model(x.to(device))).cpu().numpy()`	Applies sigmoid `sigma(logit)` yielding plant probability `P(plant | x)`.
139	`                probs.extend(p.tolist())`	Appends probabilities.

Line	Source Code	Line-by-Line Explanation
140	`                targets.extend(y.numpy().tolist())`	Appends binary targets.
141	`        auc = float(roc_auc_score(targets, probs))`	Computes validation ROC-AUC score (`1.0` = perfect separation).
142	`        print(f"[plant_gate] Epoch {epoch}: val ROC-AUC = {auc:.4f}")`	Prints epoch ROC-AUC.
143	`        if auc >= best_auc:`	Checks if current epoch beat or matched best AUC.
144	`            best_auc = auc`	Updates `best_auc` .
145	`            best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}`	Snapshots weights safely to CPU memory.
146		*Blank visual separator line.*
147	`    if best_state is not None:`	Restores best validation weights after loop ends.
148	`        model.load_state_dict(best_state)`	Loads `best_state` .
149	`    model.cpu().eval()`	Moves model to CPU and sets eval mode for clean ONNX tracing.
150		*Blank visual separator line.*
151	`    ckpt_path = artifacts_dir / "plant_gate.pt"`	Path for PyTorch gate checkpoint.
152	`    torch.save({"model_state_dict": model.state_dict(), "val_auc": best_auc}, ckpt_path)`	Saves PyTorch checkpoint and `val_auc` .
153		*Blank visual separator line.*
154	`    onnx_path = artifacts_dir / "plant_gate.onnx"`	Output path `artifacts/plant_gate.onnx` .
155	`    dummy = torch.randn(1, 3, img_size, img_size)`	Creates dummy `(1, 3, 224, 224)` tracer input.
156	`    torch.onnx.export(`	Exports `PlantGateNet` to ONNX graph opset 17 with dynamic batch axis.
157	`        model,`	Model instance.
158	`        dummy,`	Dummy tensor.
159	`        str(onnx_path),`	Output file path.
160	`        input_names=["image"],`	Names ONNX input node `'image'` .
161	`        output_names=["plant_logit"],`	Names ONNX output node `'plant_logit'` .
162	`        dynamic_axes={"image": {0: "batch"}, "plant_logit": {0: "batch"}},`	Marks batch dim `0` dynamic.
163	`        opset_version=17,`	Targets ONNX Opset 17.
164	`    )`	Closes export call.
165	`    save_json({"val_auc": best_auc, "onnx_path": str(onnx_path)}, artifacts_dir / "plant_gate_meta.json")`	Saves gate metadata (`val_auc` , `onnx_path`) to `plant_gate_meta.json` .
166	`    print(f"[plant_gate] Exported {onnx_path} (val AUC={best_auc:.4f}).")`	Logs exported path and final AUC.
167	`    return onnx_path`	Returns `onnx_path` .
168		*Blank visual separator line.*
169		*Blank visual separator line.*
170	`def parse_args() -> argparse.Namespace:`	Defines CLI parser `parse_args()` .
171	`    parser = argparse.ArgumentParser(description="Train binary Plant/Pest vs. Non-Plant OOD gate.")`	Creates `ArgumentParser` .
172	`    parser.add_argument("--base", default="configs/base.yaml", help="Base config YAML.")`	Adds `--base` flag.
173	`    parser.add_argument("--epochs", type=int, default=4)`	Adds `--epochs` flag (`4`).
174	`    parser.add_argument("--batch-size", type=int, default=64)`	Adds `--batch-size` flag (`64`).
175	`    return parser.parse_args()`	Returns parsed args.
176		*Blank visual separator line.*
177		*Blank visual separator line.*
178	`def main() -> None:`	CLI entrypoint `main()` .
179	`    args = parse_args()`	Parses CLI arguments.
180	`    cfg = load_yaml(args.base)`	Loads base YAML.
181	`    train_plant_gate(cfg, epochs=args.epochs, batch_size=args.batch_size)`	Runs `train_plant_gate` .
182		*Blank visual separator line.*
183		*Blank visual separator line.*
184	`if __name__ == "__main__":`	Standard module execution guard.
185	`    main()`	Invokes `main()` .
186		*Blank visual separator line.*

Chapter 8: Training, Calibration, OOD, Explainability & ONNX Export Code (Line-by-Line)

This chapter provides verbatim code and line-by-line explanations for Albumentations augmentations, MixUp/CutMix, the two-stage mixed-precision EMA trainer, L-BFGS temperature scaling, Helmholtz energy OOD thresholding, PyTorch-vs-ONNX Grad-CAM verification, and dual-output ONNX/INT8 export.

8.1 Field-Realistic Augmentations & MixUp/CutMix — ml/training/augmentations.py

File Summary: Constructs Albumentations spatial/photometric/noise transforms bridging PlantVillage lab leaves to messy outdoor lighting, plus GPU-friendly batch MixUp & CutMix soft-target mixing.

ml/training/augmentations.py (Lines 1–167)

Line	Source Code	Line-by-Line Explanation
1	`"""Albumentations field-realistic image augmentations and batch MixUp/CutMix collator."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from typing import Sequence`	Imports `Sequence` typing protocol.
6		*Blank visual separator line.*
7	`import albumentations as A`	Imports Albumentations library as `A`.
8	`import numpy as np`	Imports NumPy.
9	`import torch`	Imports PyTorch.
10	`from albumentations.pytorch import ToTensorV2`	Imports `ToTensorV2` converting HWC arrays to CHW PyTorch tensors.
11		*Blank visual separator line.*
12		*Blank visual separator line.*
13	`def _make_random_resized_crop(image_size: int) -> A.BasicTransform:`	Defines version-safe helper `_make_random_resized_crop(image_size)` (handles Albumentations v1 vs v2 APIs).
14	`    try:`	Tries Albumentations v2.x `size=(h, w)` signature first.
15	`        return A.RandomResizedCrop(`	Constructs `A.RandomResizedCrop` with `65%–100%` crop scale and `0.8–1.25` aspect ratio.
16	`            size=(image_size, image_size),`	Target `(H, W)` tuple.
17	`            scale=(0.65, 1.0),`	Zoom scale range.
18	`            ratio=(0.8, 1.25),`	Aspect ratio jitter.
19	`            p=1.0,`	Always applied (`p=1.0`) so output is guaranteed `(224, 224)`.
20	`        )`	Closes v2 constructor.
21	`    except TypeError:`	Catches `TypeError` on older Albumentations v1.x.
22	`        return A.RandomResizedCrop(`	Falls back to v1 `height=` / `width=` keyword arguments.
23	`            height=image_size,`	Sets height.
24	`            width=image_size,`	Sets width.
25	`            scale=(0.65, 1.0),`	Zoom scale range.
26	`            ratio=(0.8, 1.25),`	Aspect jitter.
27	`            p=1.0,`	Probability `1.0`.
28	`        )`	Closes fallback constructor.
29		*Blank visual separator line.*
30		*Blank visual separator line.*
31	`def _make_coarse_dropout(image_size: int) -> A.BasicTransform:`	Defines version-safe `_make_coarse_dropout(image_size)` simulating random leaf occlusion.
32	`    max_hole = max(12, int(image_size * 0.14))`	Scales max cutout hole size proportionally (`14%` of image width -> ~ `31px` on `224px`).
33	`    try:`	Tries Albumentations v2 range signature.
34	`        return A.CoarseDropout(`	Drops `1–4` rectangular holes of `8..max_hole` pixels with `25%` probability.
35	`            num_holes_range=(1, 4),`	Hole count range.
36	`            hole_height_range=(8, max_hole),`	Hole height range.
37	`            hole_width_range=(8, max_hole),`	Hole width range.
38	`            p=0.25,`	Applies occlusion to 25% of training images.
39	`        )`	Closes v2 call.
40	`    except TypeError:`	Catches `TypeError` for older Albumentations.
41	`        return A.CoarseDropout(p=0.25)`	Falls back to default `CoarseDropout(p=0.25)`.
42		*Blank visual separator line.*
43		*Blank visual separator line.*
44	`def build_train_transforms(`	Defines `build_train_transforms(image_size=224, mean=..., std=...)`.
45	`    image_size: int = 224,`	Target spatial side length (`224`).
46	`    mean: Sequence[float] = (0.485, 0.456, 0.406),`	ImageNet RGB channel means.
47	`    std: Sequence[float] = (0.229, 0.224, 0.225),`	ImageNet RGB channel stds.
48	`) -> A.Compose:`	Returns `A.Compose` transform pipeline.
49	`    """Build Albumentations training pipeline simulating real-field lighting, noise, and angles."""`	Docstring.

Line	Source Code	Line-by-Line Explanation
50	`    return A.Compose(`	Wraps list of sequential transforms in `A.Compose`.
51	`        [`	Starts transform list.
52	`            _make_random_resized_crop(image_size),`	Applies RandomResizedCrop to `(224, 224)`.
53	`            A.HorizontalFlip(p=0.5),`	Random left-right mirror (`p=0.5`); leaves are bilaterally symmetric.
54	`            A.VerticalFlip(p=0.3),`	Random top-bottom flip (`p=0.3`) handling arbitrary camera orientation.
55	`            A.RandomRotate90(p=0.5),`	Random discrete 90-degree rotation (`p=0.5`).
56	`            A.Affine(`	Continuous geometric perturbation (`p=0.6`): zoom `±10%`, shift `±5%`, rotate `±25°`.
57	`                scale=(0.90, 1.10),`	Affine scale bounds.
58	`                translate_percent=(-0.05, 0.05),`	Translation bounds.
59	`                rotate=(-25, 25),`	Rotation angle bounds.
60	`                p=0.6,`	Probability `0.6`.
61	`            ),`	Closes `Affine`.
62	`            A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05, p=0.6),`	Color jitter (`p=0.6`) simulating camera white-balance shifts and seasonal leaf tint.
63	`            A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.4),`	Exposure variation (`p=0.4`) simulating harsh sun vs cloudy shade.
64	`            A.RandomShadow(p=0.2),`	Synthesizes cast shadows across the leaf (`p=0.2`) directly combating lab-only backgrounds.
65	`            A.OneOf(`	Applies either slight Gaussian defocus OR camera shake motion blur (`p=0.25`).
66	`                [`	Starts `OneOf` blur block.
67	`                    A.GaussianBlur(blur_limit=(3, 5), p=1.0),`	Symmetric `3x3` – `5x5` Gaussian defocus.
68	`                    A.MotionBlur(blur_limit=(3, 5), p=1.0),`	Directional motion blur.
69	`                ],`	Closes blur list.
70	`                p=0.25,`	Overall blur trigger probability `25%`.
71	`            ),`	Closes `OneOf`.
72	`            A.OneOf(`	Applies realistic smartphone sensor grain (`p=0.25`).
73	`                [`	Starts `OneOf` noise block.
74	`                    A.GaussNoise(p=1.0),`	Additive Gaussian sensor noise.
75	`                    A.ISONoise(p=1.0),`	Realistic chrominance/luminance ISO noise.
76	`                ],`	Closes noise list.
77	`                p=0.25,`	Overall noise probability `25%`.
78	`            ),`	Closes `OneOf`.
79	`            _make_coarse_dropout(image_size),`	Applies CoarseDropout rectangular occlusion.
80	`            A.Normalize(mean=list(mean), std=list(std)),`	Normalizes `[0,255]` RGB pixels by `(x/255 - mean) / std`.
81	`            ToTensorV2(),`	Permutes `HWC -> CHW` and casts to `torch.FloatTensor`.
82	`        ]`	Closes transform list.
83	`    )`	Closes `A.Compose`.
84		*Blank visual separator line.*
85		*Blank visual separator line.*
86	`def build_eval_transforms(`	Defines deterministic `build_eval_transforms(image_size=224, mean=..., std=...)`.
87	`    image_size: int = 224,`	Target crop side (`224`).
88	`    mean: Sequence[float] = (0.485, 0.456, 0.406),`	ImageNet mean.
89	`    std: Sequence[float] = (0.229, 0.224, 0.225),`	ImageNet std.
90	`) -> A.Compose:`	Returns `A.Compose`.
91	`    """Build deterministic validation/test preprocessing pipeline."""`	Docstring.
92	`    resize_dim = int(round(image_size * 1.14))`	Computes standard `256` intermediate resize dim (`round(224 * 1.14)`) for `87.5%` center crop.
93	`    return A.Compose(`	Wraps deterministic eval transforms in `A.Compose`.
94	`        [`	Starts list.
95	`            A.Resize(resize_dim, resize_dim),`	Resizes image cleanly to `(256, 256)`.
96	`            A.CenterCrop(image_size, image_size),`	Takes deterministic center `224x224` crop (MUST match backend `InferenceService.preprocess`).
97	`            A.Normalize(mean=list(mean), std=list(std)),`	Applies ImageNet mean/std normalization.
98	`            ToTensorV2(),`	Converts to `CHW` PyTorch tensor.
99	`        ]`	Closes list.
100	`    )`	Closes `A.Compose`.
101		*Blank visual separator line.*
102		*Blank visual separator line.*
103	`class MixupCutmixCollator:`	Defines batch collator `MixupCutmixCollator` mixing images + targets directly on GPU tensors.
104	`    """Batch-level MixUp and CutMix producing soft label distributions safely for any batch size."""`	Docstring noting support for arbitrary batch sizes.
105		*Blank visual separator line.*
106	`    def __init__(`	Constructor configuring MixUp & CutMix hyperparameters.
107	`        self,`	`self` reference.
108	`        num_classes: int,`	Total disease class count `K`.
109	`        mixup_alpha: float = 0.2,`	Beta distribution alpha for MixUp (`0.2`).

Line	Source Code	Line-by-Line Explanation
110	`        cutmix_alpha: float = 1.0,`	Beta distribution alpha for CutMix (`1.0`).
111	`        prob: float = 0.5,`	Probability of applying batch mixing vs returning unmixed smoothed labels (`0.5`).
112	`        switch_prob: float = 0.5,`	Probability of choosing CutMix over MixUp when triggered (`0.5`).
113	`        label_smoothing: float = 0.1,`	Uniform label smoothing epsilon (`0.1`).
114	`    ) -> None:`	Returns `None` .
115	`        self.num_classes = num_classes`	Stores `num_classes` .
116	`        self.mixup_alpha = mixup_alpha`	Stores `mixup_alpha` .
117	`        self.cutmix_alpha = cutmix_alpha`	Stores `cutmix_alpha` .
118	`        self.prob = prob`	Stores trigger probability.
119	`        self.switch_prob = switch_prob`	Stores CutMix-vs-MixUp switch probability.
120	`        self.label_smoothing = label_smoothing`	Stores smoothing epsilon.
121		*Blank visual separator line.*
122	`    def _one_hot_smooth(self, targets: torch.Tensor) -> torch.Tensor:`	Defines `_one_hot_smooth(targets)` converting integer IDs `(B,)` into label-smoothed probability vectors `(B, K)` .
123	`        off_value = self.label_smoothing / self.num_classes`	Off-target probability mass `epsilon / K` .
124	`        on_value = 1.0 - self.label_smoothing + off_value`	True-class probability mass `1 - epsilon + epsilon / K` .
125	`        soft = torch.full(`	Allocates filled `(B, K)` tensor on `targets.device` .
126	`            (targets.shape[0], self.num_classes),`	Shape tuple.
127	`            fill_value=off_value,`	Fills every cell with `off_value` .
128	`            dtype=torch.float32,`	Float32 dtype.
129	`            device=targets.device,`	Matches GPU/CPU device.
130	`        )`	Closes `torch.full` .
131	`        soft.scatter_(1, targets.unsqueeze(1).long(), on_value)`	Scatters `on_value` into the true class column of each row.
132	`        return soft`	Returns smoothed `(B, K)` target tensor.
133		*Blank visual separator line.*
134	`    def __call__(self, images: torch.Tensor, targets: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:`	Defines `__call__(images, targets)` transforming a mini-batch.
135	`        soft_targets = self._one_hot_smooth(targets)`	Always converts integer targets into smoothed `(B, K)` vectors first.
136	`        batch_size, _, height, width = images.shape`	Reads batch dimensions `(B, C, H, W)` .
137	`        if batch_size < 2 or np.random.rand() > self.prob:`	Skips mixing if batch size < 2 or random coin exceeds `self.prob` .
138	`            return images, soft_targets`	Returns unmixed images with label-smoothed soft targets.
139		*Blank visual separator line.*
140	`        perm = torch.randperm(batch_size, device=images.device)`	Generates random permutation of batch indices `0..B-1` directly on GPU.
141	`        use_cutmix = self.cutmix_alpha > 0 and (`	Decides between CutMix (rectangular patch swap) vs MixUp (alpha pixel blend).
142	`            self.mixup_alpha <= 0 or np.random.rand() < self.switch_prob`	Uses CutMix if `cutmix_alpha > 0` and coin flip passes.
143	`        )`	Closes boolean check.
144		*Blank visual separator line.*
145	`        if use_cutmix:`	Executes CutMix branch.
146	`            lam = float(np.random.beta(self.cutmix_alpha, self.cutmix_alpha))`	Samples area ratio `lambda ~ Beta(alpha, alpha)` .
147	`            cut_rat = np.sqrt(1.0 - lam)`	Computes patch side fraction `sqrt(1 - lambda)` .
148	`            cut_w = int(width * cut_rat)`	Computes patch pixel width.
149	`            cut_h = int(height * cut_rat)`	Computes patch pixel height.
150	`            cx = int(np.random.randint(width))`	Samples random box center X coordinate.
151	`            cy = int(np.random.randint(height))`	Samples random box center Y coordinate.
152	`            x1 = int(np.clip(cx - cut_w // 2, 0, width))`	Clamps left box edge inside `[0, width]` .
153	`            x2 = int(np.clip(cx + cut_w // 2, 0, width))`	Clamps right box edge inside `[0, width]` .
154	`            y1 = int(np.clip(cy - cut_h // 2, 0, height))`	Clamps top box edge inside `[0, height]` .
155	`            y2 = int(np.clip(cy + cut_h // 2, 0, height))`	Clamps bottom box edge inside `[0, height]` .
156		*Blank visual separator line.*
157	`            mixed = images.clone()`	Clones batch tensor before in-place patch paste.
158	`            mixed[:, :, y1:y2, x1:x2] = images[perm, :, y1:y2, x1:x2]`	Pastes rectangular patch from permuted donor image onto host image.
159	`            lam_adj = 1.0 - ((x2 - x1) * (y2 - y1) / float(width * height))`	Recomputes exact area-adjusted `lambda_adj` after edge clamping.
160	`            mixed_targets = lam_adj * soft_targets + (1.0 - lam_adj) * soft_targets[perm]`	Linearly blends soft target distributions according to exact visible pixel area.
161	`            return mixed, mixed_targets`	Returns CutMix `(mixed_images, mixed_targets)` .
162		*Blank visual separator line.*
163	`        lam = float(np.random.beta(self.mixup_alpha, self.mixup_alpha))`	Executes MixUp branch: samples blend weight `lambda ~ Beta(alpha, alpha)` .
164	`        mixed = lam * images + (1.0 - lam) * images[perm]`	Linearly interpolates pixels: `lambda*x_i + (1-lambda)*x_pi(i)` .
165	`        mixed_targets = lam * soft_targets + (1.0 - lam) * soft_targets[perm]`	Linearly interpolates corresponding smoothed labels.
166	`        return mixed, mixed_targets`	Returns MixUp `(mixed_images, mixed_targets)` .
167		*Blank visual separator line.*

8.2 Two-Stage AMP + EMA + Cosine Warmup Trainer — ml/training/trainer.py

File Summary: Implements step-level cosine+warmup scheduling, mixed-precision evaluation, Stage 1 frozen-head warmup, Stage 2 discriminative fine-tuning with gradient clipping, ModelEmaV2 shadow weights, and macro-F1 early stopping.

ml/training/trainer.py (Lines 1–378)

Line	Source Code	Line-by-Line Explanation
1	`"""Two-stage trainer with AMP, cosine LR + warmup, EMA, gradient clipping, and early stopping."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`import math`	Imports `math` (`math.cos` , `math.pi`).
6	`import time`	Imports `time` for wall-clock training duration tracking.
7	`from collections.abc import Callable`	Imports `Callable` for optional epoch hooks (e.g., Optuna pruning).
8	`from pathlib import Path`	Imports `Path` .
9	`from typing import Any`	Imports `Any` .
10		*Blank visual separator line.*
11	`import numpy as np`	Imports NumPy.
12	`import torch`	Imports PyTorch.
13	`import torch.nn as nn`	Imports `torch.nn` .
14	`from sklearn.metrics import accuracy_score, f1_score`	Imports `accuracy_score` and macro-averaged `f1_score` .
15	`from timm.utils import ModelEmaV2`	Imports timm's `ModelEmaV2` Exponential Moving Average parameter shadow.
16	`from torch.utils.data import DataLoader`	Imports `DataLoader` .
17	`from tqdm import tqdm`	Imports `tqdm` .
18		*Blank visual separator line.*
19	`from ml.models.factory import CropClassifier`	Imports `CropClassifier` .
20	`from ml.models.losses import build_loss_fn`	Imports `build_loss_fn` .
21	`from ml.training.augmentations import MixupCutmixCollator`	Imports `MixupCutmixCollator` .
22	`from ml.utils.io import ensure_dir, save_json`	Imports dir/JSON helpers.
23	`from ml.utils.seed import get_default_device`	Imports `get_default_device` .
24	`from ml.utils.tracking import ExperimentTracker`	Imports `ExperimentTracker` (MLflow/W&B/CSV logger).
25		*Blank visual separator line.*
26		*Blank visual separator line.*
27	`def build_cosine_warmup_scheduler(`	Defines `build_cosine_warmup_scheduler` stepping once per optimizer update.
28	`    optimizer: torch.optim.Optimizer,`	Target optimizer instance.
29	`    warmup_epochs: int,`	Number of linear warmup epochs.
30	`    total_epochs: int,`	Total epochs across this stage.
31	`    steps_per_epoch: int,`	Exact number of mini-batches per epoch.
32	`    min_lr_ratio: float = 0.01,`	Terminal LR multiplier floor (default `0.01`).
33	`) -> torch.optim.lr_scheduler.LambdaLR:`	Returns PyTorch `LambdaLR` scheduler.
34	`    """Create step-level linear warmup followed by cosine decay LR scheduler."""`	Docstring.
35	`    total_steps = max(1, total_epochs * steps_per_epoch)`	Total training steps guarded against `<= 0` .
36	`    warmup_steps = max(0, warmup_epochs * steps_per_epoch)`	Total warmup steps.
37		*Blank visual separator line.*
38	`    def lr_lambda(current_step: int) -> float:`	Inner LR multiplier lambda called at step `current_step` (`0` -indexed).
39	`        if warmup_steps > 0 and current_step < warmup_steps:`	Checks whether we are inside linear warmup window.
40	`            return float(current_step + 1) / float(warmup_steps)`	Linearly ramps multiplier `(step + 1) / warmup_steps` up toward `1.0` .
41	`        progress = float(current_step - warmup_steps) / float(max(1, total_steps - warmup_steps))`	Computes normalized post-warmup progress `t in [0, 1]` .
42	`        progress = min(max(progress, 0.0), 1.0)`	Clamps `progress` cleanly within `[0.0, 1.0]` .
43	`        cosine = 0.5 * (1.0 + math.cos(math.pi * progress))`	Evaluates half-period cosine curve `0.5 * (1 + cos(pi * t))` decaying smoothly from `1.0` to `0.0` .
44	`        return min_lr_ratio + (1.0 - min_lr_ratio) * cosine`	Scales curve between `1.0` and `min_lr_ratio` .
45		*Blank visual separator line.*
46	`    return torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=lr_lambda)`	Returns `LambdaLR` wrapping `lr_lambda` .
47		*Blank visual separator line.*
48		*Blank visual separator line.*
49	`@torch.no_grad()`	Decorator disabling autograd graph construction across entire validation function (saves VRAM & speed).
50	`def evaluate_epoch(`	Defines `evaluate_epoch(model, loader, device, criterion, amp_enabled=True)` .
51	`    model: nn.Module,`	Model or `ema.module` to evaluate.
52	`    loader: DataLoader,`	Validation/Test `DataLoader` .
53	`    device: torch.device,`	Target `torch.device` .
54	`    criterion: nn.Module,`	Unweighted CrossEntropy evaluation loss.

Line	Source Code	Line-by-Line Explanation
55	`    amp_enabled: bool = True,`	Enables FP16/BF16 autocast on CUDA.
56	`) -> dict[str, float]:`	Returns `{loss, acc, macro_f1}` dict.
57	`    """Run validation pass and return loss, accuracy, and macro-F1."""`	Docstring.
58	`    model.eval()`	Puts BatchNorm and Dropout layers into eval mode.
59	`    total_loss = 0.0`	Accumulates sample-weighted validation loss.
60	`    total_samples = 0`	Accumulates total sample count.
61	`    all_preds: list[int] = []`	Collects hard predicted indices.
62	`    all_targets: list[int] = []`	Collects ground-truth indices.
63	`    device_type = "cuda" if device.type == "cuda" else "cpu"`	Determines `'cuda'` or `'cpu'` autocast backend string.
64		*Blank visual separator line.*
65	`    for images, targets in loader:`	Loops over validation batches.
66	`        images = images.to(device, non_blocking=True)`	Asynchronous host-to-GPU image transfer (`non_blocking=True` pairs with `pin_memory`).
67	`        targets = targets.to(device, non_blocking=True)`	Asynchronous target transfer.
68	`        with torch.amp.autocast(device_type=device_type, enabled=amp_enabled and device.type == "cuda"):`	Runs forward pass under AMP autocast when on GPU.
69	`            logits = model(images)`	Computes logits `(B, K)` .
70	`            loss = criterion(logits, targets)`	Computes mean batch loss.
71		*Blank visual separator line.*
72	`        bs = images.shape[0]`	Reads current batch size `bs` (handles trailing partial batch accurately).
73	`        total_loss += float(loss.item()) * bs`	Adds batch loss weighted by `bs` .
74	`        total_samples += bs`	Updates sample count.
75	`        preds = logits.argmax(dim=-1).cpu().tolist()`	Takes argmax predicted classes and moves to CPU list.
76	`        all_preds.extend(preds)`	Appends predictions.
77	`        all_targets.extend(targets.cpu().tolist())`	Appends targets.
78		*Blank visual separator line.*
79	`    acc = float(accuracy_score(all_targets, all_preds))`	Computes overall validation Accuracy.
80	`    macro_f1 = float(f1_score(all_targets, all_preds, average="macro", zero_division=0))`	Computes unweighted Macro-F1 across all classes (crucial for imbalanced disease data).
81	`    return {`	Returns summary metrics dictionary.
82	`        "loss": total_loss / max(total_samples, 1),`	Weighted mean loss.
83	`        "acc": acc,`	Accuracy score.
84	`        "macro_f1": macro_f1,`	Macro-F1 score.
85	`    }`	Closes return dict.
86		*Blank visual separator line.*
87		*Blank visual separator line.*
88	`class TwoStageTrainer:`	Defines `TwoStageTrainer` orchestrating full supervised training.
89	`    """Two-stage supervised trainer: frozen-head warmup followed by discriminative fine-tuning."""`	Docstring.
90		*Blank visual separator line.*
91	`    def __init__(`	Constructor.
92	`        self,`	`self` reference.
93	`        model: CropClassifier,`	`CropClassifier` model.
94	`        train_loader: DataLoader,`	Training `DataLoader` .
95	`        val_loader: DataLoader,`	Validation `DataLoader` .
96	`        cfg: dict[str, Any],`	Merged YAML configuration dictionary.
97	`        classes: list[str],`	Canonical ordered class names list.
98	`        class_weights: torch.Tensor | None = None,`	Optional 1D class weights tensor.
99	`        output_dir: str | Path | None = None,`	Optional custom run directory.
100	`        tracker: ExperimentTracker | None = None,`	Optional MLflow/W&B experiment logger.
101	`        epoch_callback: Callable[[int, dict[str, float]], None] | None = None,`	Optional epoch callback (used by Optuna for intermediate reporting/pruning).
102	`    ) -> None:`	Returns `None` .
103	`        self.model = model`	Stores model reference.
104	`        self.train_loader = train_loader`	Stores train loader.
105	`        self.val_loader = val_loader`	Stores val loader.
106	`        self.cfg = cfg`	Stores config.
107	`        self.classes = classes`	Stores classes list.
108	`        self.num_classes = len(classes)`	Computes `num_classes` .
109	`        self.tracker = tracker`	Stores tracker.
110	`        self.epoch_callback = epoch_callback`	Stores epoch callback.
111		*Blank visual separator line.*
112	`        self.device = get_default_device()`	Auto-selects safe CUDA or CPU device.
113	`        self.model.to(self.device)`	Moves model weights to compute device.
114	`        self.class_weights = class_weights.to(self.device) if class_weights is not None else None`	Moves optional class weights tensor to device.

Line	Source Code	Line-by-Line Explanation
115		*Blank visual separator line.*
116	`exp_name = str(cfg.get("experiment_name", model.backbone_name))`	Reads experiment name for artifact subfolder.
117	`artifacts_dir = Path(cfg["paths"]["artifacts_dir"])`	Reads `artifacts/` base folder.
118	`self.output_dir = ensure_dir(output_dir or (artifacts_dir / "runs" / exp_name))`	Creates run folder `artifacts/runs/<experiment_name>`.
119		*Blank visual separator line.*
120	`tr_cfg = cfg["training"]`	Shortcut to `cfg['training']`.
121	`self.amp_enabled = bool(tr_cfg.get("amp", True)) and self.device.type == "cuda"`	Enables AMP only if config allows it AND device is CUDA.
122	`self.grad_clip_norm = float(tr_cfg.get("grad_clip_norm", 1.0))`	Reads gradient L2 norm clip limit (`1.0`).
123	`self.label_smoothing = float(tr_cfg.get("label_smoothing", 0.1))`	Reads label smoothing (`0.1`).
124		*Blank visual separator line.*
125	`mix_cfg = tr_cfg.get("mixup", {})`	Reads MixUp/CutMix config.
126	`self.mixup_enabled = bool(mix_cfg.get("enabled", True))`	Stores `mixup_enabled` flag.
127	`self.mixup = (`	Instantiates `MixupCutmixCollator` if enabled, else `None`.
128	`MixupCutmixCollator(`	Collator class.
129	`num_classes=self.num_classes,`	Passes class count.
130	`mixup_alpha=float(mix_cfg.get("mixup_alpha", 0.2)),`	Passes MixUp alpha.
131	`cutmix_alpha=float(mix_cfg.get("cutmix_alpha", 1.0)),`	Passes CutMix alpha.
132	`prob=float(mix_cfg.get("prob", 0.5)),`	Passes batch trigger probability.
133	`switch_prob=float(mix_cfg.get("switch_prob", 0.5)),`	Passes MixUp/CutMix switch probability.
134	`label_smoothing=self.label_smoothing,`	Passes label smoothing.
135	`)`	Closes collator.
136	`if self.mixup_enabled`	Ternary condition.
137	`else None`	Else `None`.
138	`)`	Closes conditional.
139		*Blank visual separator line.*
140	`self.train_criterion = build_loss_fn(`	Builds training loss (`WeightedSoftTargetCrossEntropy` when MixUp active, `CrossEntropyLoss` otherwise).
141	`class_weights=self.class_weights,`	Passes class weights.
142	`label_smoothing=self.label_smoothing,`	Passes label smoothing.
143	`soft_targets=self.mixup_enabled,`	Matches `mixup_enabled`.
144	`).to(self.device)`	Moves criterion buffers to compute device.
145	`self.val_criterion = build_loss_fn(`	Builds clean unweighted hard-label CE loss for validation monitoring.
146	`class_weights=None,`	Unweighted validation loss.
147	`label_smoothing=0.0,`	Zero smoothing for true validation NLL.
148	`soft_targets=False,`	Hard targets.
149	`).to(self.device)`	Moves criterion to device.
150		*Blank visual separator line.*
151	`amp_device = "cuda" if self.device.type == "cuda" else "cpu"`	Determines AMP GradScaler device string.
152	`self.scaler = torch.amp.GradScaler(amp_device, enabled=self.amp_enabled)`	Creates `GradScaler` preventing FP16 gradient underflow during backprop.
153	`self.ema_cfg = tr_cfg.get("ema", {})`	Reads EMA config block.
154	`self.ema: ModelEmaV2 | None = None`	Initializes `self.ema` placeholder (activated in Stage 2).
155		*Blank visual separator line.*
156	`def _run_train_epoch(`	Defines `_run_train_epoch` executing one full training pass.
157	`self,`	`self` reference.
158	`optimizer: torch.optim.Optimizer,`	Active optimizer (`s1_opt` or `s2_opt`).
159	`scheduler: torch.optim.lr_scheduler.LambdaLR,`	Active step-level LR scheduler.
160	`epoch_label: str,`	Progress bar label string.
161	`) -> float:`	Returns epoch mean training loss.
162	`self.model.train()`	Sets model to training mode.
163	`running_loss = 0.0`	Initializes sample-weighted training loss sum.
164	`total_samples = 0`	Initializes sample count.
165	`device_type = "cuda" if self.device.type == "cuda" else "cpu"`	Sets autocast device type.
166		*Blank visual separator line.*
167	`pbar = tqdm(self.train_loader, desc=epoch_label, leave=False)`	Wraps train loader in `tqdm` progress bar.
168	`for images, targets in pbar:`	Iterates mini-batches.
169	`images = images.to(self.device, non_blocking=True)`	Moves batch images asynchronously to device.
170	`targets = targets.to(self.device, non_blocking=True)`	Moves integer targets asynchronously to device.
171		*Blank visual separator line.*
172	`if self.mixup is not None:`	Applies MixUp/CutMix + label smoothing if active.
173	`images, batch_targets = self.mixup(images, targets)`	Receives mixed images and soft probability vectors.
174	`else:`	Otherwise keeps raw images and integer targets.
175	`batch_targets = targets`	Uses integer targets.
176		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
177	`            optimizer.zero_grad(set_to_none=True)`	Clears gradients (`set_to_none=True` avoids zero-memset overhead).
178	`            with torch.amp.autocast(device_type=device_type, enabled=self.amp_enabled):`	Runs forward pass & loss under AMP autocast.
179	`                logits = self.model(images)`	Computes model logits.
180	`                loss = self.train_criterion(logits, batch_targets)`	Computes weighted soft/hard training loss.
181		*Blank visual separator line.*
182	`            self.scaler.scale(loss).backward()`	Scales loss to avoid FP16 underflow and backpropagates gradients.
183	`            if self.grad_clip_norm > 0:`	Checks if gradient norm clipping is enabled.
184	`                self.scaler.unscale_(optimizer)`	Unscales FP16 gradients back to FP32 BEFORE computing global norm.
185	`                torch.nn.utils.clip_grad_norm_(self.model.parameters(), self.grad_clip_norm)`	Clips L2 gradient norm across all trainable weights to `<= 1.0` .
186		*Blank visual separator line.*
187	`            self.scaler.step(optimizer)`	Executes optimizer step (skips automatically if Inf/NaN overflow occurred).
188	`            self.scaler.update()`	Updates AMP scale factor for next iteration.
189	`            scheduler.step()`	Steps cosine+warmup LR scheduler once per batch.
190		*Blank visual separator line.*
191	`            if self.ema is not None:`	Updates shadow EMA weights if Stage 2 EMA is active.
192	`                self.ema.update(self.model)`	Computes `theta_ema = decay * theta_ema + (1 - decay) * theta` .
193		*Blank visual separator line.*
194	`            bs = images.shape[0]`	Accumulates batch loss weighted by sample count.
195	`            running_loss += float(loss.item()) * bs`	Updates `running_loss` .
196	`            total_samples += bs`	Updates `total_samples` .
197	`            pbar.set_postfix(loss=f"{running_loss / max(total_samples, 1):.4f}")`	Updates progress bar postfix display.
198		*Blank visual separator line.*
199	`        return running_loss / max(total_samples, 1)`	Returns epoch average training loss.
200		*Blank visual separator line.*
201	`    def _save_checkpoint(`	Defines `_save_checkpoint` writing complete self-contained inference/resume metadata.
202	`        self,`	`self` reference.
203	`        path: Path,`	Output `.pt` file path (`best.pt` or `last.pt`).
204	`        eval_model: nn.Module,`	Model whose weights are saved (`self.model` in S1, `self.ema.module` in S2).
205	`        epoch: int,`	Current global epoch number.
206	`        val_metrics: dict[str, float],`	Validation metrics dict.
207	`    ) -> None:`	Returns `None` .
208	`        torch.save(`	Serializes checkpoint dictionary via `torch.save` .
209	`            {`	Opens checkpoint dict.
210	`                "epoch": epoch,`	Saves epoch number.
211	`                "backbone": self.model.backbone_name,`	Saves timm backbone ID.
212	`                "num_classes": self.num_classes,`	Saves class count.
213	`                "classes": self.classes,`	Saves ordered class list (critical for label parity!).
214	`                "image_size": int(self.cfg["image"]["size"]),`	Saves input resolution.
215	`                "mean": list(self.cfg["image"]["mean"]),`	Saves normalization means.
216	`                "std": list(self.cfg["image"]["std"]),`	Saves normalization stds.
217	`                "drop_rate": float(self.cfg["model"].get("drop_rate", 0.3)),`	Saves `drop_rate` for clean reconstruction.
218	`                "val_macro_f1": val_metrics["macro_f1"],`	Saves validation macro-F1.
219	`                "val_acc": val_metrics["acc"],`	Saves validation accuracy.
220	`                "val_loss": val_metrics["loss"],`	Saves validation loss.
221	`                "model_state_dict": eval_model.state_dict(),`	Saves model `state_dict` .
222	`            },`	Closes checkpoint dict.
223	`            path,`	Destination file path.
224	`        )`	Closes `torch.save` .
225		*Blank visual separator line.*
226	`    def fit(self, max_stage2_epochs: int | None = None) -> dict[str, Any]:`	Defines main training driver `fit(max_stage2_epochs=None)` .
227	`        """Execute Stage 1 (frozen head) + Stage 2 (unfrozen fine-tune) training."""`	Docstring.
228	`        start_time = time.time()`	Records wall-clock start timestamp.
229	`        tr_cfg = self.cfg["training"]`	Training config shortcut.
230	`        s1_cfg = tr_cfg["stage1"]`	Stage 1 config (`epochs` , `lr` , `warmup_epochs` , `weight_decay`).
231	`        s2_cfg = tr_cfg["stage2"]`	Stage 2 config (`epochs` , `backbone_lr` , `head_lr` , `patience`).
232		*Blank visual separator line.*
233	`        best_f1 = -1.0`	Initializes best Macro-F1 sentinel (`-1.0`).
234	`        best_acc = 0.0`	Initializes best Accuracy.
235	`        best_epoch = 0`	Tracks epoch achieving `best_f1` .
236	`        global_epoch = 0`	Monotonic epoch counter across Stage 1 + Stage 2.
237	`        history: list[dict[str, Any]] = []`	Stores per-epoch metrics history list.
238	`        best_ckpt_path = self.output_dir / "best.pt"`	Output path for best checkpoint (`best.pt`).

Line	Source Code	Line-by-Line Explanation
239		*Blank visual separator line.*
240	`        # --- Stage 1: Frozen backbone head warmup ---`	Section comment: Stage 1 frozen-backbone warmup.
241	`        s1_epochs = int(s1_cfg.get("epochs", 4))`	Reads Stage 1 epoch budget.
242	`        if s1_epochs > 0:`	Executes Stage 1 only if `s1_epochs > 0`.
243	`            print(f"[trainer] Stage 1: Training classifier head ({s1_epochs} epochs, backbone frozen)")`	Logs Stage 1 start.
244	`            self.model.freeze_backbone()`	Freezes pretrained backbone weights; unlocks only linear classifier head.
245	`            s1_opt = torch.optim.AdamW(`	Creates Stage 1 AdamW optimizing only the classifier head (`backbone_lr=0`).
246	`                self.model.get_param_groups(`	Builds parameter groups.
247	`                    backbone_lr=0.0,`	Zero backbone LR.
248	`                    head_lr=float(s1_cfg["lr"]),`	High head LR (`1e-3`).
249	`                    weight_decay=float(s1_cfg.get("weight_decay", 1e-2)),`	Weight decay.
250	`                )`	Closes `get_param_groups` .
251	`            )`	Closes `AdamW` .
252	`            s1_sched = build_cosine_warmup_scheduler(`	Creates Stage 1 cosine+warmup LR scheduler decaying to `0.1 * head_lr` .
253	`                s1_opt,`	Stage 1 optimizer.
254	`                warmup_epochs=int(s1_cfg.get("warmup_epochs", 1)),`	Warmup epochs.
255	`                total_epochs=s1_epochs,`	Total Stage 1 epochs.
256	`                steps_per_epoch=len(self.train_loader),`	Steps per epoch.
257	`                min_lr_ratio=0.1,`	Terminal LR ratio `0.1` .
258	`            )`	Closes scheduler.
259		*Blank visual separator line.*
260	`            for ep in range(1, s1_epochs + 1):`	Loops over Stage 1 epochs.
261	`                global_epoch += 1`	Increments global epoch.
262	`                tr_loss = self._run_train_epoch(s1_opt, s1_sched, f"Stage1 {ep}/{s1_epochs}")`	Runs one Stage 1 training epoch.
263	`                val_metrics = evaluate_epoch(`	Evaluates validation accuracy & Macro-F1.
264	`                    self.model, self.val_loader, self.device, self.val_criterion, self.amp_enabled`	Passes `self.model` , loaders, criterion, and AMP flag.
265	`                )`	Closes evaluation call.
266	`                row = {`	Builds epoch telemetry dictionary.
267	`                    "epoch": global_epoch,`	Global epoch.
268	`                    "stage": 1,`	Stage indicator `1` .
269	`                    "train_loss": tr_loss,`	Train loss.
270	`                    "val_loss": val_metrics["loss"],`	Val loss.
271	`                    "val_acc": val_metrics["acc"],`	Val accuracy.
272	`                    "val_macro_f1": val_metrics["macro_f1"],`	Val Macro-F1.
273	`                }`	Closes row dict.
274	`                history.append(row)`	Appends row to training history.
275	`                if self.tracker:`	Logs metrics to MLflow/W&B/CSV if tracker attached.
276	`                    self.tracker.log_metrics(row, step=global_epoch)`	Logs step metrics.
277		*Blank visual separator line.*
278	`                print(`	Prints Stage 1 epoch metrics table line.
279	`                    f"  [S1 E{ep:02d}] train_loss={tr_loss:.4f} | "`	Format part 1.
280	`                    f"val_loss={val_metrics['loss']:.4f} | val_acc={val_metrics['acc']:.4f} | "`	Format part 2.
281	`                    f"val_macro_f1={val_metrics['macro_f1']:.4f}"`	Format part 3.
282	`                )`	Closes print.
283	`                if val_metrics["macro_f1"] > best_f1:`	Checks if validation Macro-F1 improved over global best.
284	`                    best_f1 = val_metrics["macro_f1"]`	Updates `best_f1` .
285	`                    best_acc = val_metrics["acc"]`	Updates `best_acc` .
286	`                    best_epoch = global_epoch`	Updates `best_epoch` .
287	`                    self._save_checkpoint(best_ckpt_path, self.model, global_epoch, val_metrics)`	Saves `best.pt` .
288		*Blank visual separator line.*
289	`                if self.epoch_callback is not None:`	Invokes optional callback (e.g. Optuna intermediate score).
290	`                    self.epoch_callback(global_epoch, row)`	Passes `(global_epoch, row)` .
291		*Blank visual separator line.*
292	`        # --- Stage 2: Unfreeze all layers with discriminative learning rates + EMA ---`	Section comment: Stage 2 full end-to-end discriminative fine-tuning + EMA.
293	`        s2_epochs = int(max_stage2_epochs or s2_cfg.get("epochs", 16))`	Reads Stage 2 max epochs (honors override if provided).
294	`        patience = int(s2_cfg.get("early_stopping_patience", 5))`	Reads early stopping patience (`5` epochs).
295	`        no_improve = 0`	Resets patience stall counter to `0` .
296		*Blank visual separator line.*
297	`        print(`	Logs Stage 2 start and discriminative learning rates.

Line	Source Code	Line-by-Line Explanation
298	`            f"[trainer] Stage 2: Fine-tuning full model ({s2_epochs} epochs, "`	Format part 1.
299	`            f"backbone_lr={s2_cfg['backbone_lr']}, head_lr={s2_cfg['head_lr']})"`	Format part 2.
300	`        )`	Closes print.
301	`        self.model.unfreeze_backbone()`	Unfreezes all backbone + head layers.
302	`        s2_opt = torch.optim.AdamW(`	Creates Stage 2 AdamW with discriminative `backbone_lr` vs `head_lr` and norm/bias no-decay groups.
303	`            self.model.get_param_groups(`	Calls `get_param_groups`.
304	`                backbone_lr=float(s2_cfg["backbone_lr"]),`	Lower LR for backbone (`1e-4`).
305	`                head_lr=float(s2_cfg["head_lr"]),`	Higher LR for classifier (`5e-4`).
306	`                weight_decay=float(s2_cfg.get("weight_decay", 1e-2)),`	Decoupled weight decay.
307	`            )`	Closes `get_param_groups`.
308	`        )`	Closes `AdamW`.
309	`        min_lr_ratio = float(s2_cfg.get("min_lr", 1e-6)) / max(float(s2_cfg["head_lr"]), 1e-9)`	Computes exact `min_lr_ratio = min_lr / head_lr`.
310	`        s2_sched = build_cosine_warmup_scheduler(`	Builds Stage 2 step-level cosine+warmup scheduler.
311	`            s2_opt,`	Stage 2 optimizer.
312	`            warmup_epochs=int(s2_cfg.get("warmup_epochs", 2)),`	Warmup epochs (`2`).
313	`            total_epochs=s2_epochs,`	Total Stage 2 epochs.
314	`            steps_per_epoch=len(self.train_loader),`	Steps per epoch.
315	`            min_lr_ratio=min_lr_ratio,`	Computed `min_lr_ratio`.
316	`        )`	Closes scheduler.
317		*Blank visual separator line.*
318	`        if bool(self.ema_cfg.get("enabled", True)):`	Initializes Exponential Moving Average shadow weights (`ModelEmaV2`) if enabled.
319	`            self.ema = ModelEmaV2(self.model, decay=float(self.ema_cfg.get("decay", 0.999)))`	Clones current Stage 1 warmed model with decay `0.999`.
320		*Blank visual separator line.*
321	`        for ep in range(1, s2_epochs + 1):`	Loops over Stage 2 epochs.
322	`            global_epoch += 1`	Increments global epoch.
323	`            tr_loss = self._run_train_epoch(s2_opt, s2_sched, f"Stage2 {ep}/{s2_epochs}")`	Runs one Stage 2 training epoch (updating `self.ema` every batch).
324	`            eval_model = self.ema.module if self.ema is not None else self.model`	Selects smoothed EMA shadow model (`ema.module`) for validation when enabled.
325	`            val_metrics = evaluate_epoch(`	Evaluates validation metrics on `eval_model`.
326	`                eval_model, self.val_loader, self.device, self.val_criterion, self.amp_enabled`	Passes validation arguments.
327	`            )`	Closes `evaluate_epoch`.
328	`            row = {`	Assembles Stage 2 telemetry row.
329	`                "epoch": global_epoch,`	Global epoch.
330	`                "stage": 2,`	Stage `2`.
331	`                "train_loss": tr_loss,`	Train loss.
332	`                "val_loss": val_metrics["loss"],`	Val loss.
333	`                "val_acc": val_metrics["acc"],`	Val accuracy.
334	`                "val_macro_f1": val_metrics["macro_f1"],`	Val Macro-F1.
335	`                "backbone_lr": float(s2_opt.param_groups[0]["lr"]),`	Records current backbone learning rate.
336	`            }`	Closes row dict.
337	`            history.append(row)`	Appends row to `history`.
338	`            if self.tracker:`	Logs row to tracker.
339	`                self.tracker.log_metrics(row, step=global_epoch)`	Passes metrics + step.
340		*Blank visual separator line.*
341	`            print(`	Prints formatted Stage 2 progress summary.
342	` f" [S2 E{ep:02d}] train_loss={tr_loss:.4f}	"`	Format part 1.
343	` f"val_loss={val_metrics['loss']:.4f}	val_acc={val_metrics['acc']:.4f}	"`	Format part 2.
344	`                f"val_macro_f1={val_metrics['macro_f1']:.4f}"`	Format part 3.
345	`            )`	Closes print.
346		*Blank visual separator line.*
347	`            if val_metrics["macro_f1"] > best_f1:`	Checks if validation Macro-F1 reached a new all-time best.
348	`                best_f1 = val_metrics["macro_f1"]`	Updates `best_f1`.
349	`                best_acc = val_metrics["acc"]`	Updates `best_acc`.
350	`                best_epoch = global_epoch`	Updates `best_epoch`.
351	`                no_improve = 0`	Resets patience counter to `0`.
352	`                self._save_checkpoint(best_ckpt_path, eval_model, global_epoch, val_metrics)`	Saves EMA weights to `best.pt`.
353	`            else:`	Else no F1 improvement.
354	`                no_improve += 1`	Increments `no_improve` stall counter.
355		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
356	`            self._save_checkpoint(self.output_dir / "last.pt", eval_model, global_epoch, val_metrics)`	Always saves latest epoch weights to `last.pt`.
357		*Blank visual separator line.*
358	`            if self.epoch_callback is not None:`	Invokes optional epoch callback.
359	`                self.epoch_callback(global_epoch, row)`	Passes `(global_epoch, row)`.
360		*Blank visual separator line.*
361	`            if no_improve >= patience:`	Checks early stopping patience trigger.
362	`                print(f"[trainer] Early stopping triggered after {patience} epochs without F1 improvement.")`	Logs early stopping message.
363	`                break`	Breaks out of Stage 2 training loop.
364		*Blank visual separator line.*
365	`        duration = time.time() - start_time`	Computes total elapsed seconds.
366	`        summary = {`	Builds final summary dictionary returned to caller.
367	`            "experiment_name": str(self.cfg.get("experiment_name", self.model.backbone_name)),`	Experiment name.
368	`            "backbone": self.model.backbone_name,`	Backbone name.
369	`            "best_epoch": best_epoch,`	Epoch of best model.
370	`            "best_val_macro_f1": round(best_f1, 6),`	Best validation Macro-F1.
371	`            "best_val_acc": round(best_acc, 6),`	Best validation accuracy.
372	`            "checkpoint_path": str(best_ckpt_path.as_posix()),`	Path to `best.pt`.
373	`            "duration_sec": round(duration, 2),`	Total duration in seconds.
374	`        }`	Closes summary dict.
375	`        save_json({"summary": summary, "history": history}, self.output_dir / "train_history.json")`	Writes complete summary + epoch history to `train_history.json`.
376	`        if self.tracker:`	Logs best checkpoint file as tracked experiment artifact if tracker present.
377	`            self.tracker.log_artifact(best_ckpt_path, artifact_type="model")`	Uploads/records `best.pt`.
378	`        return summary`	Returns `summary` dict.

8.3 Post-Hoc Temperature Scaling & ECE — ml/evaluation/calibrate.py

File Summary: Optimizes a strictly positive scalar temperature T via L-BFGS to minimize validation Negative Log-Likelihood, computes 15-bin Expected Calibration Error (ECE), and plots before/after reliability diagrams.

ml/evaluation/calibrate.py (Lines 1–163)

Line	Source Code	Line-by-Line Explanation
1	`"""Post-hoc temperature scaling calibration and Expected Calibration Error (ECE)."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from pathlib import Path`	Imports `Path`.
6	`from typing import Any`	Imports `Any`.
7		*Blank visual separator line.*
8	`import matplotlib`	Imports `matplotlib`.
9	`import numpy as np`	Imports NumPy.
10	`import torch`	Imports PyTorch.
11	`import torch.nn as nn`	Imports `torch.nn`.
12	`from scipy.special import softmax`	Imports `scipy.special.softmax` for fast NumPy calibration metrics.
13		*Blank visual separator line.*
14	`matplotlib.use("Agg")`	Forces headless `'Agg'` rendering backend so server/Kaggle runs never fail without X11 display.
15	`import matplotlib.pyplot as plt`	Imports `matplotlib.pyplot`.
16		*Blank visual separator line.*
17		*Blank visual separator line.*
18	`class TemperatureScaler(nn.Module):`	Defines `TemperatureScaler(nn.Module)` holding a single learnable scalar `log(T)`.
19	`    """Positive scalar temperature T fitted via NLL minimization on validation logits."""`	Docstring.
20		*Blank visual separator line.*
21	`    def __init__(self, init_temp: float = 1.0) -> None:`	Initializes `log_temperature` parameter at `log(init_temp)` (default `0.0` -> `T=1.0`).
22	`        super().__init__()`	Calls `super().__init__()`.
23	`        self.log_temperature = nn.Parameter(torch.tensor([np.log(init_temp)], dtype=torch.float32))`	Parameterizes `log(T)` so `exp(log(T))` stays strictly positive throughout optimization.
24		*Blank visual separator line.*
25	`    @property`	Property decorator exposing clamped scalar `T` as Python float.
26	`    def temperature(self) -> float:`	Getter method.
27	`        return float(torch.exp(self.log_temperature).clamp(0.1, 10.0).item())`	Exponentiates and clamps temperature inside safe `[0.1, 10.0]` range.
28		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
29	`    def forward(self, logits: torch.Tensor) -> torch.Tensor:`	Defines `forward(logits)` returning `logits / T`.
30	`        temp = torch.exp(self.log_temperature).clamp(0.1, 10.0)`	Computes clamped differentiable temperature tensor.
31	`        return logits / temp`	Divides raw logits by scalar temperature `T` (preserves argmax ranking, calibrates confidence!).
32		*Blank visual separator line.*
33		*Blank visual separator line.*
34	`def compute_ece(`	Defines `compute_ece(probs, targets, n_bins=15)` measuring confidence-vs-accuracy gap.
35	`    probs: np.ndarray,`	`probs`: calibrated probability matrix `(N, K)`.
36	`    targets: np.ndarray,`	`targets`: true class indices `(N,)`.
37	`    n_bins: int = 15,`	`n_bins`: equal-width `[0, 1]` confidence histogram bins (default `15`).
38	`) -> tuple[float, dict[str, Any]]:`	Returns `(ece_scalar, bin_stats_dict)`.
39	`    """Compute Expected Calibration Error (ECE) and bin statistics for reliability diagrams."""`	Docstring.
40	`    confidences = np.max(probs, axis=-1)`	Takes top predicted probability per sample `max_c P(y=c|x)`.
41	`    predictions = np.argmax(probs, axis=-1)`	Takes top predicted class index `argmax_c`.
42	`    accuracies = (predictions == targets).astype(np.float64)`	Boolean-to-float indicator (`1.0` if prediction correct, `0.0` otherwise).
43		*Blank visual separator line.*
44	`    bin_edges = np.linspace(0.0, 1.0, n_bins + 1)`	Constructs `16` bin boundary edges `[0.0, 1/15, ..., 1.0]`.
45	`    bin_acc = []`	Stores per-bin empirical accuracy.
46	`    bin_conf = []`	Stores per-bin mean predicted confidence.
47	`    bin_counts = []`	Stores per-bin sample count.
48	`    ece = 0.0`	Accumulates ECE weighted sum.
49	`    n = max(len(targets), 1)`	Total sample count guarded against `0`.
50		*Blank visual separator line.*
51	`    for idx in range(n_bins):`	Iterates across all `n_bins` confidence intervals.
52	`        low, high = bin_edges[idx], bin_edges[idx + 1]`	Reads bin boundaries `(low, high]`.
53	`        if idx == 0:`	Includes `0.0` inclusively in bin 0.
54	`            mask = (confidences >= low) & (confidences <= high)`	First bin mask.
55	`        else:`	Subsequent half-open bin mask.
56	`            mask = (confidences > low) & (confidences <= high)`	Bins 1..14 mask `(low, high]`.
57	`        cnt = int(mask.sum())`	Counts samples falling inside current confidence bin.
58	`        bin_counts.append(cnt)`	Records bin count.
59	`        if cnt > 0:`	Computes empirical accuracy vs mean confidence if bin is non-empty.
60	`            acc_b = float(accuracies[mask].mean())`	Fraction of correct predictions in bin `b`.
61	`            conf_b = float(confidences[mask].mean())`	Average predicted confidence in bin `b`.
62	`            ece += (cnt / n) * abs(acc_b - conf_b)`	Adds weighted gap `(|B_b| / N) * |acc(B_b) - conf(B_b)|` to ECE.
63	`        else:`	Handles empty bins cleanly.
64	`            acc_b = 0.0`	Sets accuracy `0.0`.
65	`            conf_b = float((low + high) / 2.0)`	Sets confidence placeholder to bin midpoint.
66	`        bin_acc.append(acc_b)`	Appends bin accuracy.
67	`        bin_conf.append(conf_b)`	Appends bin confidence.
68		*Blank visual separator line.*
69	`    return float(ece), {`	Returns ECE float and diagram statistics dictionary.
70	`        "bin_edges": bin_edges.tolist(),`	Bin edge list.
71	`        "bin_acc": bin_acc,`	Bin accuracies.
72	`        "bin_conf": bin_conf,`	Bin confidences.
73	`        "bin_counts": bin_counts,`	Bin sample counts.
74	`    }`	Closes dict.
75		*Blank visual separator line.*
76		*Blank visual separator line.*
77	`def plot_reliability_diagram(`	Defines `plot_reliability_diagram(...)` visualizing calibration before vs after temperature scaling.
78	`    before_stats: dict[str, Any],`	Uncalibrated bin stats.
79	`    after_stats: dict[str, Any],`	Calibrated bin stats.
80	`    ece_before: float,`	Uncalibrated ECE.
81	`    ece_after: float,`	Calibrated ECE.
82	`    temperature: float,`	Fitted scalar temperature `T`.
83	`    out_path: str | Path,`	PNG output path.
84	`) -> None:`	Returns `None`.
85	`    """Render before/after temperature scaling reliability diagrams."""`	Docstring.
86	`    edges = np.array(before_stats["bin_edges"])`	Converts bin edges to NumPy array.
87	`    centers = 0.5 * (edges[:-1] + edges[1:])`	Computes bin bar centers.
88	`    width = (edges[1] - edges[0]) * 0.85`	Sets bar width to `85%` of bin step.
89		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
90	`    fig, axes = plt.subplots(1, 2, figsize=(11, 4.6), sharey=True)`	Creates side-by-side 2-panel Matplotlib figure.
91	`    panels = [`	Pairs left axis with uncalibrated stats and right axis with calibrated stats.
92	`        (axes[0], before_stats, f"Uncalibrated (T=1.00, ECE={ece_before:.4f})"),`	Left panel spec.
93	`        (axes[1], after_stats, f"Temperature Scaled (T={temperature:.2f}, ECE={ece_after:.4f})"),`	Right panel spec showing fitted `T` and reduced ECE.
94	`    ]`	Closes list.
95		*Blank visual separator line.*
96	`    for ax, stats, title in panels:`	Draws bars + diagonal `y = x` perfect-calibration reference line on each panel.
97	`        ax.bar(centers, stats["bin_acc"], width=width, color="#2d6a4f", alpha=0.82, edgecolor="#1b4332", label="Empirical Accuracy")`	Draws empirical accuracy bars.
98	`        ax.plot([0, 1], [0, 1], "--", color="#d62828", linewidth=1.6, label="Perfect Calibration")`	Draws dashed ideal diagonal line.
99	`        ax.set_title(title, fontsize=11, fontweight="bold")`	Sets subplot title.
100	`        ax.set_xlabel("Predicted Confidence")`	Sets X-axis label.
101	`        ax.set_xlim(0.0, 1.0)`	Fixes X-limits `[0, 1]`.
102	`        ax.set_ylim(0.0, 1.0)`	Fixes Y-limits `[0, 1]`.
103	`        ax.grid(alpha=0.3, linestyle="--")`	Adds subtle dashed grid.
104	`        ax.legend(loc="upper left", fontsize=8.5)`	Places legend.
105		*Blank visual separator line.*
106	`    axes[0].set_ylabel("Accuracy")`	Labels shared Y-axis `'Accuracy'`.
107	`    fig.tight_layout()`	Adjusts subplot spacing.
108	`    fig.savefig(out_path, dpi=180)`	Saves 180-DPI figure to disk.
109	`    plt.close(fig)`	Closes Matplotlib figure to prevent memory leaks.
110		*Blank visual separator line.*
111		*Blank visual separator line.*
112	`def fit_temperature_scaling(`	Defines `fit_temperature_scaling(val_logits, val_targets, plot_path=None)`.
113	`    val_logits: np.ndarray,`	Raw unscaled validation logits `(N, K)`.
114	`    val_targets: np.ndarray,`	Validation true class IDs `(N,)`.
115	`    plot_path: str | Path | None = None,`	Optional path to save reliability diagram PNG.
116	`) -> dict[str, Any]:`	Returns calibration report dict.
117	`    """Fit scalar temperature T on validation logits via L-BFGS and compute ECE."""`	Docstring.
118	`    logits_t = torch.tensor(val_logits, dtype=torch.float32)`	Converts NumPy logits to CPU float32 tensor.
119	`    targets_t = torch.tensor(val_targets, dtype=torch.long)`	Converts target array to int64 tensor.
120	`    nll_criterion = nn.CrossEntropyLoss()`	Creates NLL (`CrossEntropyLoss`) objective.
121		*Blank visual separator line.*
122	`    with torch.no_grad():`	Evaluates baseline NLL before temperature scaling (`T = 1.0`).
123	`        nll_before = float(nll_criterion(logits_t, targets_t).item())`	Records `nll_before`.
124		*Blank visual separator line.*
125	`    probs_before = softmax(val_logits, axis=-1)`	Computes uncalibrated probabilities.
126	`    ece_before, stats_before = compute_ece(probs_before, val_targets)`	Computes baseline `ece_before` and bin stats.
127		*Blank visual separator line.*
128	`    scaler = TemperatureScaler(init_temp=1.0)`	Initializes `TemperatureScaler` at `T = 1.0`.
129	`    optimizer = torch.optim.LBFGS([scaler.log_temperature], lr=0.05, max_iter=80)`	Uses L-BFGS second-order quasi-Newton optimizer converging in 1D cleanly within 80 iterations.
130		*Blank visual separator line.*
131	`    def _closure() -> torch.Tensor:`	Defines closure required by PyTorch `LBFGS.step()`.
132	`        optimizer.zero_grad()`	Zeros gradient on `log_temperature`.
133	`        loss = nll_criterion(scaler(logits_t), targets_t)`	Computes NLL loss of scaled logits `val_logits / T`.
134	`        loss.backward()`	Backpropagates scalar gradient `d(NLL)/d(log T)`.
135	`        return loss`	Returns scalar NLL for L-BFGS line search.
136		*Blank visual separator line.*
137	`    optimizer.step(_closure)`	Runs L-BFGS optimization step.
138	`    temperature = scaler.temperature`	Extracts optimal clamped scalar temperature `T`.
139		*Blank visual separator line.*
140	`    with torch.no_grad():`	Evaluates calibrated validation NLL without gradients.
141	`        nll_after = float(nll_criterion(scaler(logits_t), targets_t).item())`	Records `nll_after`.
142		*Blank visual separator line.*
143	`    probs_after = softmax(val_logits / temperature, axis=-1)`	Computes calibrated probabilities `softmax(val_logits / T)`.
144	`    ece_after, stats_after = compute_ece(probs_after, val_targets)`	Computes post-scaling `ece_after` and bin stats.
145		*Blank visual separator line.*
146	`    if plot_path is not None:`	Generates reliability diagram if `plot_path` was given.
147	`        plot_reliability_diagram(`	Calls `plot_reliability_diagram`.
148	`            stats_before,`	Before stats.
149	`            stats_after,`	After stats.

Line	Source Code	Line-by-Line Explanation
150	`            ece_before,`	Before ECE.
151	`            ece_after,`	After ECE.
152	`            temperature,`	Optimal `T`.
153	`            plot_path,`	Output path.
154	`        )`	Closes call.
155		*Blank visual separator line.*
156	`    return {`	Returns rounded calibration summary saved later into `calibration_report.json` & `model_meta.json`.
157	`        "temperature": round(temperature, 6),`	Fitted temperature.
158	`        "nll_before": round(nll_before, 6),`	Pre-calibration NLL.
159	`        "nll_after": round(nll_after, 6),`	Post-calibration NLL.
160	`        "ece_before": round(ece_before, 6),`	Pre-calibration ECE.
161	`        "ece_after": round(ece_after, 6),`	Post-calibration ECE.
162	`    }`	Closes return dict.
163		*Blank visual separator line.*

8.4 Energy Score & Out-of-Distribution Thresholding — ml/evaluation/ood.py

File Summary: Computes Liu et al. (2020) Helmholtz free-energy score E(x) = -T*logsumexp(z/T), evaluates AUROC and FPR@95%TPR against CIFAR-10 OOD negatives, tunes production confidence & energy thresholds, and plots ID-vs-OOD histograms.

ml/evaluation/ood.py (Lines 1–125)

Line	Source Code	Line-by-Line Explanation
1	`"""Out-of-distribution (OOD) detection via Energy Score, calibrated max-softmax, and PlantGate."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from pathlib import Path`	Imports `Path`.
6	`from typing import Any`	Imports `Any`.
7		*Blank visual separator line.*
8	`import matplotlib`	Imports `matplotlib`.
9	`import numpy as np`	Imports NumPy.
10	`from scipy.special import logsumexp, softmax`	Imports numerically stable `logsumexp` and `softmax`.
11	`from sklearn.metrics import roc_auc_score, roc_curve`	Imports `roc_auc_score` and `roc_curve` for OOD threshold tuning.
12		*Blank visual separator line.*
13	`matplotlib.use("Agg")`	Selects non-interactive Matplotlib backend.
14	`import matplotlib.pyplot as plt`	Imports `pyplot`.
15		*Blank visual separator line.*
16		*Blank visual separator line.*
17	`def compute_energy_score(logits: np.ndarray, temperature: float = 1.0) -> np.ndarray:`	Defines `compute_energy_score(logits, temperature=1.0)` implementing Helmholtz free energy.
18	`    """Compute Helmholtz free-energy score E(x) = -T * logsumexp(z / T).`	Docstring explaining formula.
19		*Blank visual separator line.*
20	`    More negative values indicate strong in-distribution certainty; values closer to 0`	Explains sign convention: very negative energy = high certainty in-distribution; closer to 0 = OOD.
21	`    (or positive) indicate out-of-distribution / uncertain inputs.`	Continuation of docstring.
22	`    """`	Closes docstring.
23	`    t = max(float(temperature), 1e-6)`	Guards temperature strictly positive.
24	`    return -t * logsumexp(logits / t, axis=-1)`	Vectorized `-T * log(sum_k exp(z_k / T))` along class axis `-1`.
25		*Blank visual separator line.*
26		*Blank visual separator line.*
27	`def _fpr_at_tpr(y_true_id: np.ndarray, id_scores: np.ndarray, target_tpr: float = 0.95) -> tuple[float, float]:`	Defines `_fpr_at_tpr(y_true_id, id_scores, target_tpr=0.95)` finding threshold preserving 95% of real crops.
28	`    """Compute FPR and threshold at `target_tpr` (where 1 = In-Distribution, 0 = OOD)."""`	Docstring.
29	`    fpr, tpr, thresholds = roc_curve(y_true_id, id_scores, pos_label=1)`	Computes full ROC curve (`fpr`, `tpr`, `thresholds`) treating ID (`1`) as positive class.
30	`    idx = int(np.searchsorted(tpr, target_tpr, side="left"))`	Finds first threshold index achieving `TPR >= 0.95`.
31	`    idx = min(idx, len(thresholds) - 1)`	Clamps index safely inside array bounds.
32	`    return float(fpr[idx]), float(thresholds[idx])`	Returns `(false_positive_rate_at_95_tpr, decision_threshold)`.
33		*Blank visual separator line.*
34		*Blank visual separator line.*
35	`def plot_ood_distributions(`	Defines `plot_ood_distributions(...)` plotting density histograms for Confidence & Energy.
36	`    id_conf: np.ndarray,`	ID validation confidences.
37	`    ood_conf: np.ndarray,`	OOD non-plant confidences.
38	`    id_energy: np.ndarray,`	ID validation energies.

Line	Source Code	Line-by-Line Explanation
39	`    ood_energy: np.ndarray,`	OOD non-plant energies.
40	`    conf_thresh: float,`	Vertical threshold line on confidence plot.
41	`    energy_thresh: float,`	Vertical threshold line on energy plot.
42	`    out_path: str | Path,`	PNG output path.
43	`) -> None:`	Returns `None`.
44	`    """Plot ID vs. OOD distributions for calibrated max-softmax and Energy Score."""`	Docstring.
45	`    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))`	Creates 2-panel comparison figure.
46		*Blank visual separator line.*
47	`    axes[0].hist(id_conf, bins=30, density=True, alpha=0.6, color="#2d6a4f", label="In-Distribution (Val)")`	Plots normalized ID confidence histogram (forest green).
48	`    axes[0].hist(ood_conf, bins=30, density=True, alpha=0.6, color="#e63946", label="OOD (Non-Plant)")`	Plots normalized OOD confidence histogram (crimson red).
49	`    axes[0].axvline(conf_thresh, color="#1d3557", linestyle="--", linewidth=1.8, label=f"Threshold={conf_thresh:.2f}")`	Draws chosen confidence cutoff line.
50	`    axes[0].set_title("Calibrated Max-Softmax Confidence")`	Sets left subplot title.
51	`    axes[0].set_xlabel("Max Probability")`	X-label.
52	`    axes[0].set_ylabel("Density")`	Y-label.
53	`    axes[0].legend(fontsize=8.5)`	Legend.
54		*Blank visual separator line.*
55	`    axes[1].hist(id_energy, bins=30, density=True, alpha=0.6, color="#2d6a4f", label="In-Distribution (Val)")`	Plots normalized ID energy histogram (more negative / left).
56	`    axes[1].hist(ood_energy, bins=30, density=True, alpha=0.6, color="#e63946", label="OOD (Non-Plant)")`	Plots normalized OOD energy histogram (less negative / right).
57	`    axes[1].axvline(energy_thresh, color="#1d3557", linestyle="--", linewidth=1.8, label=f"Threshold={energy_thresh:.2f}")`	Draws chosen energy cutoff line.
58	`    axes[1].set_title("Energy Score E(x)  [Lower = More Certain]")`	Sets right subplot title.
59	`    axes[1].set_xlabel("Energy Score")`	X-label.
60	`    axes[1].legend(fontsize=8.5)`	Legend.
61		*Blank visual separator line.*
62	`    fig.tight_layout()`	Adjusts layout.
63	`    fig.savefig(out_path, dpi=180)`	Saves 180-DPI figure.
64	`    plt.close(fig)`	Frees Matplotlib memory.
65		*Blank visual separator line.*
66		*Blank visual separator line.*
67	`def tune_ood_thresholds(`	Defines `tune_ood_thresholds(...)` selecting production confidence & energy cutoffs.
68	`    id_logits: np.ndarray,`	Validation ID logits.
69	`    ood_logits: np.ndarray | None,`	Optional OOD logits (from CIFAR-10 negatives).
70	`    temperature: float = 1.0,`	Fitted scalar temperature `T`.
71	`    default_conf_threshold: float = 0.55,`	Fallback confidence threshold (`0.55`).
72	`    default_energy_threshold: float = -3.5,`	Fallback energy threshold (`-3.5`).
73	`    plot_path: str | Path | None = None,`	Optional OOD histogram path.
74	`) -> dict[str, Any]:`	Returns metrics + thresholds dictionary.
75	`    """Tune confidence & energy thresholds separating in-distribution crops from OOD images."""`	Docstring.
76	`    id_probs = softmax(id_logits / max(temperature, 1e-6), axis=-1)`	Computes temperature-calibrated ID probabilities.
77	`    id_conf = np.max(id_probs, axis=-1)`	Extracts ID max-softmax confidences.
78	`    id_energy = compute_energy_score(id_logits, temperature=1.0)`	Extracts raw-logit energy scores `E(x)` (`T=1`).
79		*Blank visual separator line.*
80	`    if ood_logits is None or len(ood_logits) == 0:`	Quantile fallback path if no OOD images were supplied.
81	`        # Quantile-based fallback when OOD negatives were not provided`	Comment explaining 5th/95th quantile heuristic.
82	`        conf_t = float(np.clip(np.quantile(id_conf, 0.05), 0.35, 0.75))`	Sets confidence threshold at 5th percentile of ID validation confidences, clipped to `[0.35, 0.75]`.
83	`        energy_t = float(np.quantile(id_energy, 0.95))`	Sets energy threshold at 95th percentile of ID energies (accepting 95% of real crops).
84	`        return {`	Returns fallback OOD calibration report.
85	`            "has_ood_eval": False,`	Marks `has_ood_eval=False`.
86	`            "confidence_threshold": round(conf_t, 4),`	Tuned confidence threshold.
87	`            "energy_threshold": round(energy_t, 4),`	Tuned energy threshold.
88	`            "default_conf_threshold": default_conf_threshold,`	Config default confidence threshold.
89	`            "default_energy_threshold": default_energy_threshold,`	Config default energy threshold.
90	`            "id_conf_p05": round(float(np.quantile(id_conf, 0.05)), 4),`	Logs 5th percentile ID confidence.
91	`            "id_energy_p95": round(float(np.quantile(id_energy, 0.95)), 4),`	Logs 95th percentile ID energy.
92	`        }`	Closes fallback dict.
93		*Blank visual separator line.*
94	`    ood_probs = softmax(ood_logits / max(temperature, 1e-6), axis=-1)`	Computes temperature-calibrated OOD probabilities.
95	`    ood_conf = np.max(ood_probs, axis=-1)`	Extracts OOD max-softmax confidences.

Line	Source Code	Line-by-Line Explanation
96	`    ood_energy = compute_energy_score(ood_logits, temperature=1.0)`	Extracts OOD energy scores.
97		*Blank visual separator line.*
98	`    y_true = np.concatenate([np.ones(len(id_conf)), np.zeros(len(ood_conf))])`	Builds binary label vector: `1` for ID crop images, `0` for OOD non-plant negatives.
99	`    conf_scores = np.concatenate([id_conf, ood_conf])`	Concatenates max-softmax scores (higher = more ID).
100	`    neg_energy_scores = np.concatenate([-id_energy, -ood_energy])  # higher = more ID`	Negates energy `-E(x)` so higher score consistently means more ID.
101		*Blank visual separator line.*
102	`    conf_auroc = float(roc_auc_score(y_true, conf_scores))`	Computes AUROC using max-softmax confidence.
103	`    energy_auroc = float(roc_auc_score(y_true, neg_energy_scores))`	Computes AUROC using negative Energy Score (typically higher than softmax!).
104		*Blank visual separator line.*
105	`    conf_fpr95, conf_thresh_95 = _fpr_at_tpr(y_true, conf_scores, target_tpr=0.95)`	Finds confidence threshold & OOD FPR at 95% ID recall.
106	`    energy_fpr95, neg_energy_thresh_95 = _fpr_at_tpr(y_true, neg_energy_scores, target_tpr=0.95)`	Finds energy threshold & OOD FPR at 95% ID recall.
107		*Blank visual separator line.*
108	`    conf_thresh = float(np.clip(conf_thresh_95, 0.40, 0.80))`	Clamps tuned confidence threshold defensively inside `[0.40, 0.80]`.
109	`    energy_thresh = float(-neg_energy_thresh_95)`	Restores original sign (`threshold = -neg_energy_thresh_95`) so `E(x) > energy_thresh` flags OOD.
110		*Blank visual separator line.*
111	`    if plot_path is not None:`	Saves OOD separation plot if `plot_path` given.
112	`        plot_ood_distributions(`	Calls `plot_ood_distributions`.
113	`            id_conf, ood_conf, id_energy, ood_energy, conf_thresh, energy_thresh, plot_path`	Passes arrays, thresholds, and path.
114	`        )`	Closes plot call.
115		*Blank visual separator line.*
116	`    return {`	Returns complete OOD evaluation + tuned thresholds dict.
117	`        "has_ood_eval": True,`	Marks `has_ood_eval=True`.
118	`        "confidence_threshold": round(conf_thresh, 4),`	Production confidence threshold.
119	`        "energy_threshold": round(energy_thresh, 4),`	Production energy threshold.
120	`        "conf_auroc": round(conf_auroc, 4),`	Softmax OOD AUROC.
121	`        "conf_fpr_at_95tpr": round(conf_fpr95, 4),`	Softmax FPR@95%TPR.
122	`        "energy_auroc": round(energy_auroc, 4),`	Energy OOD AUROC.
123	`        "energy_fpr_at_95tpr": round(energy_fpr95, 4),`	Energy FPR@95%TPR.
124	`    }`	Closes return dict.
125		*Blank visual separator line.*

8.5 Grad-CAM Explainability & Fast GAP Parity Audit — ml/evaluation/gradcam.py

File Summary: Computes exact backward-pass PyTorch Grad-CAM alongside forward-only GAP classifier CAM, blends JET heatmaps over RGB images, and verifies Pearson correlation parity between them.

ml/evaluation/gradcam.py (Lines 1–160)

Line	Source Code	Line-by-Line Explanation
1	`"""Grad-CAM explainability, PyTorch vs. ONNX CAM parity check, and overlay rendering."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from pathlib import Path`	Imports `Path`.
6	`from typing import Any`	Imports `Any`.
7		*Blank visual separator line.*
8	`import cv2`	Imports OpenCV (`cv2.resize`, `cv2.applyColorMap`).
9	`import matplotlib`	Imports `matplotlib`.
10	`import numpy as np`	Imports NumPy.
11	`import torch`	Imports PyTorch.
12	`import torch.nn.functional as F`	Imports `torch.nn.functional`.
13	`from PIL import Image, ImageOps`	Imports PIL `Image` and `ImageOps`.
14		*Blank visual separator line.*
15	`matplotlib.use("Agg")`	Sets headless `'Agg'` Matplotlib backend.
16	`import matplotlib.pyplot as plt`	Imports `pyplot`.
17		*Blank visual separator line.*
18	`from ml.models.factory import CropClassifier`	Imports `CropClassifier`.
19		*Blank visual separator line.*
20		*Blank visual separator line.*
21	`def compute_pytorch_gradcam(`	Defines exact backward-gradient `compute_pytorch_gradcam(model, input_tensor, target_class=None)`.
22	`    model: CropClassifier,`	Wrapped model.
23	`    input_tensor: torch.Tensor,`	Preprocessed image tensor `(1, 3, H, W)` or `(3, H, W)`.

Line	Source Code	Line-by-Line Explanation
24	`    target_class: int | None = None,`	Optional explicit class index (defaults to predicted argmax).
25	`) -> tuple[np.ndarray, int]:`	Returns `(normalized_cam_2d_array, predicted_class_idx)`.
26	`    """Compute spatial Grad-CAM heatmap `(H, W)` in `[0, 1]` using gradients on spatial features."""`	Docstring.
27	`    model.eval()`	Sets evaluation mode.
28	`    device = next(model.parameters()).device`	Inspects device of model weights.
29	`    x = input_tensor.to(device)`	Moves input tensor to model device.
30	`    if x.ndim == 3:`	Adds batch dimension `(1, ...)` if unbatched 3D tensor was passed.
31	`        x = x.unsqueeze(0)`	Unsqueezes dim 0.
32		*Blank visual separator line.*
33	`    model.zero_grad(set_to_none=True)`	Clears stale gradients.
34	`    raw_feats = model.backbone.forward_features(x)`	Runs backbone feature extractor.
35	`    raw_feats.retain_grad()`	Calls `.retain_grad()` on non-leaf activation tensor `raw_feats` so gradient `d y_c / d A` is preserved.
36	`    pooled = model.backbone.forward_head(raw_feats, pre_logits=True)`	Pools spatial features through backbone head.
37	`    logits = model.classifier(model.dropout(pooled))`	Runs dropout + linear classifier to obtain class logits.
38		*Blank visual separator line.*
39	`    pred_class = int(logits.argmax(dim=-1).item()) if target_class is None else int(target_class)`	Picks argmax predicted class unless `target_class` was pinned.
40	`    score = logits[0, pred_class]`	Selects pre-softmax logit `y_c` for the target class.
41	`    score.backward()`	Backpropagates `score` through the head directly into `raw_feats.grad`.
42		*Blank visual separator line.*
43	`    if raw_feats.grad is None:`	Verifies gradient capture succeeded.
44	`        raise RuntimeError("Failed to capture gradients on backbone spatial features.")`	Raises error if graph had no grad flow.
45		*Blank visual separator line.*
46	`    feats_nchw = model._to_nchw_spatial_map(raw_feats.detach())`	Converts raw activations to unified `(1, C, H_f, W_f)` layout (CNN/Swin/ViT).
47	`    grads_nchw = model._to_nchw_spatial_map(raw_feats.grad.detach())`	Converts captured gradients to matching `(1, C, H_f, W_f)` layout.
48		*Blank visual separator line.*
49	`    weights = grads_nchw.mean(dim=(2, 3), keepdim=True)`	Global-average-pools gradients spatially -> channel importance weights `alpha_k` `(1, C, 1, 1)`.
50	`    cam = F.relu((weights * feats_nchw).sum(dim=1, keepdim=True))`	Weighted channel sum passed through ReLU: `ReLU(sum_k alpha_k * A^k)`.
51	`    cam_up = F.interpolate(`	Bilinearly upsamples coarse CAM map `(H_f, W_f)` back to input resolution `(224, 224)`.
52	`        cam,`	Spatial CAM tensor.
53	`        size=(x.shape[2], x.shape[3]),`	Target `(H, W)` dimensions.
54	`        mode="bilinear",`	Bilinear interpolation.
55	`        align_corners=False,`	`align_corners=False`.
56	`    )[0, 0].cpu().numpy()`	Extracts `(H, W)` NumPy array.
57		*Blank visual separator line.*
58	`    cam_min, cam_max = float(cam_up.min()), float(cam_up.max())`	Reads min and max activations for `[0, 1]` min-max normalization.
59	`    if cam_max - cam_min > 1e-8:`	Avoids division by zero if CAM map is identically uniform.
60	`        cam_norm = (cam_up - cam_min) / (cam_max - cam_min)`	Normalizes heatmap into `[0.0, 1.0]`.
61	`    else:`	Fallback for zero-variance maps.
62	`        cam_norm = np.zeros_like(cam_up)`	Returns all-zero map.
63	`    return cam_norm.astype(np.float32), pred_class`	Returns `(cam_norm_float32, pred_class)`.
64		*Blank visual separator line.*
65		*Blank visual separator line.*
66	`def compute_classifier_cam(`	Defines forward-only `compute_classifier_cam(spatial_map, classifier_weights, target_class, out_size)`.
67	`    spatial_map: np.ndarray,`	Spatial feature tensor `(1, C, H_f, W_f)`.
68	`    classifier_weights: np.ndarray,`	Classifier weight matrix `W` `(K, C)`.
69	`    target_class: int,`	Target class row `c`.
70	`    out_size: tuple[int, int] = (224, 224),`	Target output spatial resolution.
71	`) -> np.ndarray:`	Returns `(H, W)` normalized CAM.
72	`    """Compute fast GAP-equivalent Grad-CAM directly from `(1, C, H_f, W_f)` & classifier weights."""`	Docstring explaining zero-backprop GAP equivalence.
73	`    feats = spatial_map[0] if spatial_map.ndim == 4 else spatial_map`	Squeezes batch dim if `(1, C, H_f, W_f)` -> `(C, H_f, W_f)`.
74	`    w = classifier_weights[target_class]  # (C,)`	Selects class weight vector `W_c` `(C,)`.
75	`    raw_cam = np.maximum(0.0, np.tensordot(w, feats, axes=(0, 0)))`	Contracts channel weights with feature map via `np.tensordot` and applies ReLU (`maximum(0, ...)`).
76	`    cam_resized = cv2.resize(`	Bilinearly resizes CAM via OpenCV.
77	`        raw_cam.astype(np.float32),`	Casts to float32.
78	`        (out_size[1], out_size[0]),`	Note: OpenCV takes `(width, height)` order!
79	`        interpolation=cv2.INTER_LINEAR,`	Bilinear flag.
80	`    )`	Closes `cv2.resize`.
81	`    c_min, c_max = float(cam_resized.min()), float(cam_resized.max())`	Computes min and max for normalization.

Line	Source Code	Line-by-Line Explanation
82	`    if c_max - c_min > 1e-8:`	Guards against uniform map.
83	`        return ((cam_resized - c_min) / (c_max - c_min)).astype(np.float32)`	Returns min-max normalized float32 heatmap.
84	`    return np.zeros(out_size, dtype=np.float32)`	Returns zeros if flat.
85		*Blank visual separator line.*
86		*Blank visual separator line.*
87	`def render_cam_overlay(`	Defines `render_cam_overlay(rgb_image, cam_01, alpha=0.45)`.
88	`    rgb_image: np.ndarray,`	Base RGB uint8 image `(H, W, 3)`.
89	`    cam_01: np.ndarray,`	Normalized `[0, 1]` CAM array.
90	`    alpha: float = 0.45,`	Heatmap alpha opacity (`0.45`).
91	`) -> np.ndarray:`	Returns blended RGB uint8 image.
92	`    """Blend a `[0, 1]` Grad-CAM heatmap over an RGB `uint8` image."""`	Docstring.
93	`    h, w = rgb_image.shape[:2]`	Reads `(H, W)` of target image.
94	`    cam_resized = cv2.resize(cam_01.astype(np.float32), (w, h), interpolation=cv2.INTER_LINEAR)`	Resizes CAM to match image pixel dimensions.
95	`    heatmap_uint8 = np.uint8(np.clip(cam_resized * 255.0, 0, 255))`	Scales `[0, 1]` to `[0, 255]` `uint8`.
96	`    heatmap_bgr = cv2.applyColorMap(heatmap_uint8, cv2.COLORMAP_JET)`	Maps scalar intensity to JET rainbow palette (OpenCV returns BGR).
97	`    heatmap_rgb = cv2.cvtColor(heatmap_bgr, cv2.COLOR_BGR2RGB)`	Converts OpenCV BGR heatmap back to RGB.
98	`    blended = (1.0 - alpha) * rgb_image.astype(np.float32) + alpha * heatmap_rgb.astype(np.float32)`	Alpha-blends `(1 - alpha)*Original + alpha*Heatmap`.
99	`    return np.clip(blended, 0, 255).astype(np.uint8)`	Clamps `[0, 255]` and casts back to `uint8`.
100		*Blank visual separator line.*
101		*Blank visual separator line.*
102	`def save_gradcam_gallery(`	Defines `save_gradcam_gallery(...)` rendering evaluation samples & verifying PyTorch vs ONNX CAM parity.
103	`    model: CropClassifier,`	Model instance.
104	`    samples: list[dict[str, Any]],`	List of test sample dicts.
105	`    eval_transform: Any,`	Deterministic eval Albumentations pipeline.
106	`    classes: list[str],`	Ordered class names.
107	`    out_path: str | Path,`	Output PNG figure path.
108	`) -> float:`	Returns mean Pearson correlation `r` between backward Grad-CAM and fast GAP-CAM.
109	`    """Generate a Grad-CAM comparison gallery and verify PyTorch vs. classifier-CAM correlation."""`	Docstring.
110	`    n = len(samples)`	Reads sample count.
111	`    if n == 0:`	Returns `1.0` immediately if sample list is empty.
112	`        return 1.0`	Early return.
113		*Blank visual separator line.*
114	`    weights_np = model.classifier.weight.detach().cpu().numpy()`	Extracts linear classifier weights `(K, C)` to NumPy.
115	`    corrs: list[float] = []`	Stores per-sample Pearson correlation values.
116		*Blank visual separator line.*
117	`    fig, axes = plt.subplots(n, 2, figsize=(8.5, 3.3 * n))`	Creates `N x 2` subplot grid (Original vs Grad-CAM Overlay).
118	`    if n == 1:`	Normalizes 1D axes array when `N == 1` for consistent 2D indexing.
119	`        axes = np.expand_dims(axes, axis=0)`	Expands row dimension.
120		*Blank visual separator line.*
121	`    for row_idx, item in enumerate(samples):`	Iterates through evaluation samples.
122	`        with Image.open(item["image_path"]) as raw:`	Opens image file.
123	`            img = ImageOps.exif_transpose(raw).convert("RGB")`	EXIF-rotates and converts to RGB.
124	`            img_disp = img.resize((224, 224), Image.Resampling.BILINEAR)`	Prepares `(224, 224)` display crop.
125	`            img_np = np.array(img)`	Original array for Albumentations eval transform.
126	`            disp_np = np.array(img_disp)`	Display uint8 array.
127		*Blank visual separator line.*
128	`        tensor = eval_transform(image=img_np)["image"].unsqueeze(0)`	Applies eval transform and adds batch dimension `(1, 3, 224, 224)`.
129	`        cam_pt, pred_idx = compute_pytorch_gradcam(model, tensor)`	Computes exact backward PyTorch Grad-CAM.
130		*Blank visual separator line.*
131	`        with torch.no_grad():`	Runs gradient-free `forward_with_features` mimicking ONNX Runtime output.
132	`            _, spatial = model.forward_with_features(tensor.to(next(model.parameters()).device))`	Extracts spatial map `(1, C, H_f, W_f)`.
133	`        cam_fast = compute_classifier_cam(`	Computes fast GAP-CAM matching ONNX's `1x1` Conv output.
134	`            spatial.cpu().numpy(),`	Passes NumPy spatial map.
135	`            weights_np,`	Passes classifier weights.
136	`            target_class=pred_idx,`	Uses same predicted class index.
137	`            out_size=(224, 224),`	Matches `(224, 224)` size.
138	`        )`	Closes `compute_classifier_cam`.
139	`        if cam_pt.std() > 1e-6 and cam_fast.std() > 1e-6:`	Computes Pearson correlation only if neither map has zero variance.
140	`            corr = float(np.corrcoef(cam_pt.ravel(), cam_fast.ravel())[0, 1])`	Calculates Pearson `r` between flattened exact Grad-CAM and fast GAP-CAM (`~0.99–1.00` for CNNs).

Line	Source Code	Line-by-Line Explanation
141	`            corrs.append(corr)`	Stores correlation coefficient.
142		*Blank visual separator line.*
143	`        overlay = render_cam_overlay(disp_np, cam_pt, alpha=0.45)`	Renders blended heatmap overlay.
144	`        true_lbl = item["label"].replace("___", ": ")`	Formats ground-truth label nicely.
145	`        pred_lbl = classes[pred_idx].replace("___", ": ")`	Formats predicted label nicely.
146		*Blank visual separator line.*
147	`        axes[row_idx, 0].imshow(disp_np)`	Plots input photo in left column.
148	`        axes[row_idx, 0].set_title(f"Input\nTrue: {true_lbl}", fontsize=9)`	Sets left title with true class.
149	`        axes[row_idx, 0].axis("off")`	Hides ticks/frame.
150		*Blank visual separator line.*
151	`        axes[row_idx, 1].imshow(overlay)`	Plots Grad-CAM overlay in right column.
152	`        axes[row_idx, 1].set_title(f"Grad-CAM Overlay\nPred: {pred_lbl}", fontsize=9)`	Sets right title with predicted class.
153	`        axes[row_idx, 1].axis("off")`	Hides ticks/frame.
154		*Blank visual separator line.*
155	`    fig.suptitle("Grad-CAM Explainability Samples", fontsize=13, fontweight="bold")`	Adds gallery super-title.
156	`    fig.tight_layout()`	Tightens layout.
157	`    fig.savefig(out_path, dpi=170)`	Saves 170-DPI gallery image.
158	`    plt.close(fig)`	Closes Matplotlib figure.
159	`    return float(np.mean(corrs)) if corrs else 1.0`	Returns average Pearson correlation across gallery.
160		*Blank visual separator line.*

8.6 Dual-Output ONNX, TorchScript & INT8 Export — ml/export/export_model.py

File Summary: Wraps CropClassifier in DualOutputOnnxWrapper using a 1x1 Conv2d layer initialized from linear classifier weights so ONNX emits both logits and per-class Grad-CAM maps in one fast CPU pass, exports TorchScript/INT8 variants, and writes production model_meta.json.

ml/export/export_model.py (Lines 1–164)

Line	Source Code	Line-by-Line Explanation
1	`"""Export calibrated model to dual-output ONNX (logits + per-class CAM), TorchScript, and INT8 ONNX.`	Module docstring.
2		Blank separator.
3	`Usage:`	Usage header.
4	`    python -m ml.export.export_model --checkpoint artifacts/best_model.pt --torchscript --quantize-int8`	CLI command example.
5	`"""`	Closes docstring.
6		*Blank visual separator line.*
7	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
8		*Blank visual separator line.*
9	`import argparse`	Imports `argparse`.
10	`from pathlib import Path`	Imports `Path`.
11	`from typing import Any`	Imports `Any`.
12		*Blank visual separator line.*
13	`import onnx`	Imports `onnx` for static graph structural verification (`onnx.checker`).
14	`import torch`	Imports PyTorch.
15	`import torch.nn as nn`	Imports `nn`.
16	`import torch.nn.functional as F`	Imports `nn.functional as F`.
17		*Blank visual separator line.*
18	`from ml.models.factory import CropClassifier, load_checkpoint_model`	Imports `CropClassifier` and `load_checkpoint_model`.
19	`from ml.utils.io import ensure_dir, load_json, load_yaml, save_json`	Imports JSON/YAML/dir utilities.
20		*Blank visual separator line.*
21		*Blank visual separator line.*
22	`class DualOutputOnnxWrapper(nn.Module):`	Defines `DualOutputOnnxWrapper(nn.Module)` unifying inference + Grad-CAM into a single graph.
23	`    """Wraps CropClassifier so ONNX emits both `(logits, cam_maps)` in a single forward pass.`	Docstring header.
24		Blank separator.
25	`    `cam_maps` has shape `(B, num_classes, H_f, W_f)`, computed via a 1x1 convolution initialized`	Notes output shape `(B, K, H_f, W_f)` and `1x1` Conv mathematical equivalence to GAP Grad-CAM.
26	`    with the classifier linear weights, giving pure-ONNX GAP-Grad-CAM maps in <0.1 ms on CPU.`	Notes `<0.1 ms` CPU cost.
27	`    """`	Closes docstring.
28		*Blank visual separator line.*
29	`    def __init__(self, model: CropClassifier) -> None:`	Constructor taking trained `CropClassifier`.
30	`        super().__init__()`	Calls `super().__init__()`.
31	`        self.model = model`	Stores wrapped model.

Line	Source Code	Line-by-Line Explanation
32	`        num_classes = model.num_classes`	Reads class count `K`.
33	`        num_features = model.num_features`	Reads feature channels `C`.
34	`        self.cam_conv = nn.Conv2d(num_features, num_classes, kernel_size=1, bias=False)`	Creates bias-free `1x1` Conv2d mapping `(B, C, H_f, W_f) -> (B, K, H_f, W_f)`.
35	`        with torch.no_grad():`	Copies classifier weight matrix into `1x1` conv filters without tracking gradients.
36	`            w = model.classifier.weight.detach().clone().view(num_classes, num_features, 1, 1)`	Reshapes weight `(K, C)` -> `(K, C, 1, 1)`.
37	`            self.cam_conv.weight.copy_(w)`	Copies weights directly into `self.cam_conv.weight`.
38		*Blank visual separator line.*
39	`    def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:`	Defines ONNX forward graph `(x) -> (logits, cam_maps)`.
40	`        logits, spatial_map = self.model.forward_with_features(x)`	Runs `forward_with_features(x)` returning both classification logits and NCHW feature map.
41	`        cam_maps = F.relu(self.cam_conv(spatial_map))`	Applies `1x1` convolution + ReLU across spatial locations producing exact per-class CAM heatmaps.
42	`        return logits, cam_maps`	Returns dual tuple `(logits, cam_maps)`.
43		*Blank visual separator line.*
44		*Blank visual separator line.*
45	`def export_artifacts(`	Defines `export_artifacts(...)` orchestrating ONNX, TorchScript, INT8, and metadata generation.
46	`    checkpoint_path: str | Path,`	Trained `.pt` checkpoint path.
47	`    base_cfg: dict[str, Any],`	Base YAML dict.
48	`    output_dir: str | Path | None = None,`	Optional target directory (defaults to `artifacts/`).
49	`    calibration_path: str | Path | None = None,`	Optional `calibration_report.json` path.
50	`    export_torchscript: bool = True,`	Whether to trace TorchScript `.pt`.
51	`    quantize_int8: bool = False,`	Whether to produce dynamically quantized INT8 ONNX.
52	`    opset: int = 17,`	ONNX opset version (`17`).
53	`) -> dict[str, Path]:`	Returns dict mapping artifact key -> file Path.
54	`    """Export dual-output ONNX, `model_meta.json`, and optional TorchScript / INT8 ONNX."""`	Docstring.
55	`    artifacts_dir = ensure_dir(output_dir or base_cfg["paths"]["artifacts_dir"])`	Creates output directory.
56	`    cal_file = Path(calibration_path) if calibration_path else (artifacts_dir / "calibration_report.json")`	Resolves calibration JSON report path.
57		*Blank visual separator line.*
58	`    model, ckpt = load_checkpoint_model(checkpoint_path, device="cpu")`	Loads trained `CropClassifier` and checkpoint metadata on CPU.
59	`    model.eval()`	Locks model into eval mode.
60	`    wrapper = DualOutputOnnxWrapper(model).eval()`	Wraps model in `DualOutputOnnxWrapper` in eval mode.
61		*Blank visual separator line.*
62	`    img_size = int(ckpt.get("image_size", base_cfg["image"]["size"]))`	Reads input resolution (`224`).
63	`    dummy = torch.randn(1, 3, img_size, img_size, dtype=torch.float32)`	Creates dummy `(1, 3, 224, 224)` float32 tracer input.
64		*Blank visual separator line.*
65	`    onnx_path = artifacts_dir / "model.onnx"`	Target path `artifacts/model.onnx`.
66	`    torch.onnx.export(`	Exports PyTorch graph to ONNX with named inputs/outputs and constant folding.
67	`        wrapper,`	Wrapper module.
68	`        dummy,`	Dummy tensor.
69	`        str(onnx_path),`	Destination file path.
70	`        input_names=["image"],`	Input tensor name `'image'`.
71	`        output_names=["logits", "cam_maps"],`	Output tensor names `['logits', 'cam_maps']`.
72	`        dynamic_axes={`	Dynamic batch dimension axis `0` enabling variable batch size at runtime.
73	`            "image": {0: "batch"},`	Dynamic batch on input.
74	`            "logits": {0: "batch"},`	Dynamic batch on logits.
75	`            "cam_maps": {0: "batch"},`	Dynamic batch on cam_maps.
76	`        },`	Closes `dynamic_axes`.
77	`        opset_version=opset,`	Targets ONNX Opset `17`.
78	`        do_constant_folding=True,`	Enables constant-folding graph simplification (fuses BN/constants).
79	`    )`	Closes `torch.onnx.export`.
80		*Blank visual separator line.*
81	`    onnx_model = onnx.load(str(onnx_path))`	Loads exported `.onnx` protobuf back from disk.
82	`    onnx.checker.check_model(onnx_model)`	Verifies structural tensor shape and op validity via `onnx.checker.check_model`.
83	`    exported: dict[str, Path] = {"onnx": onnx_path}`	Records exported ONNX path.
84	`    print(f"[export] Verified ONNX graph saved -> {onnx_path} ({onnx_path.stat().st_size / 1e6:.2f} MB)")`	Logs verified ONNX file path and size in MB.
85		*Blank visual separator line.*
86	`    if export_torchscript:`	Checks whether TorchScript export is requested.
87	`        ts_path = artifacts_dir / "model_ts.pt"`	Output path `artifacts/model_ts.pt`.
88	`        with torch.no_grad():`	Traces wrapper graph without gradients.
89	`            traced = torch.jit.trace(wrapper, dummy)`	Runs `torch.jit.trace`.

Line	Source Code	Line-by-Line Explanation
90	`            traced.save(str(ts_path))`	Saves TorchScript archive.
91	`        exported["torchscript"] = ts_path`	Records TorchScript path.
92	`        print(f"[export] Saved TorchScript -> {ts_path}")`	Logs saved path.
93		*Blank visual separator line.*
94	`    if quantize_int8:`	Checks whether INT8 dynamic quantization is requested.
95	`        try:`	Starts try-block (handles environments lacking quantization extras gracefully).
96	`            from onnxruntime.quantization import QuantType, quantize_dynamic`	Imports `QuantType` and `quantize_dynamic` from `onnxruntime.quantization`.
97		*Blank visual separator line.*
98	`            int8_path = artifacts_dir / "model_int8.onnx"`	Output path `artifacts/model_int8.onnx`.
99	`            quantize_dynamic(`	Compresses MatMul/Gemm/Conv weights from FP32 to QInt8 (~2.5–3x smaller).
100	`                model_input=str(onnx_path),`	Source FP32 ONNX.
101	`                model_output=str(int8_path),`	Target INT8 ONNX.
102	`                weight_type=QuantType.QInt8,`	Signed 8-bit integer weight quantization.
103	`            )`	Closes quantization call.
104	`            exported["onnx_int8"] = int8_path`	Records INT8 artifact path.
105	`            print(f"[export] Saved INT8 ONNX -> {int8_path} ({int8_path.stat().st_size / 1e6:.2f} MB)")`	Logs INT8 path and compressed MB size.
106	`        except Exception as exc:`	Catches optional quantization error without breaking export.
107	`            print(f"[export] INT8 quantization skipped ({exc}).")`	Prints friendly skip note.
108		*Blank visual separator line.*
109	`    # Assemble production metadata (temperature + OOD thresholds + classes)`	Comment: bundle calibrated temperature, OOD cutoffs, normalization stats, and class list.
110	`    cal_data = load_json(cal_file) if cal_file.exists() else {}`	Loads `calibration_report.json` if calibrated, else `{}`.
111	`    ood_data = cal_data.get("ood", {})`	Extracts nested `'ood'` thresholds block.
112	`    default_ood = base_cfg.get("ood", {})`	Reads fallback `'ood'` defaults from `configs/base.yaml`.
113		*Blank visual separator line.*
114	`    meta = {`	Constructs self-contained production metadata dictionary.
115	`        "model_version": "1.0.0",`	API/artifact schema version.
116	`        "backbone": ckpt["backbone"],`	Trained timm backbone string.
117	`        "num_classes": int(ckpt["num_classes"]),`	Class count.
118	`        "classes": list(ckpt["classes"]),`	Canonical sorted class labels matching model output indices.
119	`        "image_size": img_size,`	Expected input resolution (`224`).
120	`        "mean": list(ckpt.get("mean", base_cfg["image"]["mean"])),`	Normalization channel means.
121	`        "std": list(ckpt.get("std", base_cfg["image"]["std"])),`	Normalization channel stds.
122	`        "temperature": float(cal_data.get("temperature", 1.0)),`	Fitted temperature scaling divisor `T` (defaults to `1.0`).
123	`        "confidence_threshold": float(`	Uncertainty confidence threshold (tuned value overriding base config default).
124	`            ood_data.get("confidence_threshold", default_ood.get("confidence_threshold", 0.55))`	Lookups tuned vs default confidence threshold.
125	`        ),`	Closes field.
126	`        "energy_threshold": float(`	Energy score OOD threshold (tuned value overriding base config default).
127	`            ood_data.get("energy_threshold", default_ood.get("energy_threshold", -3.5))`	Lookups tuned vs default energy threshold.
128	`        ),`	Closes field.
129	`        "plant_gate_threshold": float(default_ood.get("plant_gate_threshold", 0.45)),`	Binary PlantGate probability threshold (`0.45`).
130	`        "outputs": ["logits", "cam_maps"],`	Documents ONNX output tensor names.
131	`    }`	Closes `meta` dict.
132	`    meta_path = save_json(meta, artifacts_dir / "model_meta.json")`	Writes `artifacts/model_meta.json`.
133	`    exported["meta"] = meta_path`	Records meta path.
134	`    print(f"[export] Saved inference metadata -> {meta_path}")`	Logs saved path.
135	`    return exported`	Returns exported artifact path mapping.
136		*Blank visual separator line.*
137		*Blank visual separator line.*
138	`def parse_args() -> argparse.Namespace:`	Defines CLI parser `parse_args()`.
139	`    parser = argparse.ArgumentParser(description="Export Crop Disease model to ONNX & TorchScript.")`	Creates `ArgumentParser`.
140	`    parser.add_argument("--checkpoint", default="artifacts/best_model.pt")`	Adds `--checkpoint` flag.
141	`    parser.add_argument("--base", default="configs/base.yaml")`	Adds `--base` flag.
142	`    parser.add_argument("--output-dir", default=None)`	Adds `--output-dir` flag.
143	`    parser.add_argument("--torchscript", action="store_true", help="Also export TorchScript.")`	Adds `--torchscript` flag.
144	`    parser.add_argument("--quantize-int8", action="store_true", help="Also export INT8 dynamic ONNX.")`	Adds `--quantize-int8` flag.
145	`    parser.add_argument("--opset", type=int, default=17)`	Adds `--opset` flag.
146	`    return parser.parse_args()`	Returns parsed args.
147		*Blank visual separator line.*
148		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
149	`def main() -> None:`	CLI entrypoint `main()`.
150	`    args = parse_args()`	Parses CLI arguments.
151	`    base_cfg = load_yaml(args.base)`	Loads `base.yaml`.
152	`    export_artifacts(`	Runs `export_artifacts` with parsed flags.
153	`        checkpoint_path=args.checkpoint,`	Passes checkpoint path.
154	`        base_cfg=base_cfg,`	Passes base config.
155	`        output_dir=args.output_dir,`	Passes output dir.
156	`        export_torchscript=args.torchscript,`	Passes TorchScript flag.
157	`        quantize_int8=args.quantize_int8,`	Passes INT8 flag.
158	`        opset=args.opset,`	Passes opset version.
159	`    )`	Closes call.
160		*Blank visual separator line.*
161		*Blank visual separator line.*
162	`if __name__ == "__main__":`	Standard script execution guard.
163	`    main()`	Invokes `main()`.
164		*Blank visual separator line.*

Chapter 9: Backend API & Frontend UI Source Code (Line-by-Line)

This chapter provides complete verbatim code and line-by-line explanations for backend configuration, SQLAlchemy ORM tables, Pydantic v2 contracts, the ONNX Runtime inference engine, FastAPI endpoints, middleware/rate-limiting, the TypeScript fetch client, the layered Grad-CAM viewer component, and the root React application.

9.1 Pydantic Settings & HF Hub Auto-Download — backend/app/core/__init__.py

File Summary: Loads `.env` and environment variables into typed backend parameters (ports, CORS, rate limits, ONNX paths, OOD cutoffs, DB URL) and optionally fetches missing ONNX artifacts from Hugging Face Hub on startup.

backend/app/core/__init__.py (Lines 1–102)

Line	Source Code	Line-by-Line Explanation
1	`"""Pydantic v2 application settings with `.env` loading and optional HF Hub auto-download."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`import os`	Imports `os` for `os.makedirs`.
6	`from pathlib import Path`	Imports `Path`.
7	`from typing import Any`	Imports `Any`.
8		*Blank visual separator line.*
9	`from pydantic import field_validator`	Imports Pydantic v2 `field_validator` decorator.
10	`from pydantic_settings import BaseSettings`	Imports `BaseSettings` from `pydantic_settings` for `.env` binding.
11		*Blank visual separator line.*
12		*Blank visual separator line.*
13	`class Settings(BaseSettings):`	Defines `Settings(BaseSettings)` holding all backend configuration fields.
14	`    """Backend configuration loaded from `.env`, environment variables, or defaults."""`	Class docstring.
15		*Blank visual separator line.*
16	`    # --- General ---`	Section comment: general metadata.
17	`    project_name: str = "Crop Disease & Pest Classifier"`	API title shown in Swagger `/docs`.
18	`    app_env: str = "development"`	Environment mode (`development` enables Uvicorn hot reload).
19	`    log_level: str = "INFO"`	Logging verbosity (`INFO`, `DEBUG`, etc.).
20		*Blank visual separator line.*
21	`    # --- Server ---`	Section comment: HTTP server configuration.
22	`    api_host: str = "0.0.0.0"`	Bind host IP (`0.0.0.0` listens on all interfaces for Docker).
23	`    api_port: int = 8000`	TCP listen port (`8000`).
24	`    cors_origins: str = "http://localhost:5173,http://localhost:3000,http://127.0.0.1:5173"`	Comma-separated allowed browser origins for CORS.
25	`    rate_limit: str = "60/minute"`	SlowAPI per-IP request rate limit (`60/minute`).
26		*Blank visual separator line.*
27	`    # --- Upload Validation ---`	Section comment: upload security bounds.
28	`    max_upload_mb: int = 10`	Maximum image upload payload in megabytes (`10` MB).
29		*Blank visual separator line.*
30	`    # --- Model Paths ---`	Section comment: serialized ML artifact paths.
31	`    model_onnx_path: str = "artifacts/model.onnx"`	Main dual-output classifier ONNX path.
32	`    model_meta_path: str = "artifacts/model_meta.json"`	JSON metadata path storing classes, temperature `T`, and tuned thresholds.
33	`    plant_gate_onnx_path: str = "artifacts/plant_gate.onnx"`	Binary plant-vs-nonplant ONNX gate path.
34	`    advice_kb_path: str = "configs/advice_kb.json"`	Agronomic disease & pest treatment JSON knowledge base path.
35		*Blank visual separator line.*
36	`    # --- Hugging Face Hub (optional cloud artifact download) ---`	Section comment: optional cloud download coordinates.
37	`    hf_token: str = ""`	Optional private HF Hub access token.
38	`    hf_model_repo: str = ""`	Optional HF repository ID (e.g. `user/crop-disease-onnx`).
39		*Blank visual separator line.*
40	`    # --- Calibration & OOD Thresholds (fallbacks; overridden by model_meta.json if present) ---`	Section comment: OOD & confidence thresholds (used only if `model_meta.json` lacks them).
41	`    uncertainty_conf_threshold: float = 0.55`	Minimum calibrated top-1 confidence before flagging uncertain (`0.55`).
42	`    ood_energy_threshold: float = -3.5`	Maximum allowed Helmholtz energy score before flagging OOD (`-3.5`).
43	`    plant_gate_threshold: float = 0.45`	Minimum PlantGate sigmoid probability to accept as crop/pest (`0.45`).
44		*Blank visual separator line.*
45	`    # --- Database ---`	Section comment: SQL persistence URI.
46	`    database_url: str = "sqlite:///./backend_data.sqlite3"`	SQLAlchemy database connection string (defaults to local SQLite file).
47		*Blank visual separator line.*
48	`    model_config = {"env_file": ".env", "env_file_encoding": "utf-8", "extra": "ignore"}`	Pydantic Settings config reading `.env` in UTF-8 and ignoring unrelated ML keys.
49		*Blank visual separator line.*
50	`    @field_validator("cors_origins", mode="before")`	Decorator running custom CORS normalization before Pydantic field validation.

Line	Source Code	Line-by-Line Explanation
51	`    @classmethod`	Classmethod marker required by Pydantic v2.
52	`    def _parse_cors(cls, v: Any) -> str:`	Normalizes YAML/JSON list CORS env values into comma-separated string.
53	`        if isinstance(v, list):`	Joins list items if supplied as Python list.
54	`            return ",".join(str(x) for x in v)`	Comma-join.
55	`        return str(v)`	Otherwise casts scalar to string.
56		*Blank visual separator line.*
57	`    @property`	Property decorator exposing parsed list of origins.
58	`    def cors_origin_list(self) -> list[str]:`	Splits comma-delimited string, strips spaces, and drops empty entries.
59	`        return [o.strip() for o in self.cors_origins.split(",") if o.strip()]`	Returns cleaned origin list for FastAPI `CORSMiddleware`.
60		*Blank visual separator line.*
61	`    @property`	Property decorator converting MB limit to raw byte count.
62	`    def max_upload_bytes(self) -> int:`	Getter method.
63	`        return self.max_upload_mb * 1024 * 1024`	Multiplies `max_upload_mb * 1024 * 1024` (`10,485,760` bytes).
64		*Blank visual separator line.*
65	`    def ensure_artifacts(self) -> None:`	Defines `ensure_artifacts()` auto-fetching missing models from Hugging Face Hub.
66	`        """Download model artifacts from Hugging Face Hub if they are missing locally."""`	Docstring.
67	`        if not self.hf_model_repo:`	Returns immediately if `HF_MODEL_REPO` is blank (local-artifact mode).
68	`            return`	Early exit.
69		*Blank visual separator line.*
70	`        needed_files = [`	Lists core ONNX + JSON artifacts to check locally.
71	`            (self.model_onnx_path, "model.onnx"),`	Tuple `(local_onnx_path, 'model.onnx')`.
72	`            (self.model_meta_path, "model_meta.json"),`	Tuple `(local_meta_path, 'model_meta.json')`.
73	`        ]`	Closes list.
74	`        # Optionally fetch plant gate`	Comment: also pull optional binary gate if missing.
75	`        if not Path(self.plant_gate_onnx_path).exists():`	Checks whether `plant_gate.onnx` is absent locally.
76	`            needed_files.append((self.plant_gate_onnx_path, "plant_gate.onnx"))`	Adds `plant_gate.onnx` to download queue.
77		*Blank visual separator line.*
78	`        missing = [(local, remote) for local, remote in needed_files if not Path(local).exists()]`	Filters queue strictly down to files missing on disk.
79	`        if not missing:`	Skips network calls if all required files already exist locally.
80	`            return`	Early exit.
81		*Blank visual separator line.*
82	`        try:`	Starts try-block so temporary network outages never crash API startup cleanly.
83	`            from huggingface_hub import hf_hub_download`	Lazy-imports `hf_hub_download` only when actually needed.
84		*Blank visual separator line.*
85	`            token = self.hf_token or None`	Uses `None` instead of empty string for public repos.
86	`            for local_path, filename in missing:`	Iterates missing artifacts.
87	`                dest_dir = str(Path(local_path).parent)`	Extracts target parent folder (e.g., `artifacts`).
88	`                os.makedirs(dest_dir, exist_ok=True)`	Creates destination folder if absent.
89	`                downloaded = hf_hub_download(`	Downloads file directly into `local_dir`.
90	`                    repo_id=self.hf_model_repo,`	Target HF repo.
91	`                    filename=filename,`	Remote filename.
92	`                    local_dir=dest_dir,`	Destination folder.
93	`                    token=token,`	Auth token.
94	`                )`	Closes download call.
95	`                print(f"[config] Downloaded {filename} from HF Hub -> {downloaded}")`	Logs downloaded file location.
96	`        except Exception as exc:`	Catches Hub download failures cleanly.
97	`            print(f"[config] HF Hub download failed: {exc}")`	Logs warning.
98		*Blank visual separator line.*
99		*Blank visual separator line.*
100	`# Singleton`	Comment marking global singleton instance.
101	`settings = Settings()`	Instantiates module-level `settings` singleton imported throughout backend.
102		*Blank visual separator line.*

9.2 SQLAlchemy ORM Tables — backend/app/db/models.py

File Summary: Defines SQLAlchemy 2.0 typed `DeclarativeBase` ORM models (`PredictionLog` and `UserFeedback`) storing inference audit logs, Top-3 JSON snapshots, uncertainty/OOD flags, latencies, and user label corrections.

backend/app/db/models.py (Lines 1–50)

Line	Source Code	Line-by-Line Explanation
1	`"""SQLAlchemy ORM models for prediction logs and user feedback."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.

Line	Source Code	Line-by-Line Explanation
4		*Blank visual separator line.*
5	`from datetime import UTC, datetime`	Imports `UTC` timezone & `datetime`.
6		*Blank visual separator line.*
7	`from sqlalchemy import JSON, Boolean, DateTime, Float, Integer, String, Text`	Imports SQLAlchemy column types (`JSON`, `Boolean`, `DateTime`, `Float`, `Integer`, `String`, `Text`).
8	`from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column`	Imports SQLAlchemy 2.0 `DeclarativeBase`, `Mapped`, and `mapped_column`.
9		*Blank visual separator line.*
10		*Blank visual separator line.*
11	`class Base(DeclarativeBase):`	Defines `Base(DeclarativeBase)` shared metadata registry.
12	`    """Shared ORM base class for all tables."""`	Docstring.
13	`    pass`	Empty body (metadata inherited).
14		*Blank visual separator line.*
15		*Blank visual separator line.*
16	`class PredictionLog(Base):`	Defines `PredictionLog` ORM table tracking every `/predict` invocation.
17	`    """Stores every inference request for audit, analytics, and active-learning mining."""`	Docstring noting audit + active-learning mining purpose.
18		*Blank visual separator line.*
19	`    __tablename__ = "prediction_logs"`	SQL table name `'prediction_logs'`.
20		*Blank visual separator line.*
21	`    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)`	Auto-incrementing integer primary key `id`.
22	`    request_id: Mapped[str] = mapped_column(String(32), index=True, nullable=False)`	Indexed 32-char `request_id` correlating `/predict` responses, logs, and `/feedback`.
23	`    predicted_class: Mapped[str] = mapped_column(String(128), nullable=False)`	Predicted `Crop___Disease` or `'Unknown'` string (`max 128` chars).
24	`    confidence: Mapped[float] = mapped_column(Float, nullable=False)`	Calibrated top-1 softmax probability (`Float`).
25	`    top3: Mapped[dict] = mapped_column(JSON, nullable=True)`	Serialized JSON array storing Top-3 candidates.
26	`    is_uncertain: Mapped[bool] = mapped_column(Boolean, default=False)`	Boolean flag recording low-confidence warning.
27	`    is_ood: Mapped[bool] = mapped_column(Boolean, default=False)`	Boolean flag recording PlantGate / Energy OOD rejection.
28	`    filename: Mapped[str | None] = mapped_column(String(256), nullable=True)`	Original uploaded filename (optional, up to `256` chars).
29	`    file_size_bytes: Mapped[int | None] = mapped_column(Integer, nullable=True)`	Uploaded image payload size in bytes.
30	`    latency_ms: Mapped[float | None] = mapped_column(Float, nullable=True)`	Server inference latency in milliseconds.
31	`    created_at: Mapped[datetime] = mapped_column(`	Timezone-aware UTC timestamp defaulting automatically per row insert.
32	`        DateTime(timezone=True), default=lambda: datetime.now(UTC)`	Column definition with `lambda: datetime.now(UTC)`.
33	`    )`	Closes `mapped_column`.
34		*Blank visual separator line.*
35		*Blank visual separator line.*
36	`class UserFeedback(Base):`	Defines `UserFeedback` ORM table collecting farmer/agronomist label confirmations & corrections.
37	`    """User-submitted label correction for active-learning retraining loops."""`	Docstring explaining active-learning retraining feedback loop.
38		*Blank visual separator line.*
39	`    __tablename__ = "user_feedback"`	SQL table name `'user_feedback'`.
40		*Blank visual separator line.*
41	`    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)`	Auto-incrementing primary key `id`.
42	`    request_id: Mapped[str] = mapped_column(String(32), index=True, nullable=False)`	Indexed `request_id` linking feedback back to original prediction.
43	`    predicted_class: Mapped[str] = mapped_column(String(128), nullable=False)`	Snapshot of what the model originally predicted.
44	`    is_correct: Mapped[bool] = mapped_column(Boolean, nullable=False)`	Boolean confirmation (`True` = accurate, `False` = misclassified).
45	`    corrected_class: Mapped[str | None] = mapped_column(String(128), nullable=True)`	Optional user-supplied canonical class label when `is_correct=False`.
46	`    comment: Mapped[str | None] = mapped_column(Text, nullable=True)`	Optional unconstrained text note (`Text`) for notes/symptoms.
47	`    created_at: Mapped[datetime] = mapped_column(`	Timezone-aware UTC creation timestamp.
48	`        DateTime(timezone=True), default=lambda: datetime.now(UTC)`	Defaults to current UTC time.
49	`    )`	Closes `mapped_column`.
50		*Blank visual separator line.*

9.3 Pydantic v2 API Request/Response Contracts — backend/app/schemas/__init__.py

File Summary: Enforces strict request validation and OpenAPI JSON response schemas across `/health`, `/classes`, `/advice/{class_id}`, `/predict`, and `/feedback`.

backend/app/schemas/__init__.py (Lines 1–82)

Line	Source Code	Line-by-Line Explanation
1	`"""Strict Pydantic v2 request/response schemas for the prediction API."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from pydantic import BaseModel, Field`	Imports Pydantic v2 `BaseModel` and `Field` validators.
6		*Blank visual separator line.*
7		*Blank visual separator line.*
8	`class Top3Entry(BaseModel):`	Defines `Top3Entry` schema representing one ranked candidate.

Line	Source Code	Line-by-Line Explanation
9	`    """Single entry in the Top-3 prediction list."""`	Docstring.
10		*Blank visual separator line.*
11	`    rank: int = Field(..., ge=1, le=3, description="Rank 1–3 (1 = most likely)")`	`rank` constrained strictly to integer `[1, 3]`.
12	`    class_id: str = Field(..., description="Canonical class label (Crop___Disease)")`	`class_id` canonical class string.
13	`    confidence: float = Field(..., ge=0.0, le=1.0, description="Calibrated probability")`	`confidence` constrained strictly inside `[0.0, 1.0]`.
14		*Blank visual separator line.*
15		*Blank visual separator line.*
16	`class PredictionResponse(BaseModel):`	Defines full `PredictionResponse` returned by `POST /predict`.
17	`    """Successful prediction result."""`	Docstring.
18		*Blank visual separator line.*
19	`    request_id: str = Field(..., description="Unique request trace ID")`	Unique hex correlation ID injected by middleware.
20	`    label: str = Field(..., description="Predicted class (Crop___Disease or 'Unknown')")`	Predicted disease label (`Crop___Disease` or `'Unknown'`).
21	`    confidence: float = Field(..., ge=0.0, le=1.0, description="Calibrated top-1 probability")`	Calibrated top-1 probability in `[0.0, 1.0]`.
22	`    top3: list[Top3Entry] = Field(..., description="Top-3 ranked predictions")`	Ranked list of Top-3 `Top3Entry` items.
23	`    gradcam: str | None = Field(None, description="Base64-encoded PNG Grad-CAM overlay")`	Optional base64-encoded PNG Grad-CAM overlay (None when OOD).
24	`    advice: AdviceEntry | None = Field(None, description="Treatment & prevention advice card")`	Optional nested `AdviceEntry` object (forward-referenced before line 30).
25	`    is_uncertain: bool = Field(False, description="True when model confidence is below threshold")`	Low-confidence uncertainty warning flag.
26	`    is_ood: bool = Field(False, description="True when input is likely out-of-distribution")`	Out-of-distribution rejection flag.
27	`    latency_ms: float = Field(..., description="Server-side inference latency in milliseconds")`	Server-side latency in milliseconds.
28		*Blank visual separator line.*
29		*Blank visual separator line.*
30	`class AdviceEntry(BaseModel):`	Defines `AdviceEntry` agronomic guidance schema.
31	`    """Structured advice card returned alongside predictions."""`	Docstring.
32		*Blank visual separator line.*
33	`    severity: str = Field("moderate", description="low | moderate | high | critical | healthy")`	Severity badge (`low`, `moderate`, `high`, `critical`, `healthy`).
34	`    symptoms: str = Field(..., description="Visible symptom description")`	Visual symptom description string.
35	`    organic_treatment: list[str] = Field(default_factory=list)`	List of organic/cultural treatment recommendations.
36	`    chemical_treatment: list[str] = Field(default_factory=list)`	List of chemical active-ingredient guidelines.
37	`    prevention: list[str] = Field(default_factory=list)`	List of preventive crop management practices.
38		*Blank visual separator line.*
39		*Blank visual separator line.*
40	`# Rebuild PredictionResponse now that AdviceEntry is defined`	Comment explaining forward-reference resolution.
41	`PredictionResponse.model_rebuild()`	Rebuilds `PredictionResponse` schema now that `AdviceEntry` class exists (required due to `from __future__ import annotations`).
42		*Blank visual separator line.*
43		*Blank visual separator line.*
44	`class FeedbackRequest(BaseModel):`	Defines `FeedbackRequest` validated body for `POST /feedback`.
45	`    """User-submitted label correction for a prior prediction."""`	Docstring.
46		*Blank visual separator line.*
47	`    request_id: str = Field(..., min_length=1, description="Request ID of the prediction to correct")`	Non-empty `request_id` string.
48	`    is_correct: bool = Field(..., description="Whether the prediction was correct")`	Required boolean confirmation flag.
49	`    corrected_class: str | None = Field(None, description="Correct label if is_correct is False")`	Optional corrected canonical label.
50	`    comment: str | None = Field(None, max_length=500, description="Optional free-text comment")`	Optional user comment capped at `500` characters.
51		*Blank visual separator line.*
52		*Blank visual separator line.*
53	`class FeedbackResponse(BaseModel):`	Defines `FeedbackResponse` acknowledgment payload.
54	`    """Acknowledgement of saved feedback."""`	Docstring.
55		*Blank visual separator line.*
56	`    status: str = "ok"`	Status string defaulting to `'ok'`.
57	`    message: str = "Feedback recorded. Thank you!"`	Confirmation message string.
58		*Blank visual separator line.*
59		*Blank visual separator line.*
60	`class HealthResponse(BaseModel):`	Defines `HealthResponse` returned by `GET /health`.
61	`    """Health-check response."""`	Docstring.
62		*Blank visual separator line.*
63	`    status: str = "healthy"`	Server health status (`'healthy'` vs `'degraded'`).
64	`    model_loaded: bool = False`	True once ONNX Runtime session is ready.

Line	Source Code	Line-by-Line Explanation
65	`    num_classes: int = 0`	Total loaded disease classes.
66	`    backend_version: str = "1.0.0"`	Static backend API version string.
67		*Blank visual separator line.*
68		*Blank visual separator line.*
69	`class ClassListResponse(BaseModel):`	Defines `ClassListResponse` returned by `GET /classes`.
70	`    """List of all known classes."""`	Docstring.
71		*Blank visual separator line.*
72	`    num_classes: int`	Class count.
73	`    classes: list[str]`	Ordered canonical class list.
74		*Blank visual separator line.*
75		*Blank visual separator line.*
76	`class ErrorResponse(BaseModel):`	Defines structured `ErrorResponse` returned for 4xx/5xx handlers.
77	`    """Structured API error format."""`	Docstring.
78		*Blank visual separator line.*
79	`    error: str`	Short error category.
80	`    detail: str = ""`	Detailed error explanation.
81	`    request_id: str | None = None`	Traceable `request_id`.
82		*Blank visual separator line.*

9.4 ONNX Runtime Inference, Magic-Byte Security & CAM Renderer — backend/app/services/inference.py

File Summary: Validates uploaded image magic bytes, loads CPU-optimized ONNX Runtime sessions, replicates eval center-crop preprocessing, runs optional PlantGate + main classifier, applies temperature calibration + Helmholtz energy OOD gating, and encodes JET Grad-CAM PNG overlays in base64.

backend/app/services/inference.py (Lines 1–254)

Line	Source Code	Line-by-Line Explanation
1	`"""ONNX Runtime inference service with preprocessing, calibration, OOD, and Grad-CAM."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`import base64`	Imports `base64` for inline PNG encoding inside JSON.
6	`import io`	Imports `io` (`BytesIO`) avoiding disk temp files during decode/encode.
7	`import os`	Imports `os` (`cpu_count`).
8	`import time`	Imports `time` (`perf_counter`) for sub-ms latency telemetry.
9	`from pathlib import Path`	Imports `Path`.
10	`from typing import Any`	Imports `Any`.
11		*Blank visual separator line.*
12	`import cv2`	Imports OpenCV (`cv2`) for fast JET colormap and bilinear resize.
13	`import numpy as np`	Imports NumPy.
14	`import onnxruntime as ort`	Imports `onnxruntime as ort` for PyTorch-free CPU inference.
15	`from PIL import Image, ImageOps`	Imports PIL `Image`, `ImageOps`.
16		*Blank visual separator line.*
17	`from app.core import settings`	Imports backend `settings` singleton.
18	`from app.core.logging import logger`	Imports structured backend `logger`.
19		*Blank visual separator line.*
20	`# Allowed MIME types & magic bytes for upload validation`	Comment: allowed image formats & magic-byte signatures.
21	`ALLOWED_MIMES = {"image/jpeg", "image/png", "image/webp", "image/bmp"}`	Set of supported MIME types.
22	`MAGIC_HEADERS = {`	Dictionary mapping file magic header bytes to MIME types (defends against renamed `.exe`/spoofed extensions).
23	`    b"\xff\xd8\xff": "image/jpeg",`	JPEG SOI marker (`FF D8 FF`).
24	`    b"\x89PNG": "image/png",`	PNG magic header (`89 50 4E 47`).
25	`    b"RIFF": "image/webp",  # WebP wraps in RIFF container`	WebP RIFF container magic.
26	`    b"BM": "image/bmp",`	BMP `'BM'` magic.
27	`}`	Closes `MAGIC_HEADERS`.
28		*Blank visual separator line.*
29		*Blank visual separator line.*
30	`def validate_image_bytes(data: bytes, max_bytes: int) -> tuple[bool, str]:`	Defines `validate_image_bytes(data, max_bytes)` checking payload size and file header.
31	`    """Validate upload is an allowed image type and within size limits."""`	Docstring.
32	`    if len(data) > max_bytes:`	Enforces `max_upload_mb` ceiling (`10 MB`).
33	`        return False, f"File size ({len(data)} bytes) exceeds {max_bytes // (1024 * 1024)} MB limit."`	Returns `(False, reason)` on oversize upload.
34	`    if len(data) < 8:`	Rejects truncated `< 8` byte payloads.
35	`        return False, "File is too small to be a valid image."`	Returns invalid size message.
36		*Blank visual separator line.*

Line	Source Code	Line-by-Line Explanation
37	`    header = data[:8]`	Inspects first 8 bytes of raw payload.
38	`    detected = None`	Initializes detected MIME sentinel.
39	`    for magic, mime in MAGIC_HEADERS.items():`	Loops over known image magic signatures.
40	`        if header[: len(magic)] == magic:`	Checks prefix match against magic bytes.
41	`            detected = mime`	Stores matched MIME.
42	`            break`	Stops scanning on first match.
43	`    if detected is None:`	Rejects unknown/unsupported binary formats.
44	`        return False, "Unsupported image format. Upload JPEG, PNG, WebP, or BMP."`	Returns friendly format guidance.
45	`    return True, detected`	Returns `(True, detected_mime)`.
46		*Blank visual separator line.*
47		*Blank visual separator line.*
48	`class InferenceService:`	Defines stateful `InferenceService` singleton powering `/predict`.
49	`    """Production inference engine backed by ONNX Runtime with dual-output CAM support."""`	Docstring.
50		*Blank visual separator line.*
51	`    def __init__(self) -> None:`	Constructor initializing default settings prior to `load()`.
52	`        self.session: ort.InferenceSession | None = None`	Main classifier ONNX Runtime session.
53	`        self.gate_session: ort.InferenceSession | None = None`	Optional `PlantGate` ONNX session.
54	`        self.meta: dict[str, Any] = {}`	Parsed `model_meta.json` dictionary.
55	`        self.classes: list[str] = []`	Ordered canonical class list.
56	`        self.num_classes: int = 0`	Total class count.
57	`        self.image_size: int = 224`	Model input resolution (`224`).
58	`        self.mean: np.ndarray = np.array([0.485, 0.456, 0.406], dtype=np.float32)`	Default ImageNet channel means.
59	`        self.std: np.ndarray = np.array([0.229, 0.224, 0.225], dtype=np.float32)`	Default ImageNet channel stds.
60	`        self.temperature: float = 1.0`	Default unscaled temperature `T = 1.0`.
61	`        self.conf_threshold: float = 0.55`	Default uncertainty threshold (`0.55`).
62	`        self.energy_threshold: float = -3.5`	Default energy OOD threshold (`-3.5`).
63	`        self.plant_gate_threshold: float = 0.45`	Default binary PlantGate threshold (`0.45`).
64		*Blank visual separator line.*
65	`    @property`	Property indicating whether primary ONNX model is ready to serve.
66	`    def loaded(self) -> bool:`	Getter.
67	`        return self.session is not None`	True when `self.session` is initialized.
68		*Blank visual separator line.*
69	`    def load(self) -> None:`	Defines `load()` executed once during FastAPI lifespan startup.
70	`        """Load ONNX model, metadata, and optional plant gate."""`	Docstring.
71	`        settings.ensure_artifacts()`	Pulls missing artifacts from HF Hub if configured.
72	`        onnx_path = Path(settings.model_onnx_path)`	Path to main `model.onnx`.
73	`        meta_path = Path(settings.model_meta_path)`	Path to `model_meta.json`.
74		*Blank visual separator line.*
75	`        if not onnx_path.exists():`	Handles missing ONNX gracefully (server starts in degraded mode so `/health` stays responsive).
76	`            logger.warning("Model ONNX file not found at %s; inference disabled.", onnx_path)`	Logs warning.
77	`            return`	Returns early.
78		*Blank visual separator line.*
79	`        sess_opts = ort.SessionOptions()`	Creates ONNX Runtime `SessionOptions`.
80	`        sess_opts.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL`	Enables all graph fusions & constant folding (`ORT_ENABLE_ALL`).
81	`        sess_opts.intra_op_num_threads = min(4, max(1, os.cpu_count() or 1))`	Caps intra-op threads at `min(4, cpu_count)` avoiding container thread contention.
82	`        sess_opts.inter_op_num_threads = 1`	Sets `inter_op_num_threads = 1` optimal for sequential CNN/ViT inference.
83		*Blank visual separator line.*
84	`        self.session = ort.InferenceSession(`	Instantiates primary CPU ONNX InferenceSession.
85	`            str(onnx_path),`	Model file string.
86	`            sess_options=sess_opts,`	Session tuning options.
87	`            providers=["CPUExecutionProvider"],`	Locks execution to `CPUExecutionProvider` (zero GPU dependency in serving container!).
88	`        )`	Closes `InferenceSession`.
89	`        logger.info("Loaded main ONNX model from %s", onnx_path)`	Logs successful model load.
90		*Blank visual separator line.*
91	`        if meta_path.exists():`	Loads `model_meta.json` synchronizing preprocessing, classes, temperature, and OOD thresholds with training.
92	`            import json`	Imports standard `json`.
93		*Blank visual separator line.*
94	`            with meta_path.open("r", encoding="utf-8") as f:`	Opens metadata JSON file in UTF-8.
95	`                self.meta = json.load(f)`	Parses JSON object.
96	`            self.classes = list(self.meta.get("classes", []))`	Loads canonical `classes` list.
97	`            self.num_classes = len(self.classes)`	Updates `num_classes`.

Line	Source Code	Line-by-Line Explanation
98	`            self.image_size = int(self.meta.get("image_size", 224))`	Updates `image_size`.
99	`            self.mean = np.array(self.meta.get("mean", [0.485, 0.456, 0.406]), dtype=np.float32)`	Loads normalization mean vector.
100	`            self.std = np.array(self.meta.get("std", [0.229, 0.224, 0.225]), dtype=np.float32)`	Loads normalization std vector.
101	`            self.temperature = float(self.meta.get("temperature", 1.0))`	Loads calibrated temperature scalar `T`.
102	`            self.conf_threshold = float(`	Loads tuned uncertainty confidence cutoff (falling back to `.env` setting).
103	`                self.meta.get("confidence_threshold", settings.uncertainty_conf_threshold)`	Lookup expression.
104	`            )`	Closes `float(...)`.
105	`            self.energy_threshold = float(`	Loads tuned Helmholtz energy OOD cutoff.
106	`                self.meta.get("energy_threshold", settings.ood_energy_threshold)`	Lookup expression.
107	`            )`	Closes `float(...)`.
108	`            self.plant_gate_threshold = float(`	Loads PlantGate probability threshold.
109	`                self.meta.get("plant_gate_threshold", settings.plant_gate_threshold)`	Lookup expression.
110	`            )`	Closes `float(...)`.
111	`        else:`	Fallback branch if metadata JSON is absent.
112	`            logger.warning("model_meta.json not found; using default thresholds.")`	Logs fallback warning.
113		*Blank visual separator line.*
114	`        # Load optional plant gate`	Comment: optionally load `plant_gate.onnx` Stage-1 filter.
115	`        gate_path = Path(settings.plant_gate_onnx_path)`	Path to `plant_gate.onnx`.
116	`        if gate_path.exists():`	Loads gate ONNX session only if file exists on disk.
117	`            self.gate_session = ort.InferenceSession(`	Creates gate CPU InferenceSession.
118	`                str(gate_path),`	Path string.
119	`                sess_options=sess_opts,`	Reuses tuned SessionOptions.
120	`                providers=["CPUExecutionProvider"],`	CPU provider.
121	`            )`	Closes gate session.
122	`            logger.info("Loaded plant gate ONNX from %s", gate_path)`	Logs gate availability.
123		*Blank visual separator line.*
124	`    def preprocess(self, image_bytes: bytes) -> tuple[np.ndarray, np.ndarray]:`	Defines `preprocess(image_bytes)` replicating `build_eval_transforms` identically without PyTorch/Albumentations overhead.
125	`        """Decode, EXIF-transpose, resize, center-crop, normalize to `(1, 3, H, W)`."""`	Docstring.
126	`        img = Image.open(io.BytesIO(image_bytes))`	Decodes raw upload bytes in memory via `BytesIO`.
127	`        img = ImageOps.exif_transpose(img).convert("RGB")`	Fixes mobile EXIF rotation and converts image to 3-channel RGB.
128	`        rgb_np = np.array(img)`	Retains original-resolution RGB array for crisp Grad-CAM overlay.
129		*Blank visual separator line.*
130	`        sz = self.image_size`	Target crop side (`224`).
131	`        rsz = int(round(sz * 1.14))`	Computes exact `256` resize dim (`round(224 * 1.14)` matching training eval!).
132	`        img_resized = img.resize((rsz, rsz), Image.Resampling.BILINEAR)`	Bilinear resize to `(256, 256)`.
133	`        arr = np.array(img_resized, dtype=np.float32)`	Converts resized image to `float32` array.
134		*Blank visual separator line.*
135	`        # Center crop`	Comment: exact center crop.
136	`        h, w = arr.shape[:2]`	Reads height and width (`256, 256`).
137	`        y0 = (h - sz) // 2`	Computes top crop offset `16`.
138	`        x0 = (w - sz) // 2`	Computes left crop offset `16`.
139	`        arr = arr[y0 : y0 + sz, x0 : x0 + sz]`	Slices centered `(224, 224, 3)` crop.
140		*Blank visual separator line.*
141	`        # Normalize to ImageNet stats`	Comment: normalize pixels cleanly.
142	`        arr = arr / 255.0`	Scales `[0, 255]` into `[0.0, 1.0]`.
143	`        arr = (arr - self.mean[None, None, :]) / self.std[None, None, :]`	Subtracts mean and divides by std across channels.
144	`        tensor = arr.transpose(2, 0, 1).astype(np.float32)[np.newaxis]`	Permutes `HWC -> CHW`, casts `float32`, and adds batch axis -> `(1, 3, 224, 224)`.
145	`        return tensor, rgb_np`	Returns `(onnx_input_tensor, original_rgb_np)`.
146		*Blank visual separator line.*
147	`    def _softmax(self, logits: np.ndarray) -> np.ndarray:`	Defines numerically stable `_softmax(logits)` subtracting max logit before `exp`.
148	`        x = logits - logits.max(axis=-1, keepdims=True)`	Shifts logits by max to prevent float64 overflow.
149	`        e = np.exp(x)`	Exponentiates shifted logits.
150	`        return e / e.sum(axis=-1, keepdims=True)`	Normalizes probabilities to sum to `1.0`.
151		*Blank visual separator line.*
152	`    def _energy_score(self, logits: np.ndarray) -> float:`	Defines `_energy_score(logits)` computing `-logsumexp(z)` with max-shift stability (`E(x) = -(max + log(sum(exp(z - max))))`).
153	`        return -float(np.log(np.sum(np.exp(logits - logits.max()))) + logits.max())`	Evaluates negative Helmholtz free energy scalar.
154		*Blank visual separator line.*
155	`    def _render_gradcam_b64(`	Defines `_render_gradcam_b64` converting ONNX `cam_maps` into base64 PNG.
156	`        self,`	`self` reference.
157	`        cam_maps: np.ndarray,`	ONNX `cam_maps` tensor `(1, K, H_f, W_f)`.

Line	Source Code	Line-by-Line Explanation
158	`        pred_idx: int,`	Predicted class index.
159	`        original_rgb: np.ndarray,`	Original RGB photo.
160	`        alpha: float = 0.45,`	Heatmap overlay alpha (`0.45`).
161	`    ) -> str:`	Returns base64 ASCII PNG string.
162	`        """Render Grad-CAM overlay from ONNX `cam_maps` output and return base64 PNG."""`	Docstring.
163	`        cam = cam_maps[0, pred_idx]  # (H_f, W_f)`	Slices `(H_f, W_f)` activation map strictly for the winning class `pred_idx` .
164	`        c_min, c_max = float(cam.min()), float(cam.max())`	Computes spatial min and max.
165	`        if c_max - c_min > 1e-8:`	Guards against uniform flat CAM maps.
166	`            cam_norm = ((cam - c_min) / (c_max - c_min)).astype(np.float32)`	Min-max normalizes CAM into `[0.0, 1.0]` .
167	`        else:`	Flat fallback.
168	`            cam_norm = np.zeros_like(cam, dtype=np.float32)`	Zero heatmap.
169		*Blank visual separator line.*
170	`        h, w = original_rgb.shape[:2]`	Reads original height/width (unused local alias).
171	`        disp = cv2.resize(original_rgb, (224, 224), interpolation=cv2.INTER_LINEAR)`	Resizes uploaded photo to crisp `(224, 224)` preview dimensions.
172	`        cam_resized = cv2.resize(cam_norm, (224, 224), interpolation=cv2.INTER_LINEAR)`	Bilinearly upsamples coarse `(7x7)` CAM grid to `(224, 224)` .
173	`        heatmap = cv2.applyColorMap(np.uint8(cam_resized * 255), cv2.COLORMAP_JET)`	Applies OpenCV `COLORMAP_JET` false-color ramp.
174	`        heatmap_rgb = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)`	Converts OpenCV BGR colormap output to RGB.
175	`        blended = np.clip(`	Alpha-blends original photo with heatmap and clamps `[0, 255]` `uint8` .
176	`            (1 - alpha) * disp.astype(np.float32) + alpha * heatmap_rgb.astype(np.float32),`	Blend formula `(1 - alpha) * image + alpha * heatmap` .
177	`            0,`	Lower clamp bound.
178	`            255,`	Upper clamp bound.
179	`        ).astype(np.uint8)`	Casts to `uint8` .
180		*Blank visual separator line.*
181	`        pil_out = Image.fromarray(blended)`	Wraps NumPy overlay into PIL Image.
182	`        buf = io.BytesIO()`	Creates in-memory PNG byte buffer.
183	`        pil_out.save(buf, format="PNG", optimize=True)`	Encodes optimized PNG image into buffer.
184	`        return base64.b64encode(buf.getvalue()).decode("ascii")`	Encodes PNG bytes to URL-safe base64 ASCII string.
185		*Blank visual separator line.*
186	`    def predict(self, image_bytes: bytes) -> dict[str, Any]:`	Defines full request inference pipeline `predict(image_bytes)` .
187	`        """Run full inference pipeline: preprocess → gate → classify → calibrate → OOD → CAM."""`	Docstring.
188	`        if self.session is None:`	Guards against calling `predict` before ONNX session loads.
189	`            raise RuntimeError("Model not loaded. Call load() during application startup.")`	Raises RuntimeError.
190		*Blank visual separator line.*
191	`        t_start = time.perf_counter()`	Starts high-resolution latency timer.
192	`        tensor, rgb_np = self.preprocess(image_bytes)`	Decodes bytes and builds normalized `(1, 3, 224, 224)` NCHW tensor.
193		*Blank visual separator line.*
194	`        # Optional plant gate check`	Comment: Gate 1 — optional binary PlantGate OOD check.
195	`        is_ood_gate = False`	Initializes gate rejection flag `False` .
196	`        if self.gate_session is not None:`	Executes PlantGate if `plant_gate.onnx` loaded.
197	`            gate_logit = self.gate_session.run(["plant_logit"], {"image": tensor})[0]`	Runs gate ONNX inference returning scalar `plant_logit` .
198	`            gate_prob = float(1.0 / (1.0 + np.exp(-gate_logit.item())))`	Computes plant probability `sigma(plant_logit) = 1 / (1 + exp(-z))` .
199	`            if gate_prob < self.plant_gate_threshold:`	Flags input as OOD if below `plant_gate_threshold` (`0.45`).
200	`                is_ood_gate = True`	Sets `is_ood_gate = True` .
201		*Blank visual separator line.*
202	`        # Main model inference`	Comment: Stage 2 — single-pass main classifier + CAM execution.
203	`        logits, cam_maps = self.session.run(["logits", "cam_maps"], {"image": tensor})`	Runs `self.session.run` returning both `logits` and spatial `cam_maps` simultaneously.
204	`        logits_1d = logits[0].astype(np.float64)`	Casts batch-0 logits to `float64` for softmax/energy numerical accuracy.
205		*Blank visual separator line.*
206	`        # Calibrated probabilities via temperature scaling`	Comment: apply fitted temperature divisor `T` before softmax.
207	`        probs = self._softmax(logits_1d / max(self.temperature, 1e-6))`	Computes calibrated posterior probabilities `softmax(logits / T)` .
208	`        pred_idx = int(np.argmax(probs))`	Finds top-1 class index.
209	`        confidence = float(probs[pred_idx])`	Reads top-1 calibrated confidence.
210		*Blank visual separator line.*
211	`        # Energy-based OOD score`	Comment: compute Helmholtz energy OOD score from unscaled logits.
212	`        energy = self._energy_score(logits_1d)`	Computes scalar energy `E(x)` .
213	`        is_uncertain = confidence < self.conf_threshold`	Flags `is_uncertain` when calibrated confidence `< conf_threshold` .
214	`        is_ood = is_ood_gate or (energy > self.energy_threshold)`	Flags `is_ood` when either PlantGate failed OR `E(x) > energy_threshold` .
215		*Blank visual separator line.*
216	`        # Top-3`	Comment: extract top-3 candidate predictions.
217	`        top3_idx = np.argsort(probs)[::-1][:3]`	Sorts probability indices descending and slices top 3.

Line	Source Code	Line-by-Line Explanation
218	`        top3 = [`	Builds structured Top-3 candidate dictionaries.
219	`            {`	Starts dict.
220	`                "rank": rank + 1,`	1-based rank (`1, 2, 3`).
221	`                "class_id": self.classes[int(idx)] if int(idx) < self.num_classes else "Unknown",`	Resolves canonical class name safely.
222	`                "confidence": round(float(probs[int(idx)]), 4),`	Rounds confidence to 4 decimal places.
223	`            }`	Closes dict.
224	`            for rank, idx in enumerate(top3_idx)`	Comprehension loop.
225	`        ]`	Closes list.
226		*Blank visual separator line.*
227	`        label = self.classes[pred_idx] if pred_idx < self.num_classes else "Unknown"`	Resolves primary label string.
228	`        if is_ood:`	Overrides output to `'Unknown'` + uncertain warning when input is flagged OOD.
229	`            label = "Unknown"`	Masks label as `'Unknown'` .
230	`            is_uncertain = True`	Forces `is_uncertain = True` .
231		*Blank visual separator line.*
232	`        # Render Grad-CAM overlay`	Comment: render Grad-CAM overlay only for in-distribution crop photos.
233	`        gradcam_b64: str | None = None`	Initializes base64 CAM output to `None` .
234	`        if cam_maps is not None and not is_ood:`	Skips Grad-CAM rendering if input was rejected as OOD.
235	`            try:`	Tries CAM rendering defensively so visualization issues never crash diagnosis.
236	`                gradcam_b64 = self._render_gradcam_b64(cam_maps, pred_idx, rgb_np)`	Renders base64 overlay PNG.
237	`            except Exception as exc:`	Catches unexpected CV2/PIL errors.
238	`                logger.warning("Grad-CAM rendering failed: %s", exc)`	Logs warning and serves text diagnosis cleanly.
239		*Blank visual separator line.*
240	`        latency_ms = round((time.perf_counter() - t_start) * 1000.0, 2)`	Computes elapsed server latency in milliseconds rounded to `0.01 ms` .
241	`        return {`	Returns clean inference payload to API route handler.
242	`            "label": label,`	Final label.
243	`            "confidence": round(confidence, 4),`	Calibrated confidence.
244	`            "top3": top3,`	Top-3 candidate list.
245	`            "gradcam": gradcam_b64,`	Optional base64 PNG Grad-CAM.
246	`            "is_uncertain": is_uncertain,`	Uncertainty flag.
247	`            "is_ood": is_ood,`	OOD flag.
248	`            "latency_ms": latency_ms,`	Server latency in ms.
249	`        }`	Closes return dict.
250		*Blank visual separator line.*
251		*Blank visual separator line.*
252	`# Module-level singleton`	Module singleton comment.
253	`inference_service = InferenceService()`	Creates module-level `inference_service` imported by FastAPI lifecycle and routes.
254		*Blank visual separator line.*

9.5 FastAPI Endpoint Handlers — backend/app/api/__init__.py

File Summary: Implements `/health` , `/classes` , `/advice/{class_id}` , `/predict` (upload validation, ONNX inference, advice join, SQLite logging), and `/feedback` active-learning submission.

backend/app/api/__init__.py (Lines 1–168)

Line	Source Code	Line-by-Line Explanation
1	`"""API route definitions for the Crop Disease & Pest Classifier."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from fastapi import APIRouter, Depends, File, HTTPException, UploadFile, status`	Imports FastAPI routing, Dependency Injection, UploadFile, and HTTP status codes.
6	`from sqlalchemy.orm import Session`	Imports SQLAlchemy `Session` type.
7		*Blank visual separator line.*
8	`from app.core.logging import logger, request_id_var`	Imports structured logger and per-request contextvar `request_id_var` .
9	`from app.db import get_db`	Imports `get_db` generator dependency yielding scoped DB sessions.
10	`from app.schemas import (`	Imports strict Pydantic v2 request/response schemas.
11	`    AdviceEntry,`	`AdviceEntry` schema.
12	`    ClassListResponse,`	`ClassListResponse` schema.
13	`    ErrorResponse,`	`ErrorResponse` schema.
14	`    FeedbackRequest,`	`FeedbackRequest` schema.
15	`    FeedbackResponse,`	`FeedbackResponse` schema.
16	`    HealthResponse,`	`HealthResponse` schema.
17	`    PredictionResponse,`	`PredictionResponse` schema.
18	`    Top3Entry,`	`Top3Entry` schema.

Line	Source Code	Line-by-Line Explanation
19	`)`	Closes schema imports.
20	`from app.services.advice import advice_service`	Imports `advice_service` JSON lookup singleton.
21	`from app.services.inference import InferenceService, inference_service, validate_image_bytes`	Imports `inference_service` and magic-byte validator.
22	`from app.services.storage import save_prediction_log, save_user_feedback`	Imports `save_prediction_log` and `save_user_feedback` DB CRUD helpers.
23	`from app.core import settings`	Imports backend `settings`.
24		*Blank visual separator line.*
25	`router = APIRouter()`	Instantiates FastAPI `APIRouter` mounted by `main.py`.
26		*Blank visual separator line.*
27		*Blank visual separator line.*
28	`@router.get("/health", response_model=HealthResponse, tags=["System"])`	Registers `GET /health` endpoint.
29	`def health_check() -> HealthResponse:`	Defines health check route handler.
30	`    """Liveness & readiness probe."""`	Docstring.
31	`    return HealthResponse(`	Returns `'healthy'` + class count if model loaded, else `'degraded'`.
32	`        status="healthy" if inference_service.loaded else "degraded",`	Status conditional.
33	`        model_loaded=inference_service.loaded,`	Boolean readiness flag.
34	`        num_classes=inference_service.num_classes,`	Loaded class count.
35	`    )`	Closes response.
36		*Blank visual separator line.*
37		*Blank visual separator line.*
38	`@router.get("/classes", response_model=ClassListResponse, tags=["System"])`	Registers `GET /classes` endpoint.
39	`def list_classes() -> ClassListResponse:`	Defines class listing handler.
40	`    """Return the canonical ordered list of disease/pest classes the model recognises."""`	Docstring.
41	`    return ClassListResponse(`	Returns ordered class taxonomy for UI dropdowns/autocomplete.
42	`        num_classes=inference_service.num_classes,`	Class count.
43	`        classes=inference_service.classes,`	Canonical class list.
44	`    )`	Closes response.
45		*Blank visual separator line.*
46		*Blank visual separator line.*
47	`@router.get(`	Registers `GET /advice/{class_id}` endpoint with OpenAPI 404 schema.
48	`    "/advice/{class_id}",`	URL path pattern with `class_id` parameter.
49	`    response_model=AdviceEntry | None,`	Response schema.
50	`    responses={404: {"model": ErrorResponse}},`	Documents 404 error shape.
51	`    tags=["Advice"],`	OpenAPI Swagger tag.
52	`)`	Closes decorator.
53	`def get_advice(class_id: str) -> AdviceEntry:`	Defines agronomic advice lookup handler.
54	`    """Retrieve treatment & prevention advice for a specific disease/pest class."""`	Docstring.
55	`    entry = advice_service.get_advice(class_id)`	Queries in-memory `advice_kb.json` index.
56	`    if entry is None:`	Raises HTTP `404` if unknown label.
57	`        raise HTTPException(`	Constructs `HTTPException`.
58	`            status_code=status.HTTP_404_NOT_FOUND,`	404 Not Found status.
59	`            detail=f"No advice found for class '{class_id}'.",`	Descriptive detail message.
60	`        )`	Closes exception.
61	`    return AdviceEntry(**entry)`	Wraps dict cleanly into validated `AdviceEntry`.
62		*Blank visual separator line.*
63		*Blank visual separator line.*
64	`@router.post(`	Registers `POST /predict` inference endpoint.
65	`    "/predict",`	Route path.
66	`    response_model=PredictionResponse,`	Strict `PredictionResponse` output model.
67	`    responses={`	OpenAPI error status documentation (`400`, `422`, `503`).
68	`        400: {"model": ErrorResponse},`	Bad upload error.
69	`        422: {"model": ErrorResponse},`	Corrupt/unprocessable image error.
70	`        503: {"model": ErrorResponse},`	Model not loaded error.
71	`    },`	Closes `responses`.
72	`    tags=["Prediction"],`	OpenAPI Swagger tag.
73	`)`	Closes decorator.
74	`async def predict(`	Async route handler accepting multipart `file` and injecting SQLAlchemy `db` session.
75	`    file: UploadFile = File(..., description="Crop leaf / pest image (JPEG, PNG, WebP, BMP, ≤10 MB)"),`	Multipart file upload parameter.
76	`    db: Session = Depends(get_db),`	Injected per-request DB session.
77	`) -> PredictionResponse:`	Return annotation.
78	`    """Upload a crop image and receive a calibrated disease/pest prediction with Grad-CAM."""`	Docstring.
79	`    rid = request_id_var.get("-")`	Reads unique request trace ID generated by HTTP middleware.

Line	Source Code	Line-by-Line Explanation
80		*Blank visual separator line.*
81	`    if not inference_service.loaded:`	Fails fast with `503 Service Unavailable` if model weights were not loaded.
82	`        raise HTTPException(`	Raises `HTTPException`.
83	`            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,`	503 status.
84	`            detail="Model is not loaded. The server is starting or misconfigured.",`	Clear error message.
85	`        )`	Closes exception.
86		*Blank visual separator line.*
87	`    # Read and validate upload`	Comment: read uploaded bytes asynchronously and verify size + magic header.
88	`    data = await file.read()`	Reads raw binary upload from async `UploadFile`.
89	`    ok, msg = validate_image_bytes(data, settings.max_upload_bytes)`	Checks `<=10 MB` cap and JPEG/PNG/WebP/BMP magic bytes.
90	`    if not ok:`	Rejects invalid uploads with `400 Bad Request` before passing anything to PIL/ONNX.
91	`        logger.warning("Upload rejected: %s", msg)`	Logs rejected upload reason.
92	`        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail=msg)`	Raises `400` HTTPException.
93		*Blank visual separator line.*
94	`    # Run inference`	Comment: run CPU ONNX Runtime inference.
95	`    try:`	Tries inference safely.
96	`        result = inference_service.predict(data)`	Executes full `InferenceService.predict(data)` pipeline.
97	`    except Exception as exc:`	Catches corrupt/truncated pixel decode or ONNX errors.
98	`        logger.error("Inference failed: %s", exc, exc_info=True)`	Logs stack trace.
99	`        raise HTTPException(`	Converts failure to `422 Unprocessable Entity`.
100	`            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,`	422 status.
101	`            detail=f"Could not process image: {exc}",`	Error detail.
102	`        ) from exc`	Preserves exception cause.
103		*Blank visual separator line.*
104	`    # Look up advice`	Comment: join agronomic treatment guidance for non-OOD predictions.
105	`    advice_entry: AdviceEntry | None = None`	Defaults advice to `None`.
106	`    if not result["is_ood"]:`	Looks up treatment card only when input is recognized in-distribution plant/pest.
107	`        raw_advice = advice_service.get_advice(result["label"])`	Fetches advice dictionary for predicted `Crop___Disease`.
108	`        if raw_advice:`	Builds validated `AdviceEntry` if entry found.
109	`            advice_entry = AdviceEntry(**raw_advice)`	Instantiates Pydantic `AdviceEntry`.
110		*Blank visual separator line.*
111	`    # Persist prediction log`	Comment: log prediction metadata non-fatally to database.
112	`    try:`	Tries DB insert inside protective try-block (DB glitch must not block returning diagnosis to farmer!).
113	`        save_prediction_log(`	Persists prediction row into `prediction_logs` table.
114	`            db,`	Passes DB session.
115	`            request_id=rid,`	Passes `request_id`.
116	`            result=result,`	Passes result dict.
117	`            filename=file.filename,`	Passes original filename.
118	`            file_size=len(data),`	Passes payload byte length.
119	`        )`	Closes call.
120	`    except Exception as exc:`	Catches transient SQLite/Postgres write failures gracefully.
121	`        logger.warning("Failed to persist prediction log: %s", exc)`	Logs warning.
122		*Blank visual separator line.*
123	`    return PredictionResponse(`	Returns complete validated `PredictionResponse` JSON.
124	`        request_id=rid,`	Trace ID matching `X-Request-ID` header.
125	`        label=result["label"],`	Final class or `'Unknown'`.
126	`        confidence=result["confidence"],`	Calibrated confidence.
127	`        top3=[Top3Entry(**e) for e in result["top3"]],`	Validated Top-3 candidates list.
128	`        gradcam=result.get("gradcam"),`	Optional base64 Grad-CAM PNG.
129	`        advice=advice_entry,`	Agronomic treatment card.
130	`        is_uncertain=result["is_uncertain"],`	Uncertainty flag.
131	`        is_ood=result["is_ood"],`	OOD flag.
132	`        latency_ms=result["latency_ms"],`	Server latency in ms.
133	`    )`	Closes response.
134		*Blank visual separator line.*
135		*Blank visual separator line.*
136	`@router.post(`	Registers `POST /feedback` active-learning endpoint.
137	`    "/feedback",`	Route path.
138	`    response_model=FeedbackResponse,`	Response schema.
139	`    responses={400: {"model": ErrorResponse}},`	Documents `400` validation error.
140	`    tags=["Feedback"],`	Swagger tag.
141	`)`	Closes decorator.

Line	Source Code	Line-by-Line Explanation
142	`def submit_feedback(`	Defines handler saving user diagnosis confirmation or correction.
143	`    body: FeedbackRequest,`	Validated `FeedbackRequest` JSON body.
144	`    db: Session = Depends(get_db),`	Injected DB session.
145	`) -> FeedbackResponse:`	Returns `FeedbackResponse`.
146	`    """Submit user correction or confirmation for a prior prediction."""`	Docstring.
147	`    if not body.is_correct and not body.corrected_class:`	Enforces business rule: rejecting a prediction (`is_correct=False`) requires providing `corrected_class`.
148	`        raise HTTPException(`	Raises `400 Bad Request`.
149	`            status_code=status.HTTP_400_BAD_REQUEST,`	400 status.
150	`            detail="corrected_class is required when is_correct is false.",`	Validation detail message.
151	`        )`	Closes exception.
152		*Blank visual separator line.*
153	`    # Look up the original prediction to record the predicted class`	Comment: enrich feedback row with the original predicted class.
154	`    from app.db.models import PredictionLog`	Imports `PredictionLog` ORM model.
155		*Blank visual separator line.*
156	`    log = db.query(PredictionLog).filter(PredictionLog.request_id == body.request_id).first()`	Queries `prediction_logs` matching submitted `request_id`.
157	`    predicted_class = log.predicted_class if log else "Unknown"`	Extracts original prediction label (or `'Unknown'` if missing).
158		*Blank visual separator line.*
159	`    save_user_feedback(`	Inserts row into `user_feedback` SQL table.
160	`        db,`	Passes DB session.
161	`        request_id=body.request_id,`	Passes `request_id`.
162	`        predicted_class=predicted_class,`	Passes recorded prediction.
163	`        is_correct=body.is_correct,`	Passes correctness boolean.
164	`        corrected_class=body.corrected_class,`	Passes corrected label.
165	`        comment=body.comment,`	Passes optional user comment.
166	`    )`	Closes CRUD call.
167	`    return FeedbackResponse()`	Returns `{'status': 'ok', 'message': ...}`.
168		*Blank visual separator line.*

9.6 FastAPI Application Factory, Lifespan & Security Middleware — backend/app/main.py

File Summary: Bootstraps SQL tables, ONNX Runtime sessions, and advice KB via ASGI lifespan; configures SlowAPI IP rate limiting, CORS, per-request trace IDs, global 500 error masking, and Uvicorn runner.

backend/app/main.py (Lines 1–100)

Line	Source Code	Line-by-Line Explanation
1	`"""FastAPI application entry-point with lifespan, CORS, rate-limiting, and error handlers."""`	Module docstring.
2		*Blank visual separator line.*
3	`from __future__ import annotations`	Enables PEP 563 postponed type annotations.
4		*Blank visual separator line.*
5	`from contextlib import asynccontextmanager`	Imports `asynccontextmanager` for clean FastAPI startup/shutdown lifecycle.
6	`from typing import AsyncGenerator`	Imports `AsyncGenerator` type hint.
7		*Blank visual separator line.*
8	`from fastapi import FastAPI, Request, status`	Imports `FastAPI`, `Request`, and HTTP `status`.
9	`from fastapi.middleware.cors import CORSMiddleware`	Imports Starlette `CORSMiddleware`.
10	`from fastapi.responses import JSONResponse`	Imports `JSONResponse`.
11	`from slowapi import Limiter, _rate_limit_exceeded_handler`	Imports `SlowAPI` rate limiter and default 429 handler.
12	`from slowapi.errors import RateLimitExceeded`	Imports `RateLimitExceeded` exception.
13	`from slowapi.util import get_remote_address`	Imports IP-extraction utility `get_remote_address`.
14		*Blank visual separator line.*
15	`from app.api import router`	Imports API `router`.
16	`from app.core import settings`	Imports `settings`.
17	`from app.core.logging import generate_request_id, logger, request_id_var`	Imports request ID generator, contextvar, and structured logger.
18	`from app.db import engine`	Imports SQLAlchemy `engine`.
19	`from app.db.models import Base`	Imports ORM `Base` metadata for `create_all`.
20	`from app.services.advice import advice_service`	Imports `advice_service`.
21	`from app.services.inference import inference_service`	Imports `inference_service`.
22		*Blank visual separator line.*
23		*Blank visual separator line.*
24	`# --- Lifespan: warm up model, DB, and advice KB once at startup ---`	Section comment: startup/shutdown lifecycle.
25	`@asynccontextmanager`	Registers async context manager as FastAPI `lifespan`.
26	`async def lifespan(_app: FastAPI) -> AsyncGenerator[None, None]:`	Defines `lifespan` warming models once when worker starts.

Line	Source Code	Line-by-Line Explanation
27	`    logger.info("Starting Crop Disease API — loading model and advice KB...")`	Logs startup initiation.
28	`    Base.metadata.create_all(bind=engine)`	Creates `prediction_logs` and `user_feedback` tables automatically if absent.
29	`    inference_service.load()`	Loads `model.onnx`, `plant_gate.onnx`, and `model_meta.json`.
30	`    advice_service.load()`	Loads `configs/advice_kb.json` into RAM.
31	`    logger.info(`	Logs startup readiness and class count.
32	`        "Startup complete: model_loaded=%s, classes=%d",`	Format string.
33	`        inference_service.loaded,`	Loaded status.
34	`        inference_service.num_classes,`	Class count.
35	`    )`	Closes log.
36	`    yield`	Yields control cleanly to serve incoming HTTP traffic.
37	`    logger.info("Shutting down Crop Disease API.")`	Logs clean shutdown upon SIGTERM/Ctrl+C.
38		*Blank visual separator line.*
39		*Blank visual separator line.*
40	`# --- Application factory ---`	Section comment: FastAPI app instantiation.
41	`limiter = Limiter(key_func=get_remote_address, default_limits=[settings.rate_limit])`	Configures per-IP SlowAPI rate limiter (`60/minute` default).
42		*Blank visual separator line.*
43	`app = FastAPI(`	Creates FastAPI application instance with OpenAPI metadata and `lifespan`.
44	`    title=settings.project_name,`	OpenAPI title.
45	`    version="1.0.0",`	API version.
46	`    description=(`	Multi-line OpenAPI description.
47	`        "Upload a crop leaf, fruit, or pest photo and receive a calibrated disease diagnosis "`	Line 1 of description.
48	`        "with Grad-CAM explainability and treatment advice."`	Line 2 of description.
49	`    ),`	Closes tuple.
50	`    lifespan=lifespan,`	Binds async `lifespan`.
51	`)`	Closes `FastAPI(...)`.
52		*Blank visual separator line.*
53	`app.state.limiter = limiter`	Attaches SlowAPI limiter to `app.state`.
54	`app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)`	Registers 429 Too Many Requests exception handler.
55		*Blank visual separator line.*
56	`# --- CORS ---`	Section comment: Cross-Origin Resource Sharing.
57	`app.add_middleware(`	Mounts `CORSMiddleware` permitting browser requests from React Vite/Docker origins.
58	`    CORSMiddleware,`	Middleware class.
59	`    allow_origins=settings.cors_origin_list,`	Explicit whitelist from `.env` (`http://localhost:5173`, etc.).
60	`    allow_credentials=True,`	Allows credentials/headers.
61	`    allow_methods=["*"],`	Allows all HTTP verbs.
62	`    allow_headers=["*"],`	Allows all request headers.
63	`)`	Closes middleware registration.
64		*Blank visual separator line.*
65		*Blank visual separator line.*
66	`# --- Request-ID injection middleware ---`	Section comment: distributed request correlation middleware.
67	`@app.middleware("http")`	Registers HTTP middleware running around every request.
68	`async def inject_request_id(request: Request, call_next):  # type: ignore[no-untyped-def]`	Defines `inject_request_id` async wrapper.
69	`    rid = request.headers.get("X-Request-ID") or generate_request_id()`	Reuses incoming `X-Request-ID` header or generates a random 8-hex-char trace ID.
70	`    request_id_var.set(rid)`	Stores trace ID in Python `contextvars.ContextVar` so logs & `/predict` share it automatically.
71	`    response = await call_next(request)`	Forwards request down to FastAPI route handler.
72	`    response.headers["X-Request-ID"] = rid`	Echoes `X-Request-ID` back in HTTP response headers for debugging.
73	`    return response`	Returns HTTP response.
74		*Blank visual separator line.*
75		*Blank visual separator line.*
76	`# --- Global exception handlers ---`	Section comment: catch-all 500 JSON error formatting.
77	`@app.exception_handler(Exception)`	Registers global `Exception` handler.
78	`async def _unhandled_exception_handler(request: Request, exc: Exception) -> JSONResponse:`	Formats unhandled internal errors cleanly instead of leaking raw HTML tracebacks.
79	`    rid = request_id_var.get("-")`	Reads trace ID from ContextVar.
80	`    logger.error("Unhandled error: %s", exc, exc_info=True)`	Logs full exception stack server-side.
81	`    return JSONResponse(`	Returns structured 500 JSON payload matching `ErrorResponse`.
82	`        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,`	HTTP 500 status.
83	`        content={"error": "Internal Server Error", "detail": str(exc), "request_id": rid},`	JSON content including `request_id`.
84	`    )`	Closes response.
85		*Blank visual separator line.*
86		*Blank visual separator line.*
87	`# --- Routes ---`	Section comment: router mounting.

Line	Source Code	Line-by-Line Explanation
88	`app.include_router(router)`	Mounts `/health`, `/classes`, `/advice/{class_id}`, `/predict`, `/feedback` routes.
89		*Blank visual separator line.*
90		*Blank visual separator line.*
91	`if __name__ == "__main__":`	Checks direct execution (`python -m app.main`).
92	`    import uvicorn`	Imports `uvicorn` ASGI server.
93		*Blank visual separator line.*
94	`    uvicorn.run(`	Starts Uvicorn ASGI loop with host/port/reload bound to `settings`.
95	`        "app.main:app",`	Module import target `'app.main:app'`.
96	`        host=settings.api_host,`	Bind IP.
97	`        port=settings.api_port,`	Bind port.
98	`        reload=settings.app_env == "development",`	Enables auto-reload only in `development` mode.
99	`        log_level=settings.log_level.lower(),`	Lowercase log level.
100	`    )`	Closes `uvicorn.run`.

9.7 TypeScript Typed HTTP Client — frontend/src/api/client.ts

File Summary: Wraps Fetch API calls to `/api` (proxied to FastAPI), decodes structured FastAPI error payloads into typed `ApiError` exceptions, and handles multipart image upload cleanly.

frontend/src/api/client.ts (Lines 1–77)

Line	Source Code	Line-by-Line Explanation
1	`import {`	Imports strict TypeScript interfaces mirroring backend Pydantic schemas.
2	`  ClassListResponse,`	`ClassListResponse` interface.
3	`  FeedbackRequest,`	`FeedbackRequest` interface.
4	`  FeedbackResponse,`	`FeedbackResponse` interface.
5	`  HealthResponse,`	`HealthResponse` interface.
6	`  PredictionResponse,`	`PredictionResponse` interface.
7	`} from "@/types/prediction";`	Closes import.
8		*Blank visual separator line.*
9	`const API_BASE = import.meta.env.VITE_API_BASE_URL || "/api";`	Reads Vite build-time env override `VITE_API_BASE_URL` or defaults to relative `'/api'` (Vite dev proxy / Nginx).
10		*Blank visual separator line.*
11	`export class ApiError extends Error {`	Defines typed `ApiError` extending built-in `Error` with HTTP `status` and parsed `detail`.
12	`  status: number;`	HTTP status code property.
13	`  detail: string;`	Server error message property.
14		*Blank visual separator line.*
15	`  constructor(status: number, detail: string) {`	Constructs `ApiError(status, detail)`.
16	`    super(detail);`	Calls base `Error` constructor.
17	`    this.name = "ApiError";`	Sets error name `'ApiError'`.
18	`    this.status = status;`	Assigns `status`.
19	`    this.detail = detail;`	Assigns `detail`.
20	`  }`	Closes constructor.
21	`}`	Closes class.
22		*Blank visual separator line.*
23	`async function request<T>(endpoint: string, options: RequestInit = {}): Promise<T> {`	Defines generic JSON request helper `request<T>(endpoint, options)`.
24	`  const url = `${API_BASE}${endpoint.startsWith("/") ? endpoint : `/${endpoint}`}`;`	Normalizes leading slash onto `API_BASE`.
25	`  const response = await fetch(url, options);`	Awaits browser `fetch(url, options)`.
26		*Blank visual separator line.*
27	`  if (!response.ok) {`	Handles non-2xx HTTP error status cleanly.
28	`    let detail = `Request failed with status ${response.status}`;`	Default fallback message.
29	`    try {`	Attempts parsing FastAPI `{detail}` or `{error}` JSON payload.
30	`      const err = await response.json();`	Reads JSON response.
31	`      detail = err.detail || err.error || detail;`	Extracts server detail string.
32	`    } catch {`	Catches non-JSON gateway HTML errors silently.
33	`      // Non-JSON response`	Comment noting fallback.
34	`    }`	Closes catch block.
35	`    throw new ApiError(response.status, detail);`	Throws typed `ApiError` caught by React `App.tsx`.
36	`  }`	Closes `if (!response.ok)`.
37		*Blank visual separator line.*
38	`  return response.json();`	Parses and returns strongly-typed JSON payload `Promise<T>`.
39	`}`	Closes `request`.
40		*Blank visual separator line.*
41	`export const api = {`	Exports cohesive `api` object exposing all backend endpoints.

Line	Source Code	Line-by-Line Explanation
42	`  checkHealth: (): Promise<HealthResponse> => request<HealthResponse>("/health"),`	`GET /health` liveness check.
43	``	*Blank visual separator line.*
44	`  getClasses: (): Promise<ClassListResponse> => request<ClassListResponse>("/classes"),`	`GET /classes` canonical label fetch.
45	``	*Blank visual separator line.*
46	`  predict: async (file: File): Promise<PredictionResponse> => {`	Multipart image upload method `predict(file)` (`FormData` , intentionally omitting manual `Content-Type` so browser sets multipart boundary!).
47	`    const formData = new FormData();`	Creates browser `FormData` body.
48	`    formData.append("file", file);`	Appends selected File under `'file'` key matching FastAPI `file: UploadFile` .
49	``	*Blank visual separator line.*
50	`    const url = `${API_BASE}/predict`;`	Builds `/predict` URL.
51	`    const response = await fetch(url, {`	Sends multipart `POST` request.
52	`      method: "POST",`	HTTP `POST` .
53	`      body: formData,`	Multipart form body.
54	`    });`	Closes fetch options.
55	``	*Blank visual separator line.*
56	`    if (!response.ok) {`	Checks for 400/422/429/500/503 failure statuses.
57	`      let detail = `Inference failed (${response.status})`;`	Fallback error message.
58	`      try {`	Tries parsing JSON detail.
59	`        const err = await response.json();`	Reads JSON.
60	`        detail = err.detail || err.error || detail;`	Extracts FastAPI detail.
61	`      } catch {`	Handles non-JSON error body cleanly.
62	`        // Non-JSON`	Comment.
63	`      }`	Closes catch.
64	`      throw new ApiError(response.status, detail);`	Throws `ApiError` for UI alert banner.
65	`    }`	Closes error branch.
66	``	*Blank visual separator line.*
67	`    return response.json();`	Returns `PredictionResponse` JSON.
68	`  },`	Closes `predict` .
69	``	*Blank visual separator line.*
70	`  submitFeedback: (data: FeedbackRequest): Promise<FeedbackResponse> =>`	Submits active-learning correction JSON via `POST /feedback` .
71	`    request<FeedbackResponse>("/feedback", {`	Calls generic helper with `'POST'` JSON options.
72	`      method: "POST",`	HTTP `POST` .
73	`      headers: { "Content-Type": "application/json" },`	Explicit `application/json` header.
74	`      body: JSON.stringify(data),`	Serializes `FeedbackRequest` payload.
75	`    }),`	Closes call.
76	`};`	Closes `api` object.
77	``	*Blank visual separator line.*

9.8 Interactive Grad-CAM Overlay Viewer — frontend/src/components/GradCamViewer.tsx

File Summary: Layers the uploaded crop leaf photo under the server-rendered base64 JET Grad-CAM overlay using CSS `mix-blend-screen` and real-time opacity slider + visibility toggle controls.

frontend/src/components/GradCamViewer.tsx (Lines 1–73)

Line	Source Code	Line-by-Line Explanation
1	`import React, { useState } from "react";`	Imports React and `useState` hook.
2	`import { useTranslation } from "react-i18next";`	Imports `useTranslation` i18next hook for localized UI labels.
3	``	*Blank visual separator line.*
4	`interface GradCamViewerProps {`	Defines TypeScript props interface for `GradCamViewer` .
5	`  originalSrc: string;`	`originalSrc` : browser object/data URL of uploaded crop photo.
6	`  gradcamBase64?: string | null;`	`gradcamBase64` : optional server base64 PNG overlay.
7	`}`	Closes interface.
8	``	*Blank visual separator line.*
9	`export const GradCamViewer: React.FC<GradCamViewerProps> = ({ originalSrc, gradcamBase64 }) => {`	Exports functional React component `GradCamViewer` .
10	`  const { t } = useTranslation();`	Reads translation helper `t` .
11	`  const [showHeatmap, setShowHeatmap] = useState(true);`	Local boolean state toggling Grad-CAM heatmap visibility (default ON).
12	`  const [opacity, setOpacity] = useState(0.5);`	Local opacity state in `[0.1, 1.0]` (default `0.5`).
13	``	*Blank visual separator line.*
14	`  const heatmapSrc = gradcamBase64 ? `${gradcamBase64}` : null;`	Builds inline `...` URI when server returned a Grad-CAM image.
15	``	*Blank visual separator line.*
16	`  return (`	Returns JSX layout.
17	`    <div className="flex flex-col items-center space-y-3">`	Outer flex column wrapper.
18	`      {/* Image container with layered overlay */}`	Comment: square stacked image container.

Line	Source Code	Line-by-Line Explanation
19	`<div className="relative w-full max-w-sm aspect-square rounded-xl overflow-hidden bg-black shadow-md border border-gray-200 dark:border-gray-800">`	Responsive square viewport with dark backdrop, rounded corners, and border.
20	`{/* Original photo */}`	Comment: base uploaded photo layer.
21	`<img`	Renders original uploaded crop leaf image filling container cleanly.
22	`src={originalSrc}`	Object/data image URL.
23	`alt="Uploaded crop"`	Accessible alt text.
24	`className="absolute inset-0 w-full h-full object-cover"`	Full-bleed absolute `object-cover` positioning.
25	`/>`	Closes base `<img>`.
26		*Blank visual separator line.*
27	`{/* Grad-CAM heatmap overlay */}`	Comment: conditional Grad-CAM layer.
28	`{heatmapSrc && showHeatmap && (`	Mounts heatmap overlay only when server provided one AND toggle is checked.
29	`<img`	Renders Grad-CAM PNG stacked directly atop the photo.
30	`src={heatmapSrc}`	Data-URI PNG source.
31	`alt="Grad-CAM attention heatmap"`	Accessible alt text.
32	`className="absolute inset-0 w-full h-full object-cover mix-blend-screen transition-opacity duration-150"`	Applies CSS `mix-blend-screen` blending luminous warm hotspots cleanly over leaves + smooth opacity transition.
33	`style={{ opacity }}`	Binds live inline `opacity` slider state.
34	`/>`	Closes overlay `<img>`.
35	`)}`	Closes conditional block.
36	`</div>`	Closes image viewport container.
37		*Blank visual separator line.*
38	`{/* Heatmap controls */}`	Comment: interactive heatmap control panel.
39	`{heatmapSrc && (`	Renders toggle + opacity slider card when a heatmap exists.
40	`<div className="w-full max-w-sm bg-gray-50 dark:bg-gray-800/60 p-3 rounded-xl border border-gray-200 dark:border-gray-700 space-y-2">`	Styled control container supporting dark mode.
41	`<div className="flex items-center justify-between text-xs font-medium">`	Top row aligning checkbox toggle and percentage readout.
42	`<label className="flex items-center space-x-2 cursor-pointer select-none">`	Clickable label wrapping checkbox.
43	`<input`	Checkbox controlling `showHeatmap` state.
44	`type="checkbox"`	Checkbox type.
45	`checked={showHeatmap}`	Controlled checked prop.
46	`onChange={(e) => setShowHeatmap(e.target.checked)}`	Updates visibility state on click.
47	`className="rounded border-gray-300 text-brand-600 focus:ring-brand-500"`	Tailwind brand styling.
48	`/>`	Closes `<input>`.
49	`<span>{t("result.heatmapToggle")}</span>`	Localized toggle label (`Grad-CAM Heatmap`).
50	`</label>`	Closes `<label>`.
51	`<span className="text-gray-500 dark:text-gray-400">`	Right-aligned live opacity percentage readout.
52	`{Math.round(opacity * 100)}%`	Formats `0.5 -> 50%`.
53	`</span>`	Closes `<span>`.
54	`</div>`	Closes header flex row.
55		*Blank visual separator line.*
56	`{showHeatmap && (`	Shows range slider only while heatmap is toggled ON.
57	`<input`	HTML5 range input slider.
58	`type="range"`	Range type.
59	`min="0.1"`	Minimum opacity `0.1`.
60	`max="1.0"`	Maximum opacity `1.0`.
61	`step="0.05"`	Step increment `0.05`.
62	`value={opacity}`	Controlled opacity value.
63	`onChange={(e) => setOpacity(parseFloat(e.target.value))}`	Parses slider string value to float state.
64	`aria-label={t("result.heatmapOpacity")}`	Accessible screen-reader label.
65	`className="w-full accent-brand-600 h-1.5 bg-gray-200 dark:bg-gray-700 rounded-lg cursor-pointer"`	Full-width styled slider bar.
66	`/>`	Closes `<input>`.
67	`)}`	Closes slider conditional.
68	`</div>`	Closes controls card.
69	`)}`	Closes overall heatmap controls conditional.
70	`</div>`	Closes root wrapper.
71	`);`	Closes return JSX.
72	`};`	Closes component definition.
73		*Blank visual separator line.*

9.9 Root React Application Orchestrator — frontend/src/App.tsx

File Summary: Coordinates dark mode, persistent local diagnosis history, `/classes` prefetching, async `/predict` upload flow, error handling, responsive 2-column result dashboard, and history drawer.

frontend/src/App.tsx (Lines 1–145)

Line	Source Code	Line-by-Line Explanation
1	`import React, { useEffect, useState } from "react";`	Imports React, `useEffect`, and `useState`.
2	`import { Header } from "@/components/Header";`	Imports `Header` navigation bar.
3	`import { UploadZone } from "@/components/UploadZone";`	Imports drag-and-drop/camera `UploadZone`.
4	`import { GradCamViewer } from "@/components/GradCamViewer";`	Imports layered `GradCamViewer`.
5	`import { ResultCard } from "@/components/ResultCard";`	Imports calibrated prediction + Top-3 `ResultCard`.
6	`import { AdviceCard } from "@/components/AdviceCard";`	Imports treatment & prevention `AdviceCard`.
7	`import { FeedbackWidget } from "@/components/FeedbackWidget";`	Imports active-learning `FeedbackWidget`.
8	`import { HistoryDrawer } from "@/components/HistoryDrawer";`	Imports slide-over `HistoryDrawer`.
9	`import { useDarkMode } from "@/hooks/useDarkMode";`	Imports custom `useDarkMode` localStorage hook.
10	`import { usePredictionHistory } from "@/hooks/usePredictionHistory";`	Imports custom `usePredictionHistory` localStorage hook.
11	`import { api, ApiError } from "@/api/client";`	Imports typed `api` client and `ApiError`.
12	`import { PredictionResponse, HistoryItem } from "@/types/prediction";`	Imports TypeScript domain interfaces.
13		*Blank visual separator line.*
14	`export const App: React.FC = () => {`	Defines root `App` component.
15	`  const { isDark, toggle: toggleDark } = useDarkMode();`	Initializes theme toggle hook.
16	`  const { history, addEntry, clearHistory } = usePredictionHistory();`	Initializes diagnosis history state + helpers.
17		*Blank visual separator line.*
18	`  const [isHistoryOpen, setIsHistoryOpen] = useState(false);`	Drawer open/closed toggle state.
19	`  const [isLoading, setIsLoading] = useState(false);`	Spinner loading state during upload/inference.
20	`  const [errorMessage, setErrorMessage] = useState<string | null>(null);`	Optional user-facing API/validation error banner message.
21		*Blank visual separator line.*
22	`  const [currentImageSrc, setCurrentImageSrc] = useState<string | null>(null);`	Stores currently displayed crop image preview URL.
23	`  const [currentPrediction, setCurrentPrediction] = useState<PredictionResponse | null>(null);`	Stores active `PredictionResponse` payload.
24	`  const [availableClasses, setAvailableClasses] = useState<string[]>([]);`	Stores canonical classes for the correction dropdown.
25		*Blank visual separator line.*
26	`  // Fetch available classes once on load for the feedback dropdown`	Comment: one-time class list fetch.
27	`  useEffect(() => {`	`useEffect` with empty `[]` dependency array running once on mount.
28	`    api`	Calls `api.getClasses()`.
29	`      .getClasses()`	GET `/classes`.
30	`      .then((res) => setAvailableClasses(res.classes))`	Populates `availableClasses` state.
31	`      .catch((err) => console.warn("Could not fetch classes list:", err));`	Logs non-fatal warning if backend is offline.
32	`  }, []);`	Closes `useEffect`.
33		*Blank visual separator line.*
34	`  const handleImageSelected = async (file: File) => {`	Defines async `handleImageSelected(file)` callback passed to `UploadZone`.
35	`    setErrorMessage(null);`	Clears previous error alerts.
36	`    setIsLoading(true);`	Activates loading spinner.
37		*Blank visual separator line.*
38	`    // Create local object URL for preview`	Comment: instant zero-latency local preview.
39	`    const previewUrl = URL.createObjectURL(file);`	Creates temporary browser blob URL for instant image preview before network response arrives.
40	`    setCurrentImageSrc(previewUrl);`	Updates preview state.
41		*Blank visual separator line.*
42	`    try {`	Starts try-block around `/predict` network call.
43	`      const pred = await api.predict(file);`	Uploads image to `POST /predict` and awaits JSON response.
44	`      setCurrentPrediction(pred);`	Stores received `PredictionResponse`.
45		*Blank visual separator line.*
46	`      // Create a small data URL thumbnail for history`	Comment: create persistent base64 thumbnail for `localStorage` history.
47	`      const reader = new FileReader();`	Instantiates `FileReader`.
48	`      reader.onload = () => {`	Saves `(pred, dataURL)` into persistent diagnosis history once read completes.
49	`        addEntry(pred, reader.result as string);`	Appends history item.
50	`      };`	Closes onload handler.
51	`      reader.readAsDataURL(file);`	Triggers asynchronous base64 DataURL encoding of file.
52	`    } catch (err) {`	Catches HTTP or network errors.
53	`      if (err instanceof ApiError) {`	Checks for structured `ApiError` (e.g. `400 Unsupported format`, `429 Rate limit`).
54	`        setErrorMessage(err.detail);`	Displays exact FastAPI error message.
55	`      } else {`	Generic fallback for offline server/network drop.
56	`        setErrorMessage("Upload or prediction failed. Please try again.");`	Displays generic failure message.
57	`      }`	Closes `if/else`.
58	`      setCurrentPrediction(null);`	Clears stale prediction card.
59	`    } finally {`	Finally block ensures spinner always stops.
60	`      setIsLoading(false);`	Sets `isLoading` back to `false`.

Line	Source Code	Line-by-Line Explanation
61	`    }`	Closes `try/catch/finally`.
62	`  };`	Closes `handleImageSelected`.
63		*Blank visual separator line.*
64	`  const handleSelectHistory = (item: HistoryItem) => {`	Defines `handleSelectHistory(item)` restoring a past diagnosis when clicked in drawer.
65	`    setCurrentImageSrc(item.thumbnail);`	Restores thumbnail image.
66	`    setCurrentPrediction(item.prediction);`	Restores full stored prediction & advice.
67	`    setErrorMessage(null);`	Clears active error banner.
68	`  };`	Closes handler.
69		*Blank visual separator line.*
70	`  const isHealthy =`	Derives boolean `isHealthy` styling flag from predicted label.
71	`    currentPrediction?.label.toLowerCase().includes("healthy") ?? false;`	Case-insensitive `'healthy'` substring check.
72		*Blank visual separator line.*
73	`  return (`	Returns root responsive application layout.
74	`    <div className="min-h-screen flex flex-col bg-gray-50 text-gray-900 dark:bg-gray-950 dark:text-gray-100 transition-colors">`	Full-height flex shell with Tailwind dark/light surface transition.
75	`      <Header`	Top `Header` bar.
76	`        isDark={isDark}`	Dark mode flag.
77	`        onToggleDark={toggleDark}`	Theme toggle callback.
78	`        onOpenHistory={() => setIsHistoryOpen(true)}`	Opens slide-over history drawer.
79	`        historyCount={history.length}`	Displays history badge count.
80	`      />`	Closes `<Header>`.
81		*Blank visual separator line.*
82	`      <main className="flex-1 max-w-4xl w-full mx-auto p-4 sm:p-6 space-y-6">`	Centered main workspace container (`max-w-4xl`).
83	`        {/* Upload Zone */}`	Comment: upload dropzone.
84	`        <UploadZone onImageSelected={handleImageSelected} isLoading={isLoading} />`	Renders interactive `UploadZone`.
85		*Blank visual separator line.*
86	`        {/* Error Alert */}`	Comment: dismissible error alert banner.
87	`        {errorMessage && (`	Conditionally renders error box when `errorMessage` is set.
88	`          <div className="p-4 rounded-xl bg-rose-50 dark:bg-rose-950/40 border border-rose-200 dark:border-rose-800 text-rose-800 dark:text-rose-300 text-xs sm:text-sm font-medium flex items-center justify-between">`	Tailwind rose alert container.
89	`            <span>❌ {errorMessage}</span>`	Displays error string.
90	`            <button`	Dismiss button resetting `errorMessage` to `null`.
91	`              onClick={() => setErrorMessage(null)}`	Click handler.
92	`              className="text-rose-500 hover:text-rose-700 text-xs px-2"`	Button styling.
93	`            >`	Close icon character.
94	`              ×`	Closes button.
95	`            </button>`	Closes alert.
96	`          </div>`	Closes error conditional.
97	`        )}`	Closes error section.
98		*Blank visual separator line.*
99	`        {/* Prediction Results & Grad-CAM */}`	Comment: 2-column responsive results workspace.
100	`        {currentPrediction && currentImageSrc && (`	Renders grid once both `currentPrediction` and `currentImageSrc` exist.
101	`          <div className="grid grid-cols-1 md:grid-cols-2 gap-6 items-start">`	2-column grid on desktop (`md:grid-cols-2`), stacked 1-column on mobile.
102	`            {/* Visual Column: Original + Grad-CAM Heatmap Overlay */}`	Comment: left column visuals.
103	`            <div className="space-y-4">`	Left column stack.
104	`              <GradCamViewer`	Renders layered original photo + interactive Grad-CAM viewer.
105	`                originalSrc={currentImageSrc}`	Preview source.
106	`                gradcamBase64={currentPrediction.gradcam}`	Base64 heatmap.
107	`              />`	Closes `<GradCamViewer>`.
108	`              <FeedbackWidget`	Renders confirm/correct active-learning `FeedbackWidget`.
109	`                requestId={currentPrediction.request_id}`	Trace ID.
110	`                predictedLabel={currentPrediction.label}`	Predicted label.
111	`                availableClasses={availableClasses}`	Dropdown class choices.
112	`              />`	Closes `<FeedbackWidget>`.
113	`            </div>`	Closes left column.
114		*Blank visual separator line.*
115	`            {/* Info Column: Diagnosis Card + Treatment Advice */}`	Comment: right column diagnosis & agronomy.
116	`            <div className="space-y-4">`	Right column stack.
117	`              <ResultCard prediction={currentPrediction} />`	Renders `ResultCard` (Top-1, warning/OOD banner, Top-3 bars, latency).
118	`              <AdviceCard`	Renders `AdviceCard` (severity badge, symptoms, organic/chemical/prevention tabs).
119	`                advice={currentPrediction.advice}`	Structured advice object.
120	`                isHealthy={isHealthy}`	Healthy green-theme toggle.
121	`              />`	Closes `<AdviceCard>`.

Line	Source Code	Line-by-Line Explanation
122	`            </div>`	Closes right column.
123	`          </div>`	Closes responsive grid.
124	`        )}`	Closes result conditional.
125	`      </main>`	Closes `<main>`.
126		*Blank visual separator line.*
127	`      {/* History Slide-over Drawer */}`	Comment: slide-over diagnosis history drawer.
128	`      <HistoryDrawer`	Mounts `HistoryDrawer` overlay.
129	`        isOpen={isHistoryOpen}`	Open state.
130	`        onClose={() => setIsHistoryOpen(false)}`	Close callback.
131	`        history={history}`	Stored diagnoses array.
132	`        onSelect={handleSelectHistory}`	Load-item callback.
133	`        onClear={clearHistory}`	Clear-all callback.
134	`      />`	Closes `<HistoryDrawer>`.
135		*Blank visual separator line.*
136	`      {/* Footer */}`	Comment: stack attribution footer.
137	`      <footer className="border-t border-gray-200 dark:border-gray-800 py-4 text-center text-xs text-gray-400">`	Minimal footer bar.
138	`        Crop Disease & Pest Classifier • PyTorch + timm + ONNX Runtime + FastAPI + React`	Displays core ML/API/UI stack summary.
139	`      </footer>`	Closes footer.
140	`    </div>`	Closes root container.
141	`  );`	Closes JSX return.
142	`};`	Closes `App` component.
143		*Blank visual separator line.*
144	`export default App;`	Default export consumed by `main.tsx`.
145		*Blank visual separator line.*